

Model Parallelism in LLM Training

Autumn 2025

Lecturer: Yuedong (Steven) Xu

Shenzhen Loop Area Institute

yuedongxu@slai.edu.cn

Fudan University

ydxu@fudan.edu.cn

Disclaimer

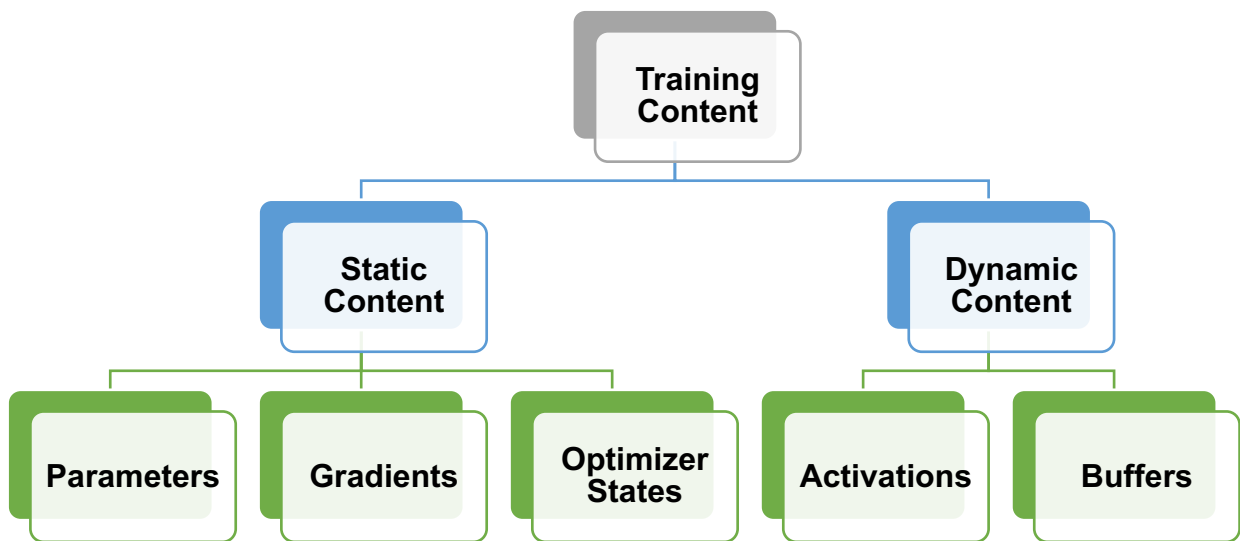
Machine learning systems is a broad and rapidly evolving field. The course material has been developed using a broad spectrum of resources, including research papers, lecture slides, blogposts, research talks, tutorial videos, and other materials shared by the research community.

Distributed LLM Training: Outline

- Data Parallelism
 - Parameter-Server
 - All-Reduce
 - Memory Optimization
- Model Parallelism
 - **Pipeline Parallelism**
 - Tensor Parallelism
 - Sequence Parallelism
- Mixture of Experts

Pipeline Training

- Retrospect: GPU HBM Content During Training



- An example of GPT-3 175B

- **Optimizer States**

- 32-bit Parameter (700 GB)
- Adam Moment (700 GB)
- Adam Variance (700 GB)

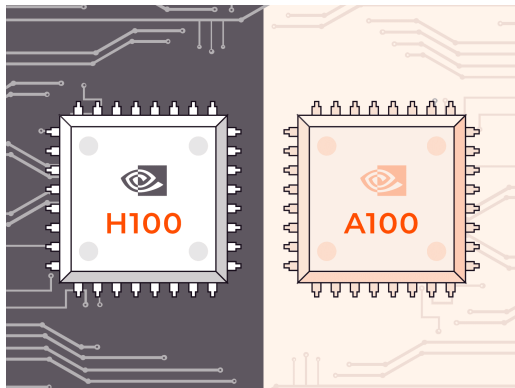
- **16-bit Parameter (350 GB)**

- **16-bit Gradient (350 GB)**

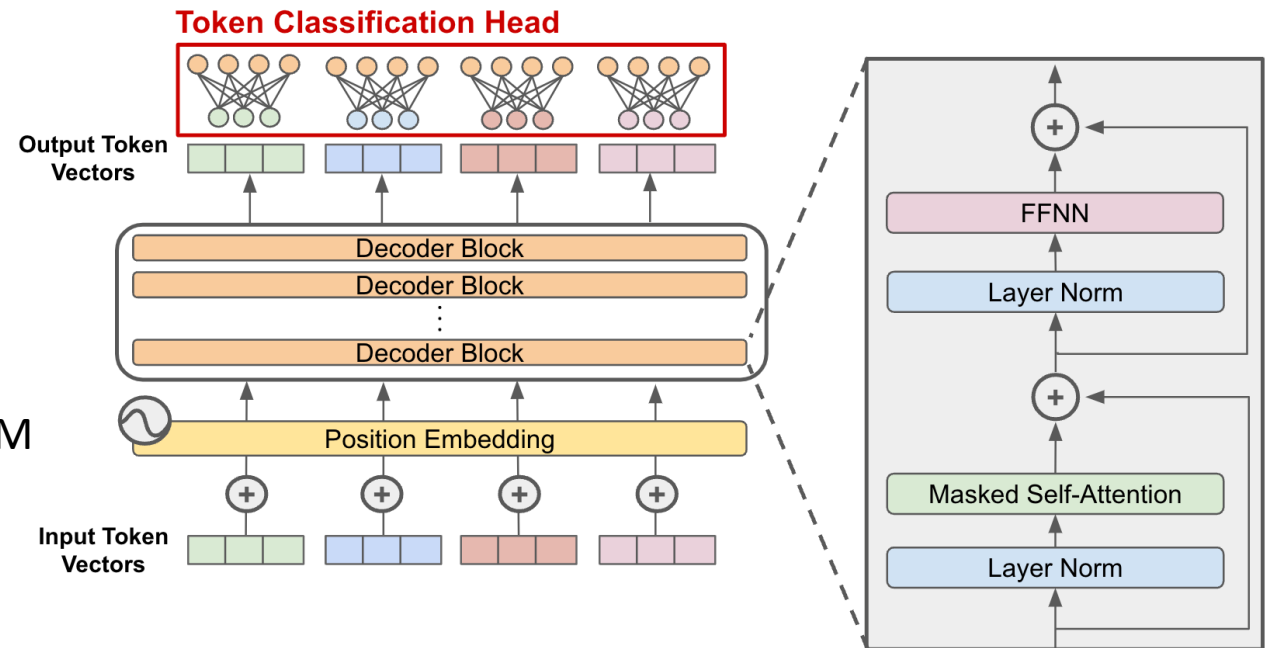
- **Activations (depending on batch size)**

- Buffer and Fragmentation

Pipeline Training

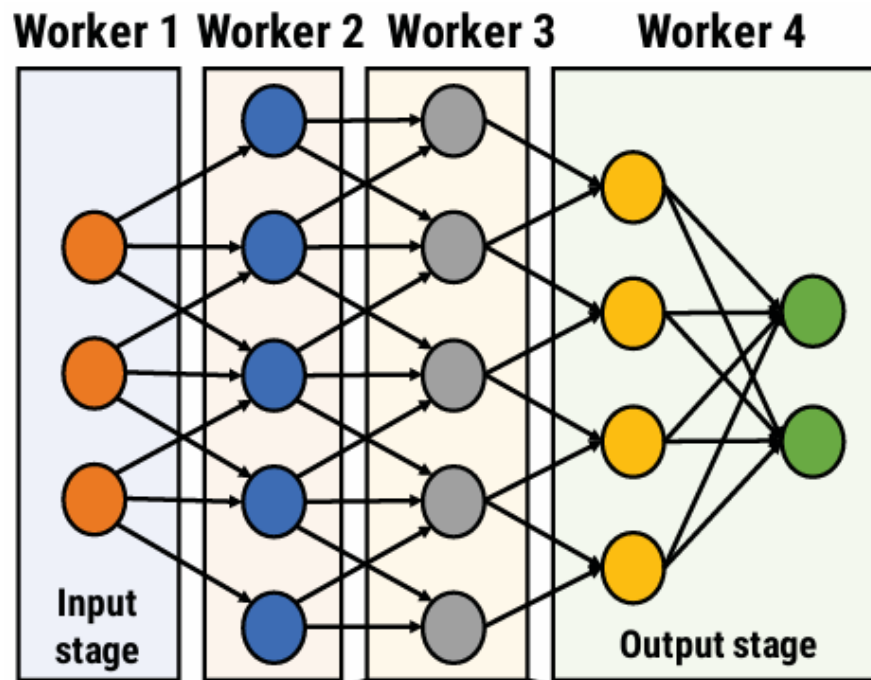


Insufficient HBM

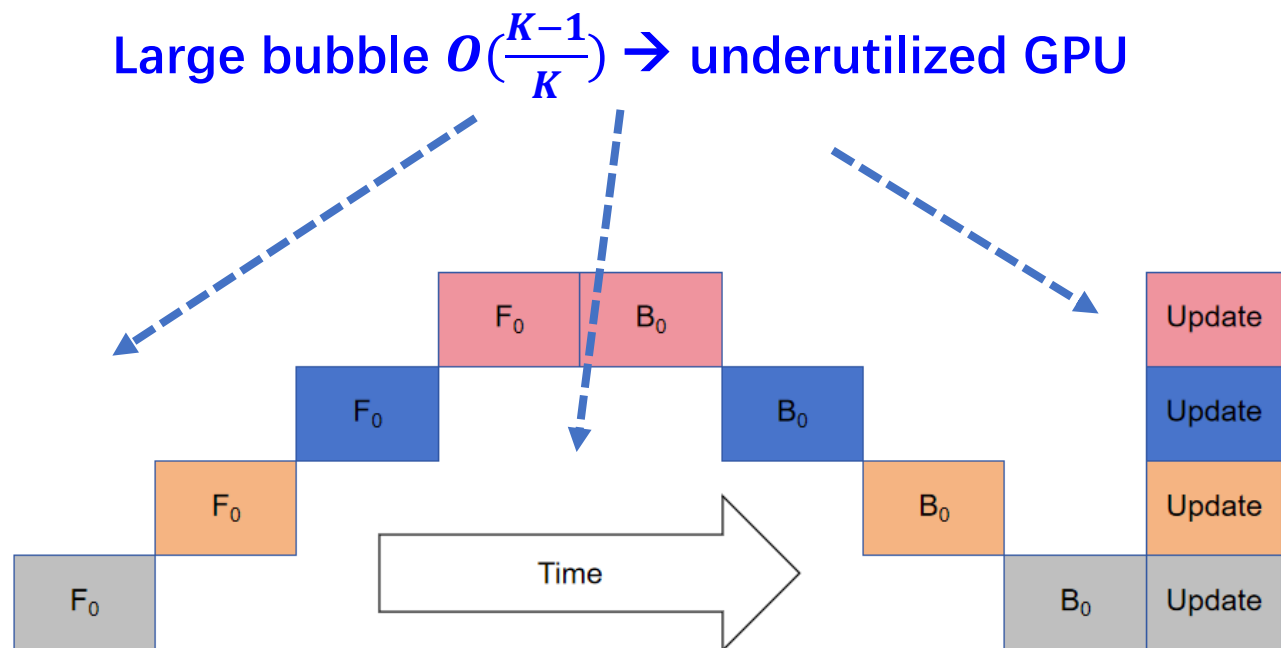


When LLM model is too LARGE!

Pipeline Training

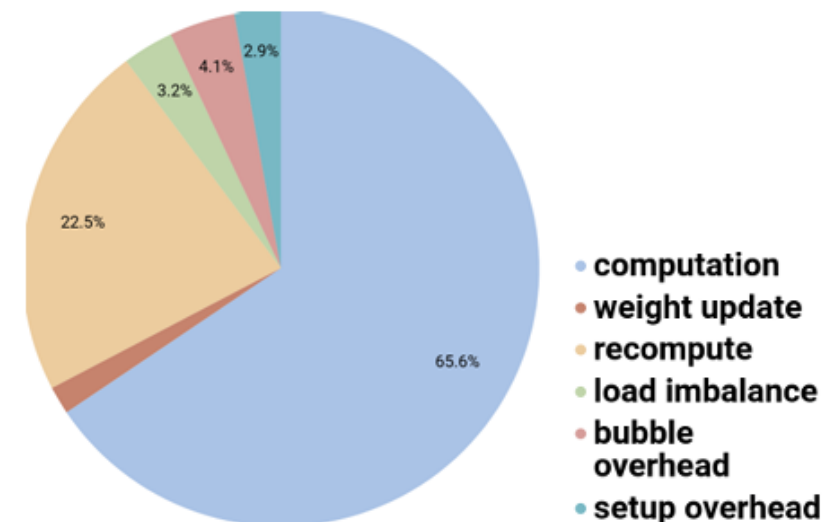
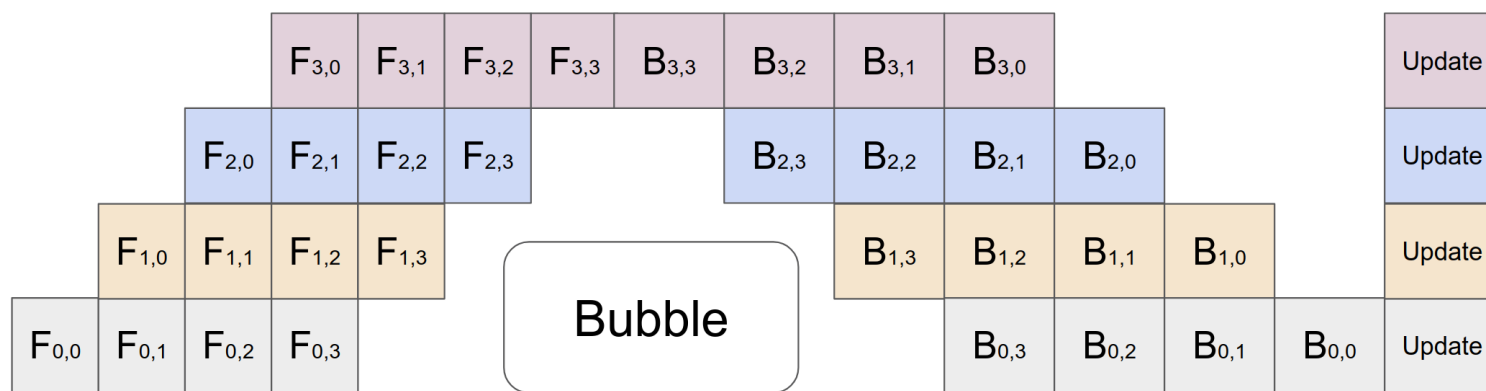


Layerwise partitioning on multiple GPUs



Naïve model parallelism
(F for forward; B for backward)

Pipeline Training



b: batch size

b_m: microbatch size

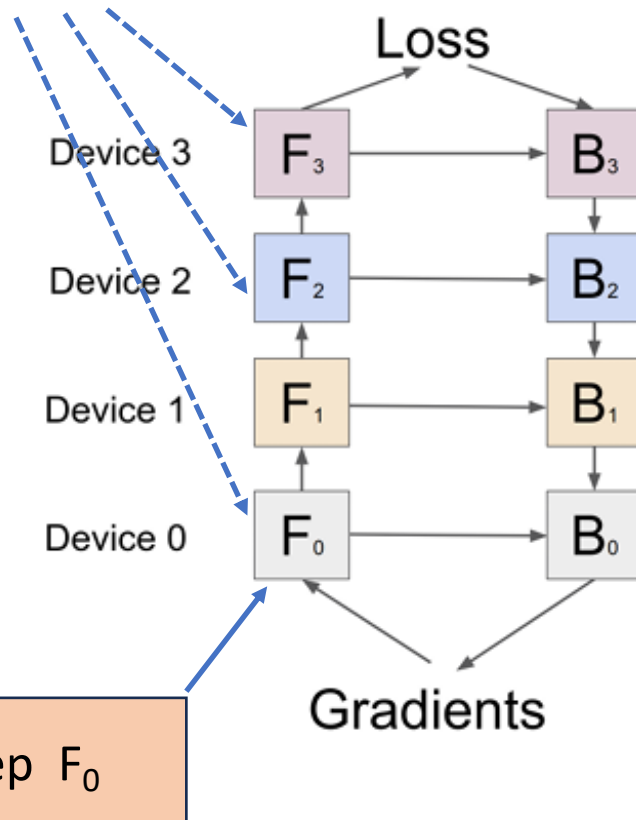
K: # pipeline stages

M: # microbatches, $b = b_m * M$

Bubble ratio: $O(\frac{K-1}{K+M-1})$

Pipeline Training

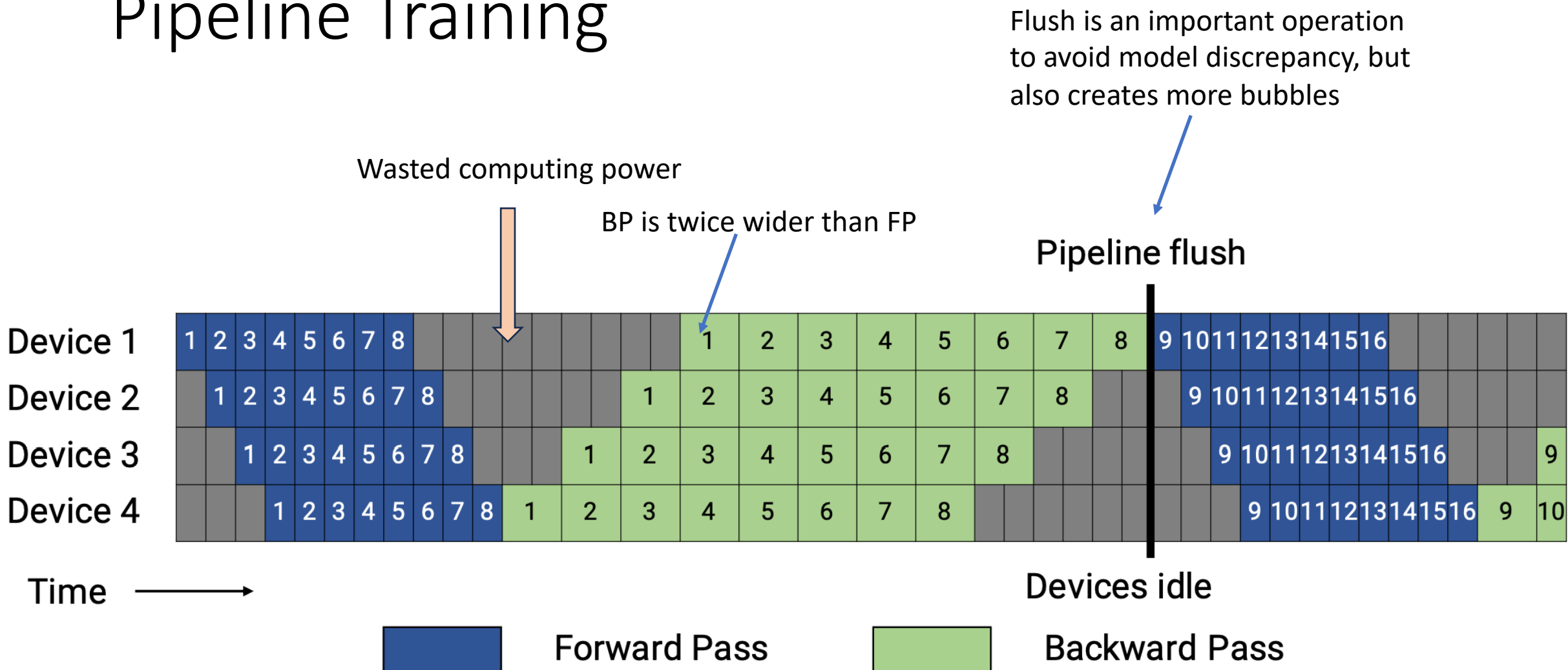
Cannot be discarded before BP



- Computation time
 - Forward time is half of backward time (why?)
- Communication load
 - Intermediate activation between two partitions
- Memory occupation
 - First stage usually needs more memory: $K * M$
 - Re-materialization (re-computation) Further reducing memory consumption

F-then-B: Forward then Backward

Pipeline Training



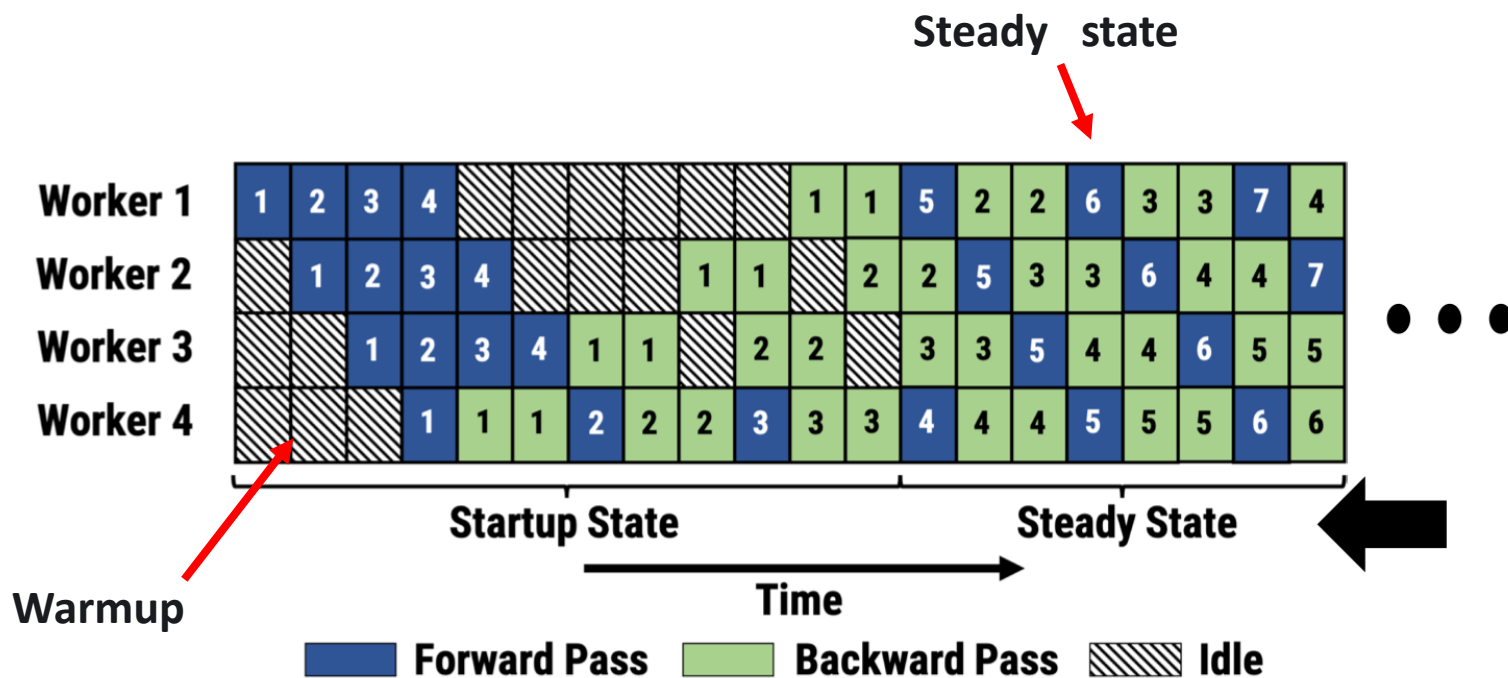
Frequent pipeline flushes lead to increased idle time

Pipeline Training

Orchestrating execution orders of forward and backward passes for microbatch so as to reduce bubbles and peak memory footprints?

- Some KEY concepts
 - Synchronous pipeline versus asynchronous pipeline
 - Interleaved versus non-interleaved

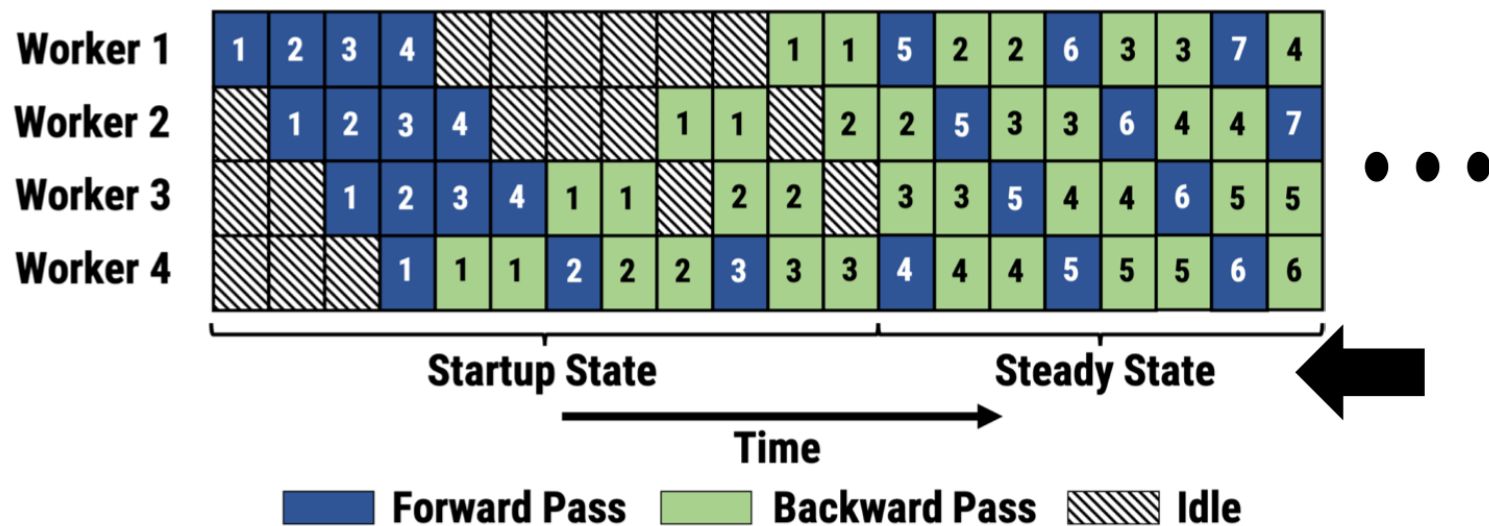
Pipeline Training



1F1B: One Forward then One Backward

- Minimizing lingering time of activations
 - Immediately executing BP after FP for the first microbatch at the output stage
- At steady state
 - Strictly alternative Forward and Backward operations
- Asynchronous weights

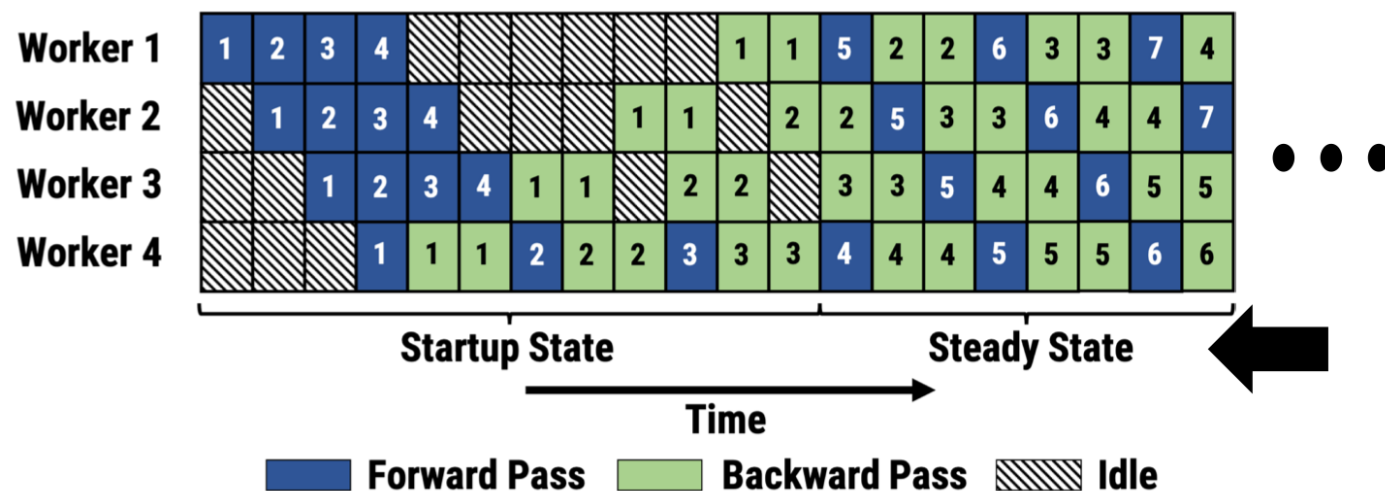
Pipeline Training



1F1B: One Forward then One Backward

- Challenges
 - How to split a DNN model into different stages?
 - LOAD BALANCING** across stages
 - How to update parameters?
After backward pass, the gradient can be used **immediately to update the parameter**

Pipeline Training

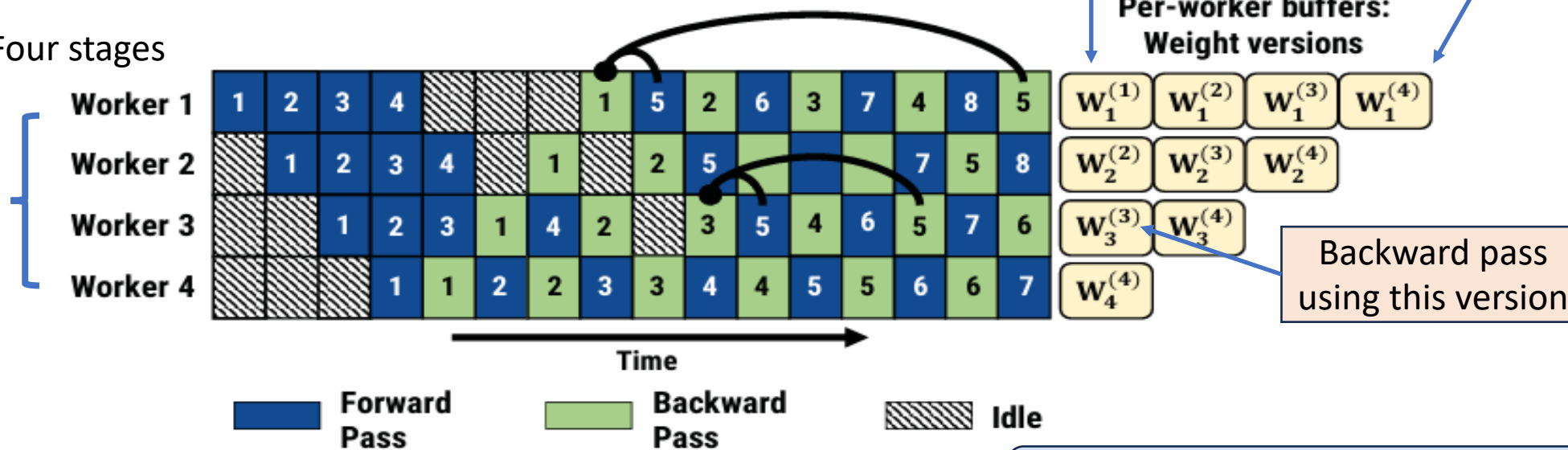


- Observations in stage 1
 - The forward pass of microbatch 5 is performed after the update of **microbatch 1**
 - The backward pass of microbatch 5 is performed after the updates of **microbatches 2, 3 and 4**

Forward and backward passes operate on different versions of parameters!

Pipeline Training

Four stages



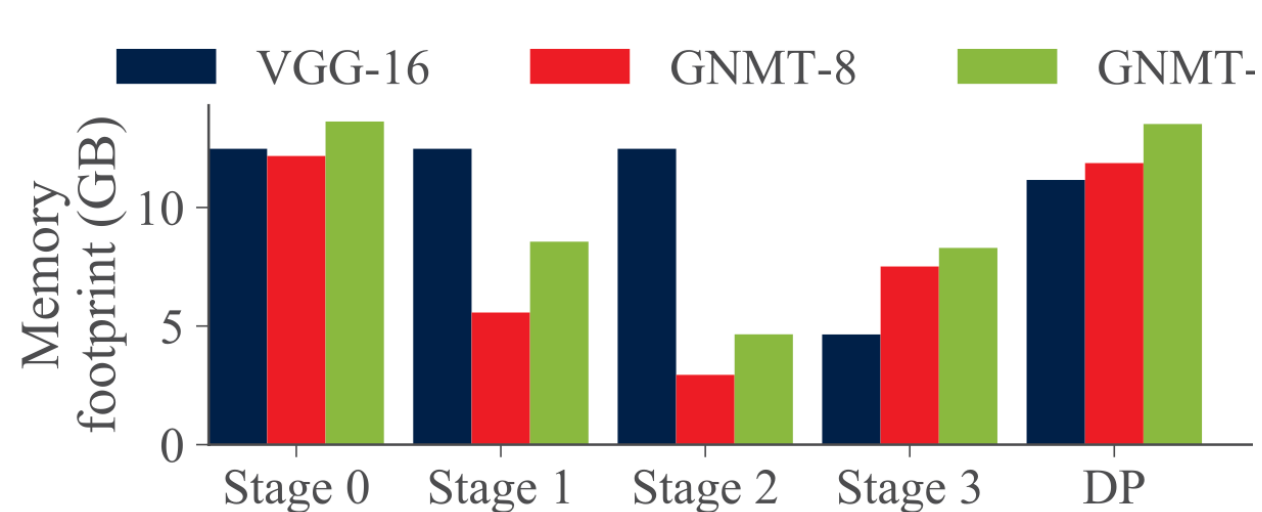
No guarantees across stages!

- **Weight stashing** of PipeDream

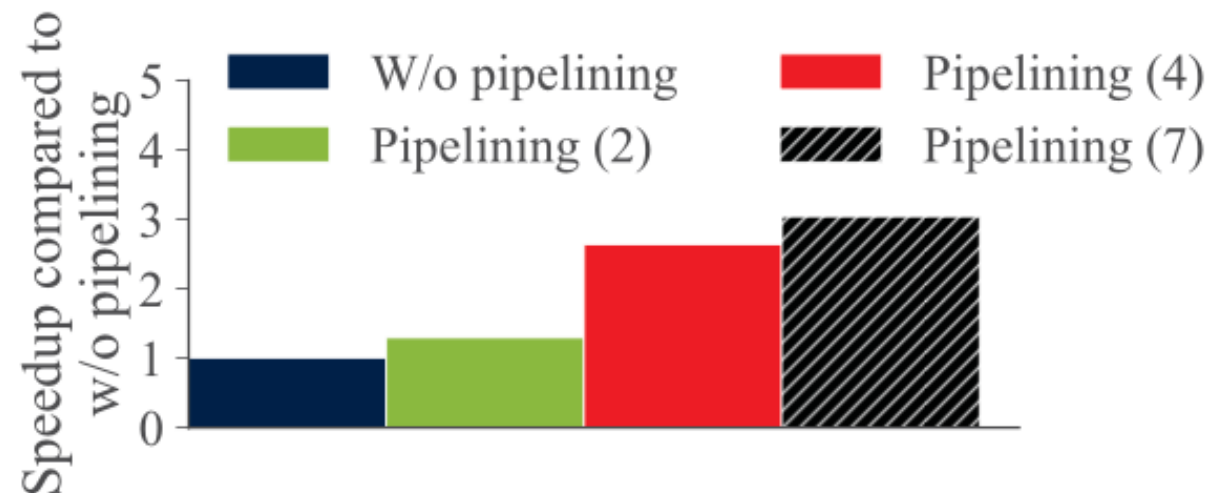
- keeps multiple versions of model weights—one for each active minibatch.
- Forward pass: each pipeline stage uses the most recent weight version available. Once the forward pass for a minibatch is completed, the corresponding weight version is stored.
- Backward pass: the same version is reused to compute gradients and update weights, ensuring that both passes for a given minibatch use identical parameters within a stage.

Pipeline Training

- Experiments



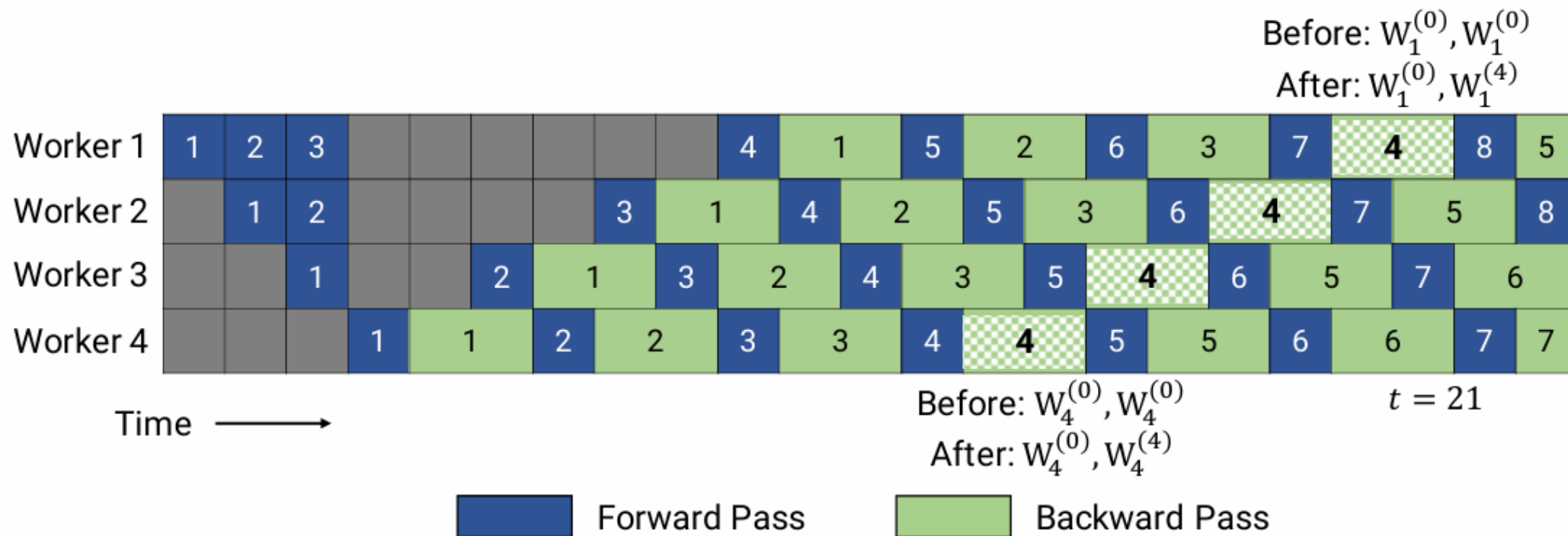
Memory footprint of PipeDream



Speedup ratios on 4 V100 GPUs

PipeDream still saves memory by trading multiple versions of parameters for large activations

Pipeline Training



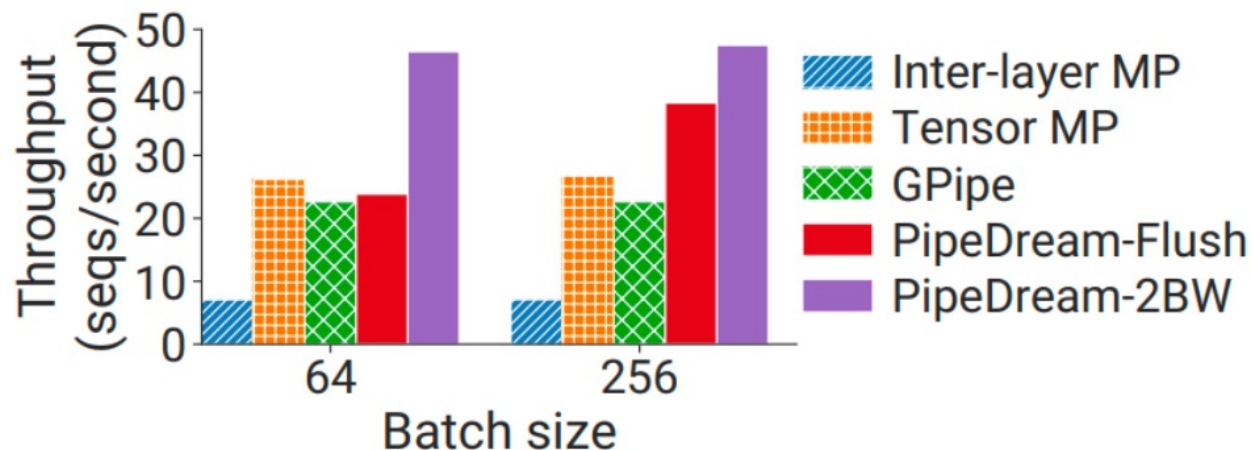
PipeDream-2BW: (1) less frequent flushing than GPipe (2) keeping no more than two model versions.

Think why? How does it relate to pipeline depth K and number of microbranches M

Pipeline Training



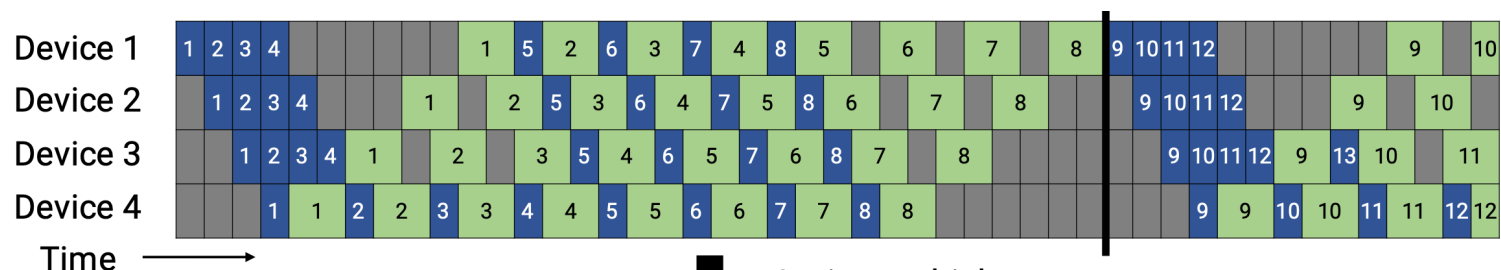
- Flushing causes more bubbles
 - Warmup + Colddown
- Flushing needs less memory



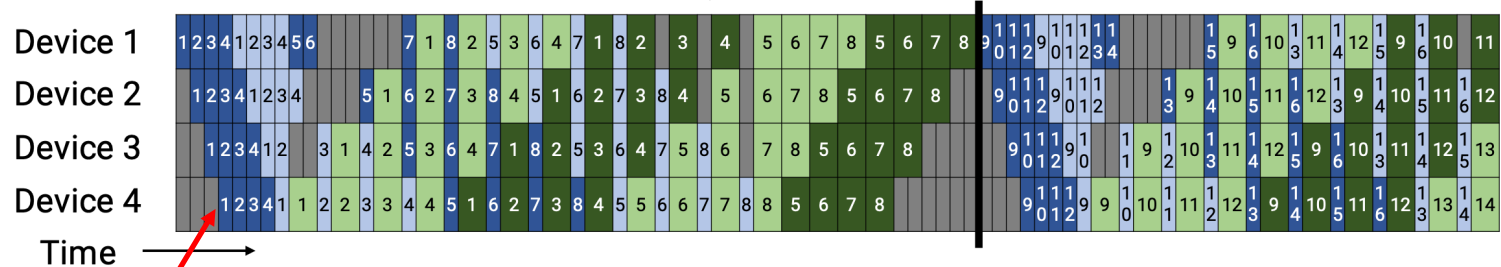
- Training BERT 2.2B with 8-way model parallelism (8 V100 GPU)
- PipeDream is much better than vanilla GPipe

Pipeline Training

- IFIB integrated in Megatron-LM later on



Assign multiple stages to each device



Forward Pass



Backward Pass

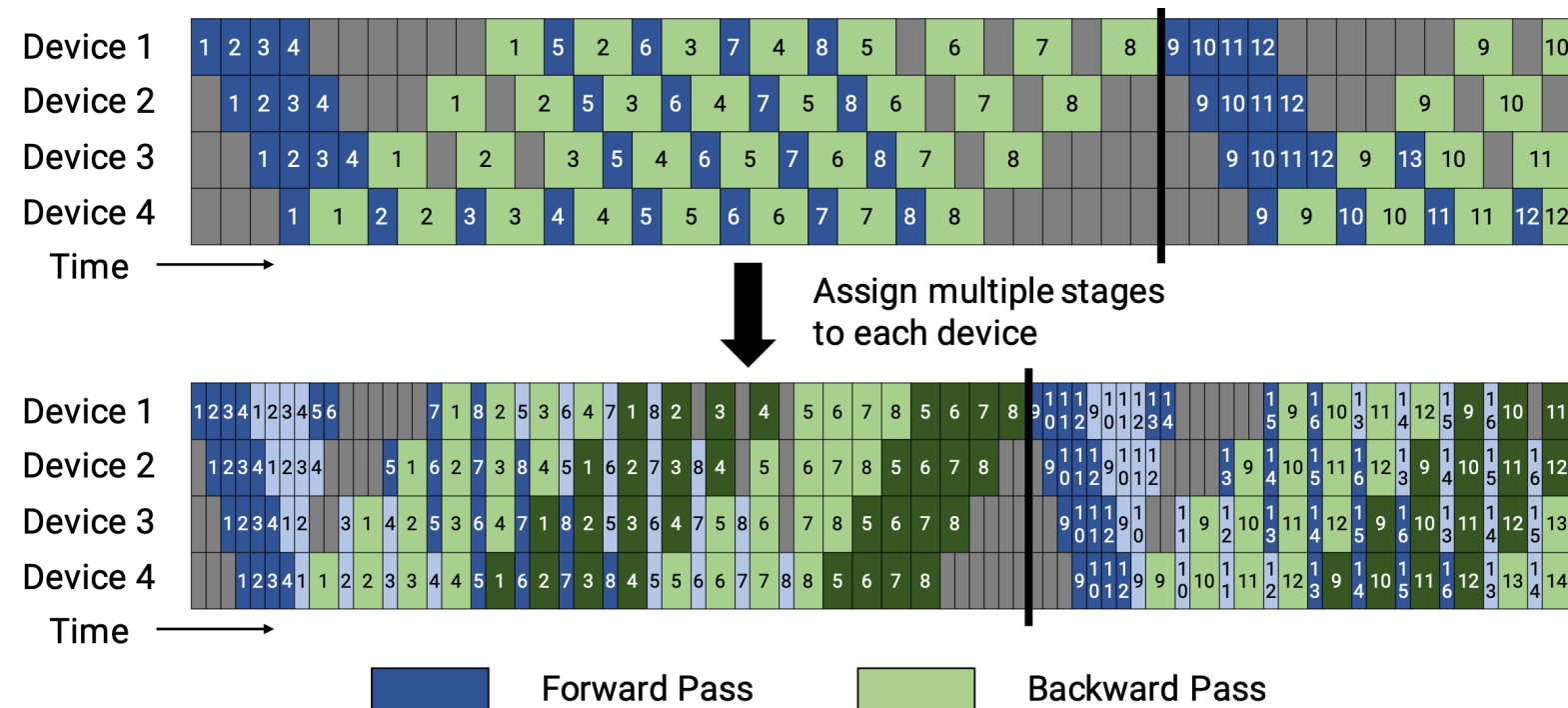
Interleaved 1F1B

(Stages: 4, Layers: 16, each device assigned with 2 chunks. Layer 0-1, 8-9 on Device 1;
Layer 2-3, 10-11 on Device 2; Layer 4-5, 12-13 on Device 3 and Layer 6-7, 14-15 on Device4)

- Key ides:
 - Further dividing each microbatch into even smaller chunks
 - Microbatch and small chunks execute alternatively
- Advantages/disadvantages
 - Smaller bubble
 - Higher inter-state traffic load

Pipeline Training

- IFIB integrated in Megatron-LM later on



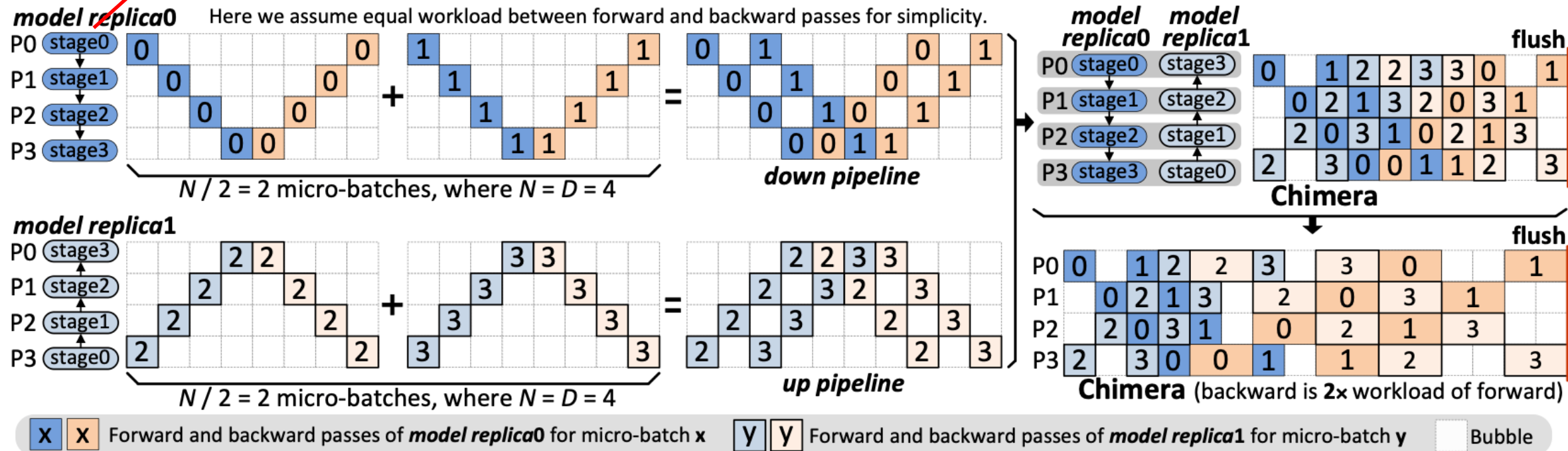
Model placement:

- Stages: 4, Layers: 16, each device assigned with 2 chunks
- Layer 0-1, 8-9 on Device 1
- Layer 2-3, 10-11 on Device 2
- Layer 4-5, 12-13 on Device 3
- Layer 6-7, 14-15 on Device 4

Be aware of the communication scheduling!

Pipeline Training

Storing the first partition of model0 and the fourth partition of model1

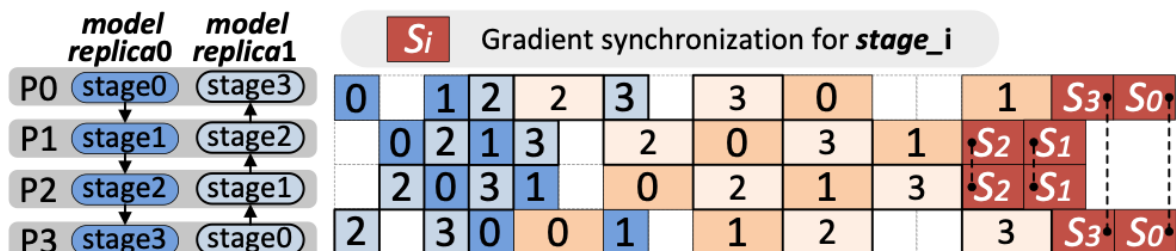


Chimera: double pipeline

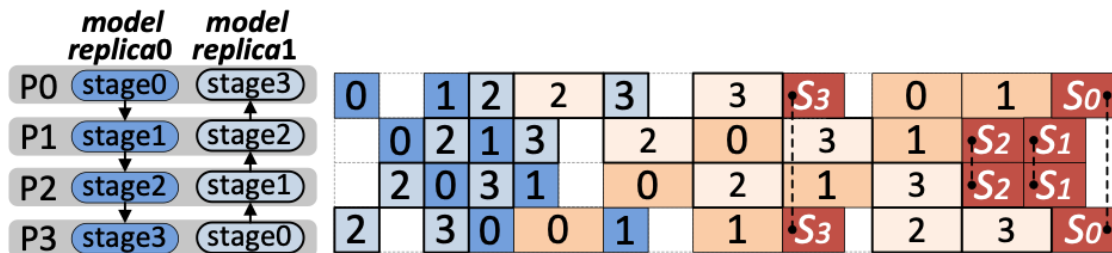
- Smaller bubbles yet doubled memory occupation for model replicas

Pipeline Training

- Orchestrating bubble mitigation and data-parallel synchronization

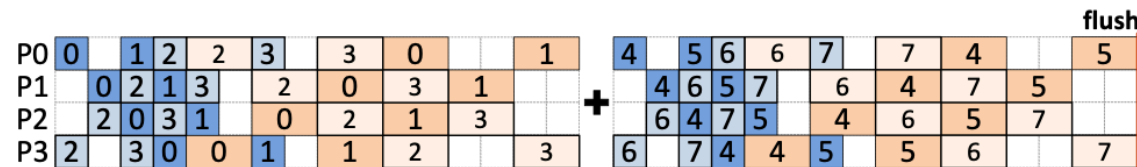


(a) Gradient synchronization after all local computation is finished

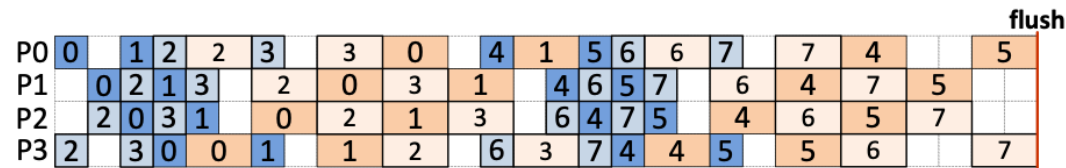


(b) Eager gradient synchronization for deeper overlapping

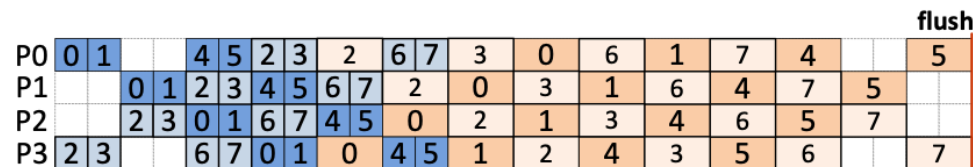
Hide the gradient synchronization overhead by overlapping



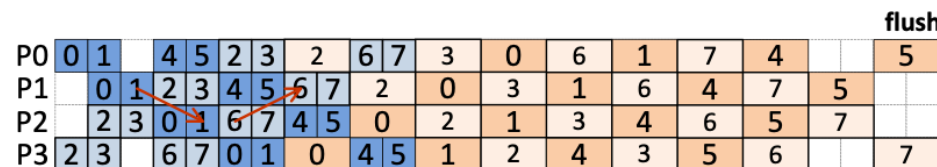
(a) $N=2D$ micro-batches, where $D=4$



(b) *Direct* concatenation (intermediate bubbles)



(c) Concatenation with forward *doubling* (no intermediate bubbles)

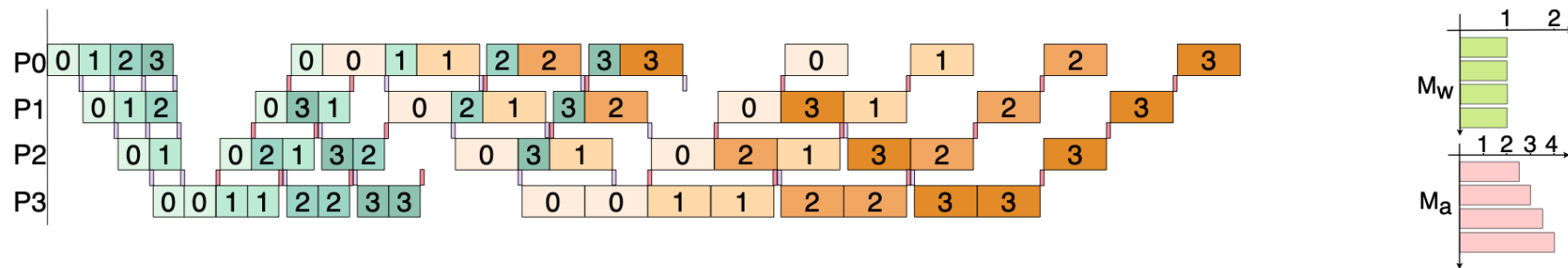


→ $p2p$ (d) The final schedule of forward *doubling* after removing half bubbles at the beginning

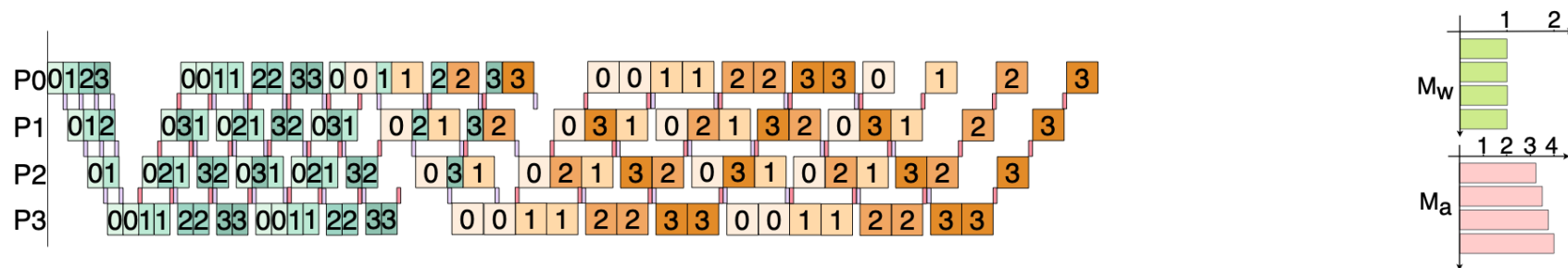
Select # replicas, # stages and Assemble blocks

Pipeline Training

- Using multiple WAVEs by dividing a microbatch into multiple smaller microbatches



(d) Hanayo with one wave

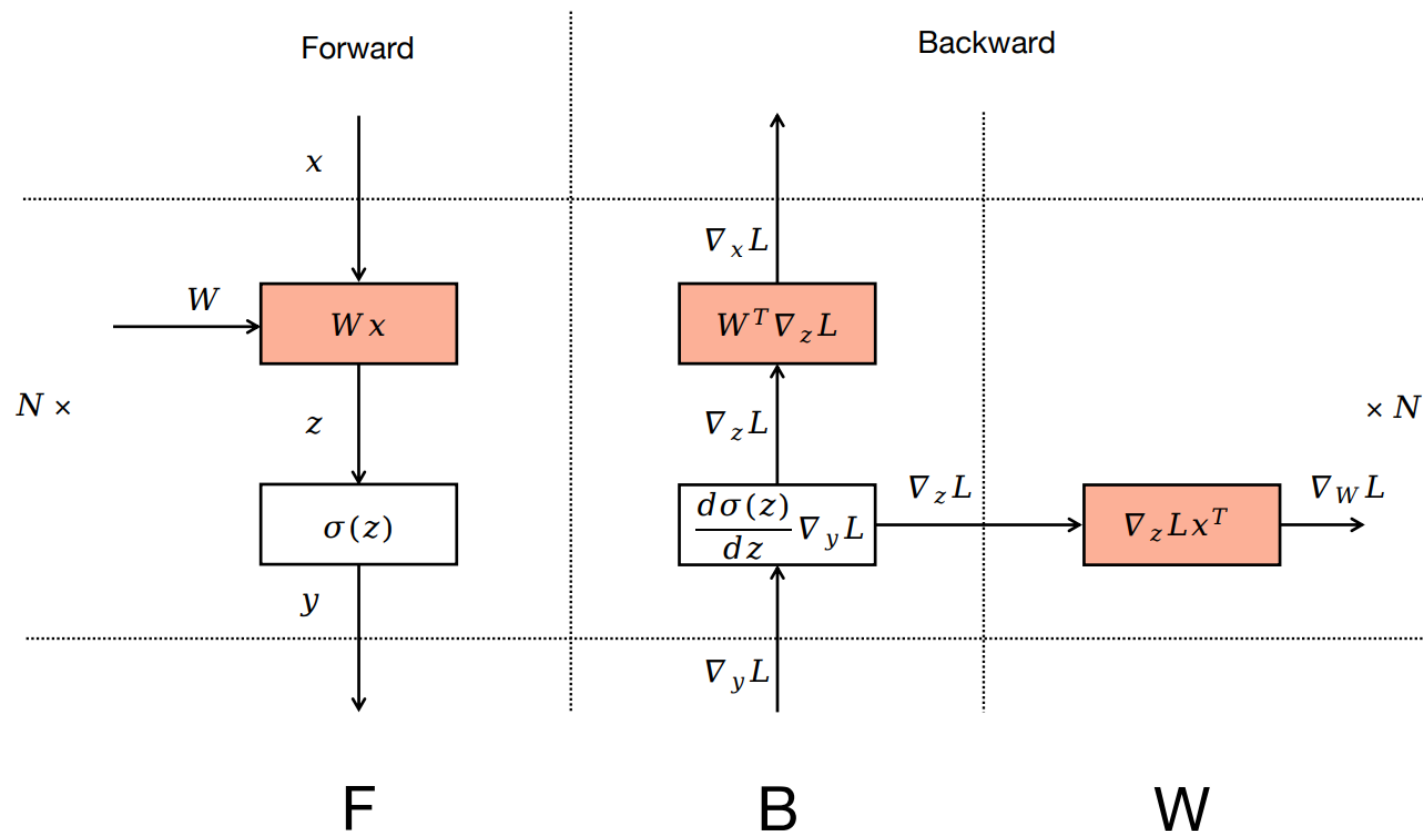


(e) Hanayo with two waves

Forward propagation marked green and Backward propagation marked orange. Back propagation illustrated twice as long as forward propagation. Using purple and pink blocks to represent P2P communication that goes downward and upward. The numbers on the blocks show which of the micro-batches is being processed.

Pipeline Training

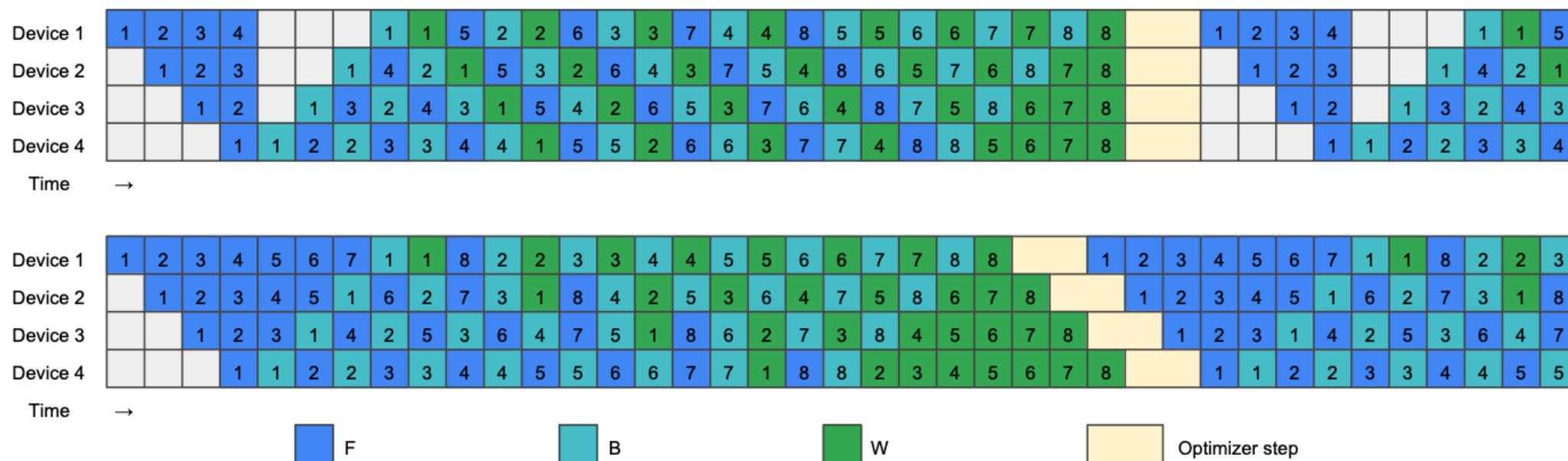
- A microscopic view of BP



- Derivative over input
 - for backward propagation
- Derivative over weight
 - for updating model directly
- $\nabla_W f(\mathbf{x}, \mathbf{W})^\top \frac{d\ell}{dy}$
 computed immediately after $\nabla_x f(\mathbf{x}, \mathbf{W})^\top \frac{d\ell}{dy}$

Pipeline Training

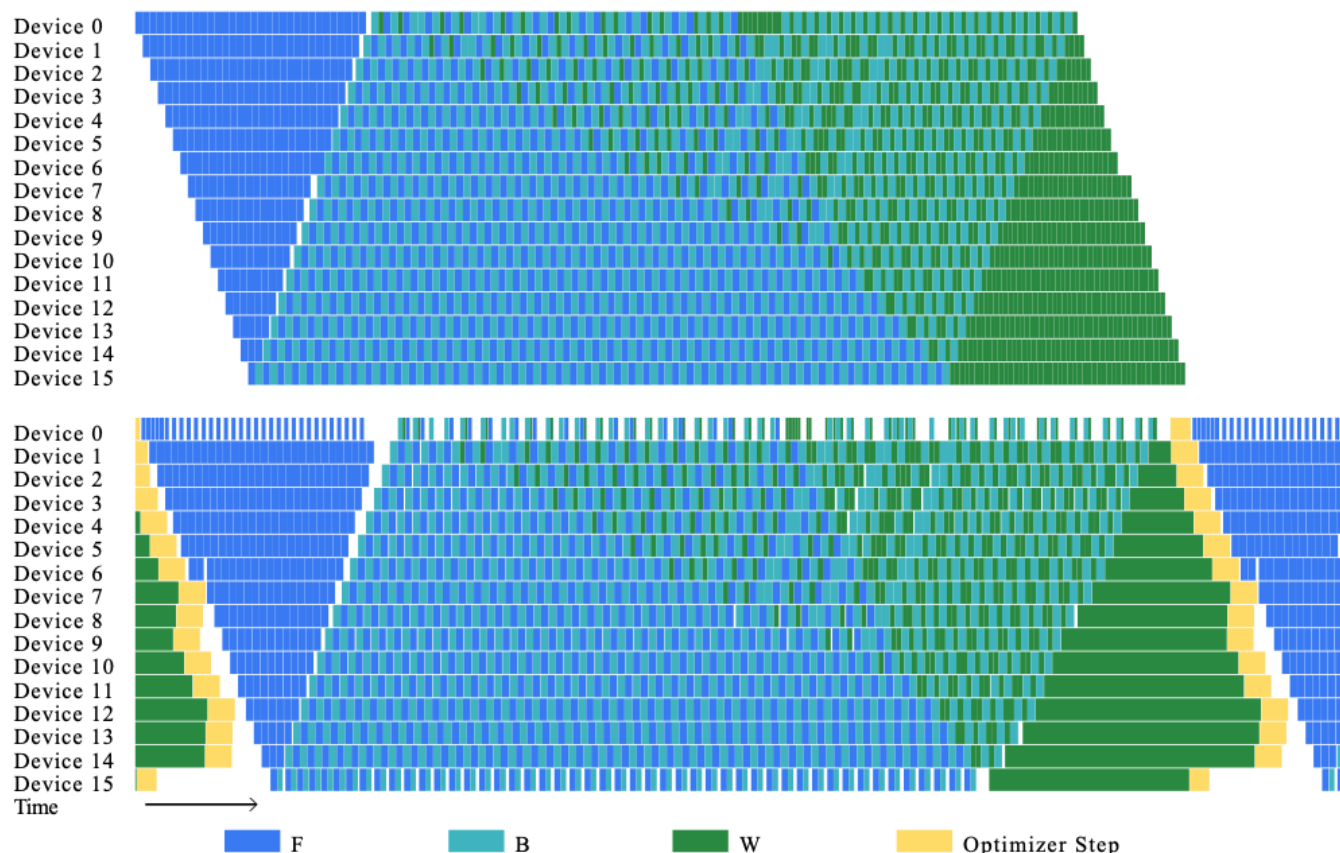
- Dividing BP into BP for input (urgent) and BP for weight (not urgent)



- Upper (ZB-H1): strictly follow 1F1B, and fill W to ensure all workers maintain the same number of in-flight microbatches.
- Lower (ZB-H2): theoretically zero bubble as # of microbatches is sufficiently large; large memory occupation over ZB-H1.

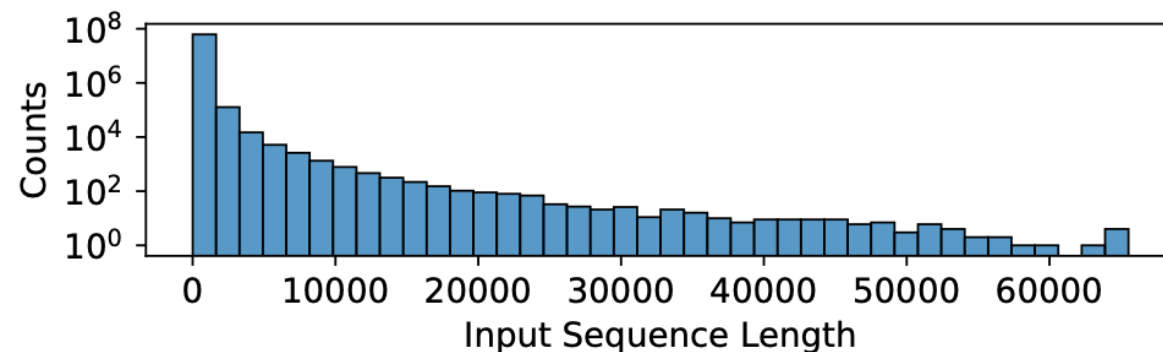
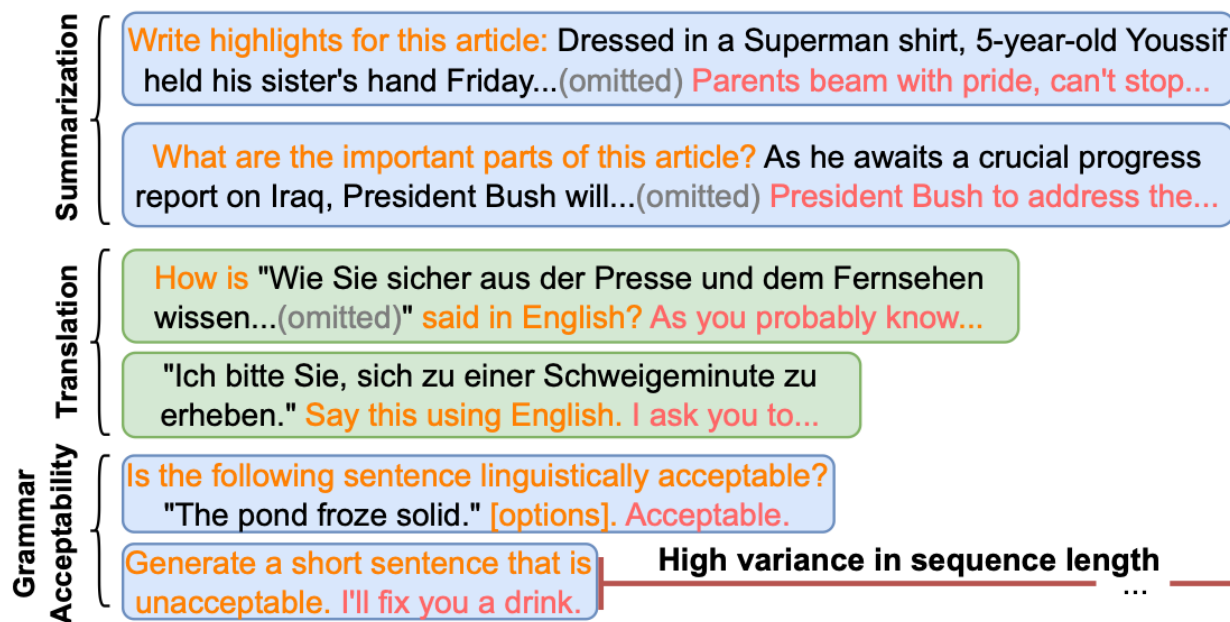
Pipeline Training

- An illustration of zero bubble with 16 stages: inserting W ops strategically



Pipeline Training

- Multi-task model pretraining
 - Input sequence lengths are non-uniform



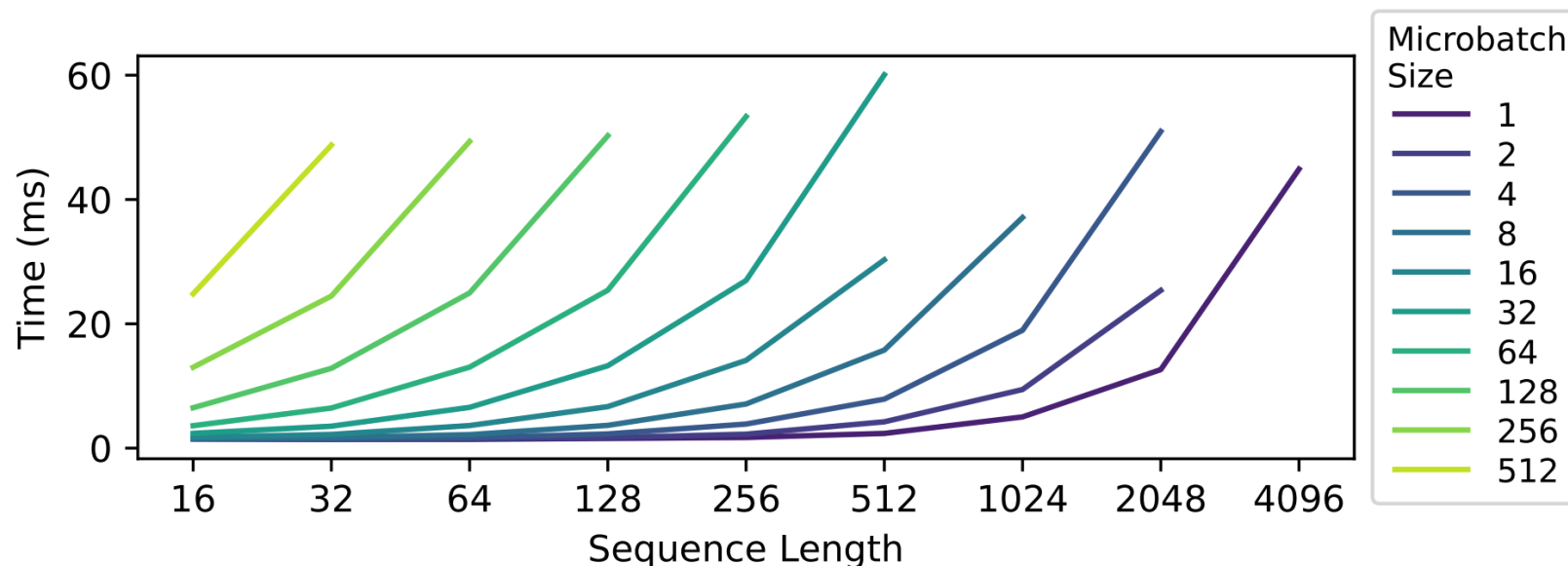
(left) Orange texts are instructions to the model. Inputs to process are colored black. Expected responses are in red.

(upper) The sequence length distribution in dataset (truncated at 65536).

Pipeline Training

- Multi-task model pretraining
 - Execution time exhibits super-linear growth with sequence length

$$\text{FLOPS per Transformer Layer} = 24bsh^2 + 4bs^2h$$

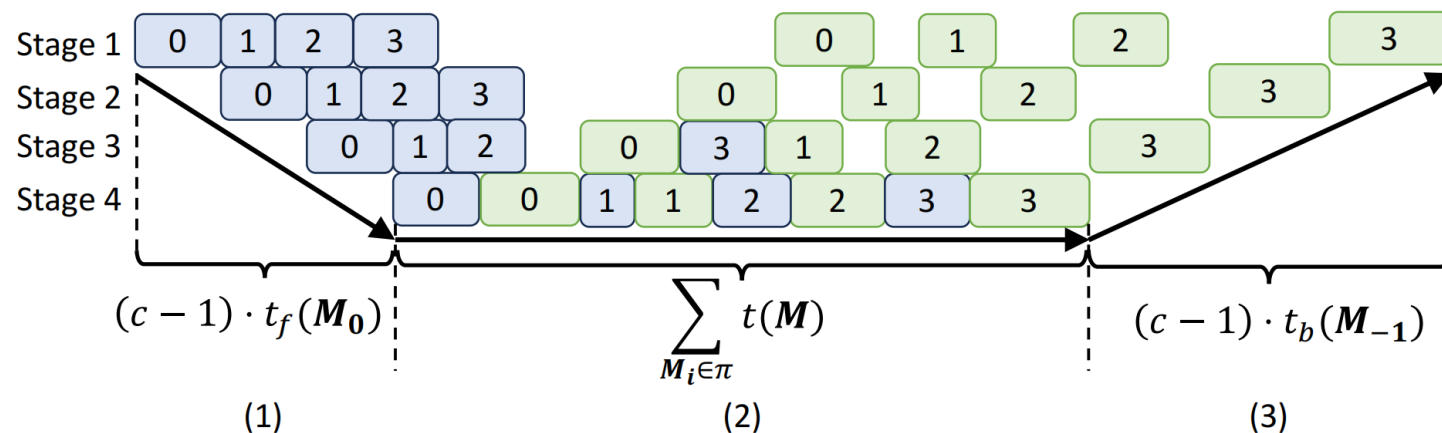


Pipeline Training

- Multi-task model pretraining
 - Seek the best micro-batch assignment π to minimize iteration time

$$\min_{\pi} \left\{ (c - 1) \cdot \max\{t(\mathbf{M}_i) | \mathbf{M}_i \in \pi\} + \sum_{\mathbf{M}_i \in \pi} t(\mathbf{M}_i) \right\}$$

- Construct a DP algorithm to optimally partition samples

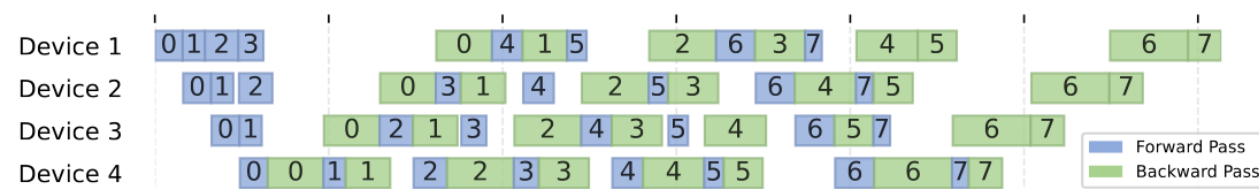


Pipeline Training

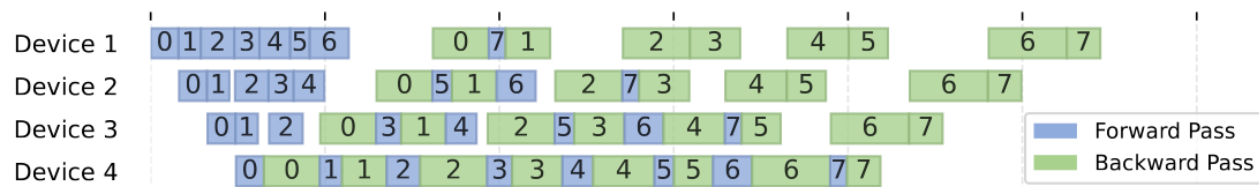
- Multi-task model pretraining
 - Packing and scheduling



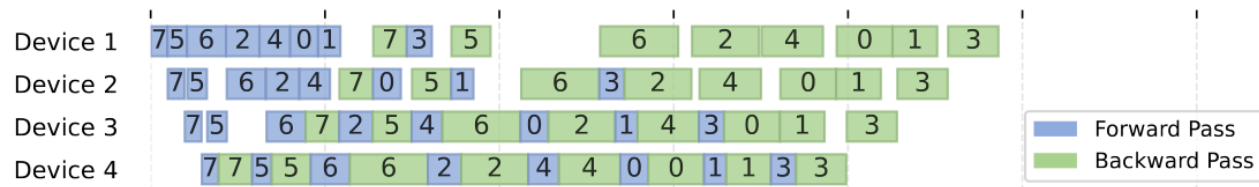
(a) 1F1B (uniform micro-batches)



(b) 1F1B (dynamic micro-batches)



(c) Adaptive Schedule



(d) Adaptive Schedule & Micro-batch Reordering

- Quite a few important studies on the similar problem
 - FlexPipe (ATC 2025), WLB-LLM (OSDI 2025), etc.

Distributed LLM Training: Outline

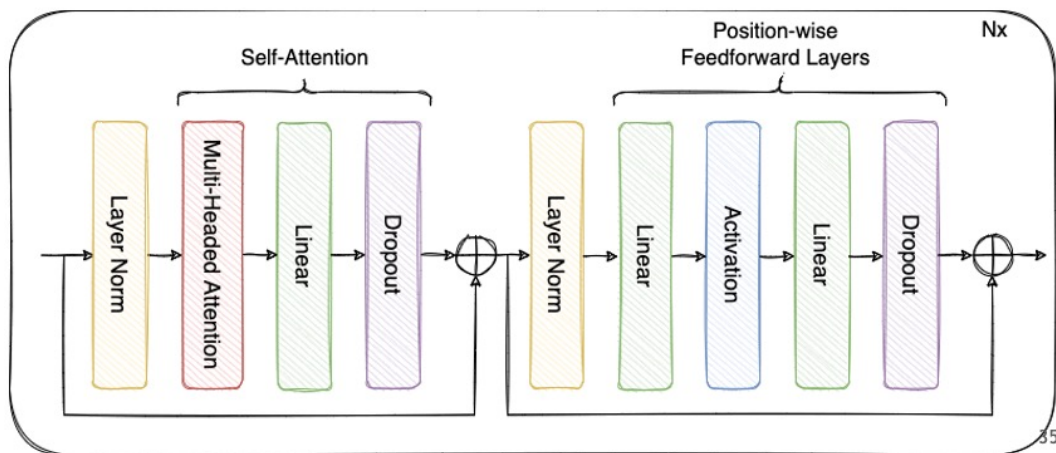
- Data Parallelism
 - Parameter-Server
 - All-Reduce
 - Memory Optimization
- Model Parallelism
 - Pipeline Parallelism
 - **Tensor Parallelism**
 - Sequence Parallelism
- Mixture of Experts

Tensor Parallelism

- A visual perception of TP

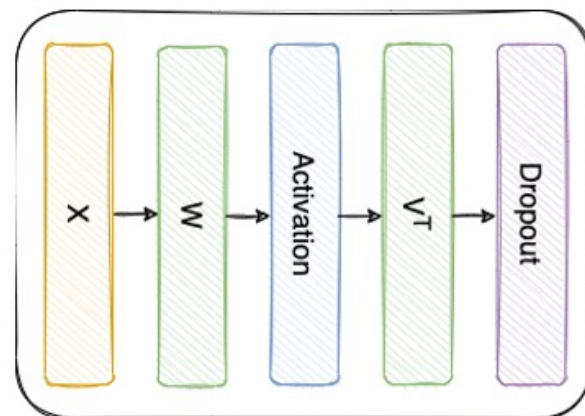
Transformers

"Pre-Layernorm" Transformer Block



Tensor Parallelism (Intra-Layer)

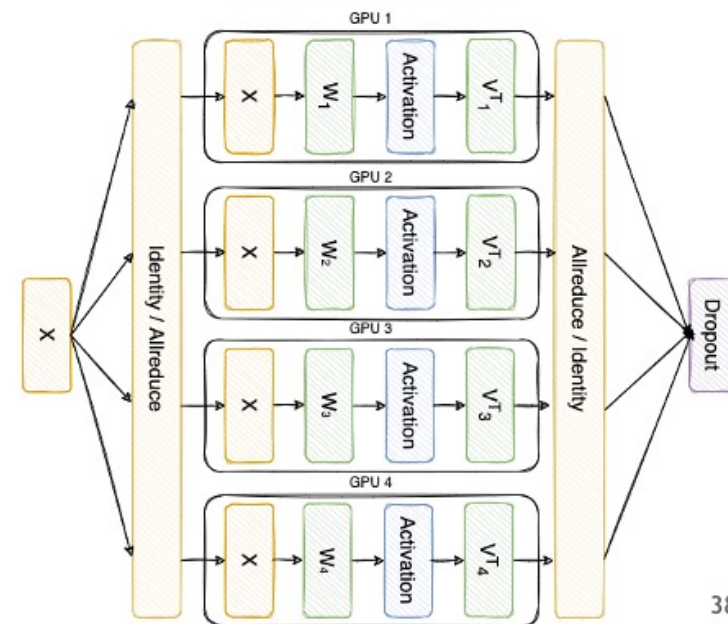
MLP



Split weights W and V^T across multiple GPUs

$$W = [W_1, W_2, W_3, W_4] \quad V = \begin{bmatrix} V_1 \\ V_2 \\ V_3 \\ V_4 \end{bmatrix}$$

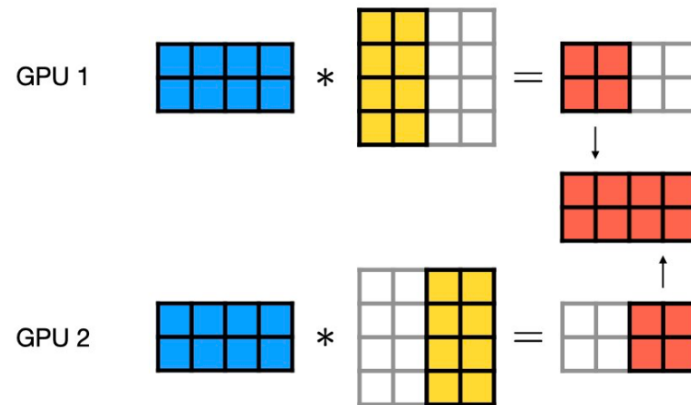
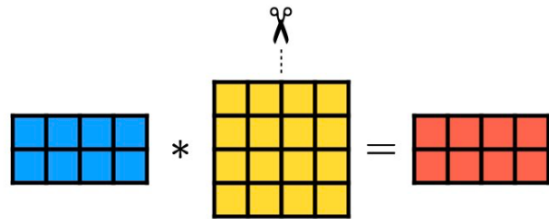
Tensor Parallel = 4 MLP



Easily understandable, but how?

Tensor Parallelism

- Mathematical principle of tensor parallelism (e.g. GEMM)
 - 1D: segmenting a matrix by **column** or row only
 - *First linear layer* $Y = XA$: $A = [A_1, A_2] \rightarrow Y = [XA_1, XA_2] = [Y_1, Y_2]$



Tensor Parallelism

- Mathematical principle of tensor parallelism (e.g. GEMM)
 - 1D: segmenting a matrix by **column** or row only

- *Second linear layer $Z = YB$:*

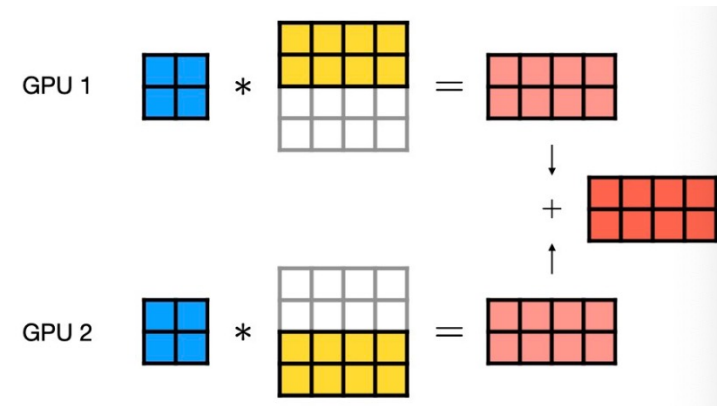
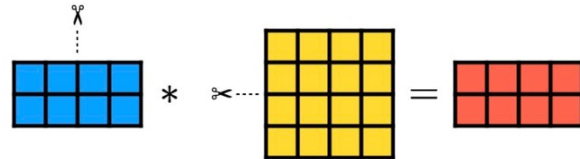
$$Z = \begin{bmatrix} Y_1 & Y_2 \end{bmatrix} \cdot \begin{bmatrix} B_1 \\ B_2 \end{bmatrix}$$

- *Compute $Y_i B_i$ independently and aggregate $Z = Y_1 B_1 + Y_2 B_2$*
 - Question: memory usage? communication load? end-to-end latency?
 - $1/P$ parameter, Full activation, $2(P - 1)/P$ communication load, $2(P-1)$ latency

Tensor Parallelism

- Mathematical principle of tensor parallelism (e.g. GEMM)
 - 1D: segmenting a matrix by column or **row** only
 - *First linear layer* $Y = XA$:

$$Y = [X_1 \ X_2] \begin{bmatrix} A_1 \\ A_2 \end{bmatrix}$$



Tensor Parallelism

- Mathematical principle of tensor parallelism (e.g. GEMM)
 - 1D: segmenting a matrix by column or **row** only

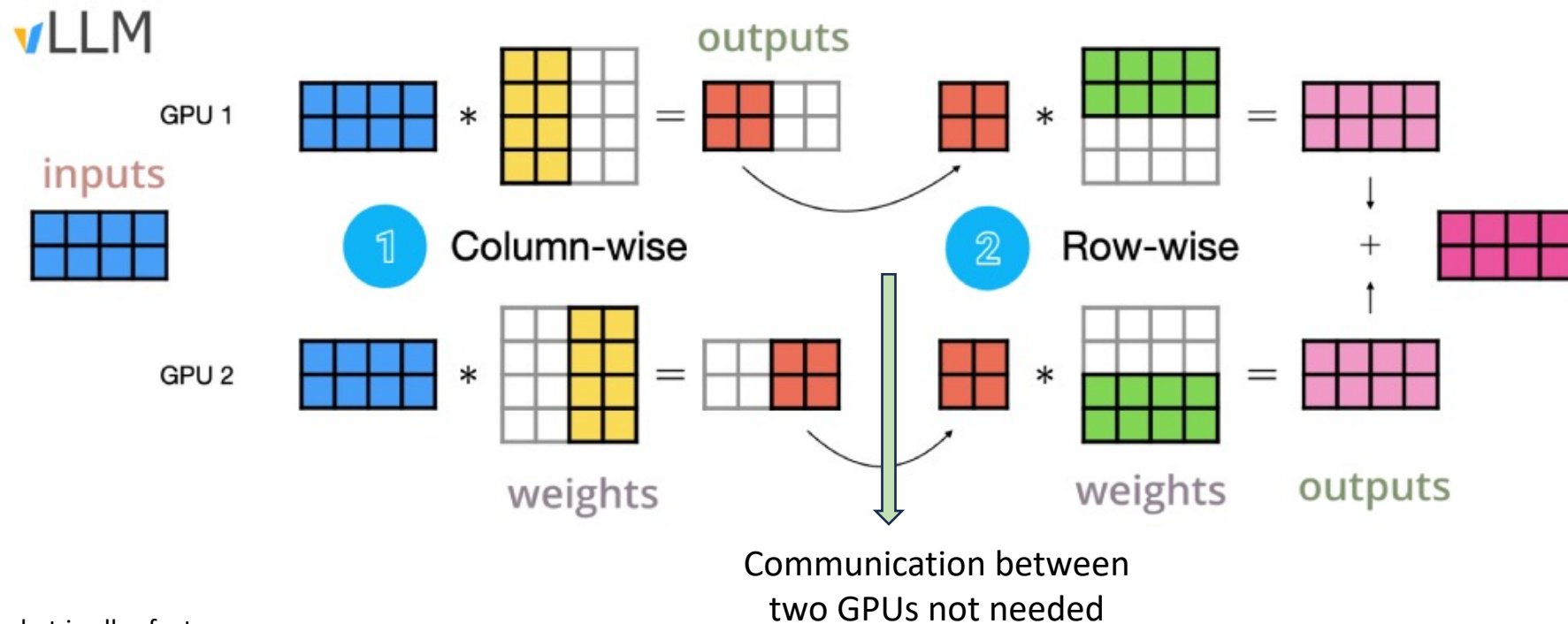
- *Second linear layer $Z = YB$:*

$$Z = \begin{bmatrix} Y_1 \\ Y_2 \end{bmatrix} \cdot [B_1 \ B_2]$$

- *Compute $Y_i B_i$ independently and aggregate $Z = Y_1 B_1 + Y_2 B_2$*
- Question: memory usage? communication load? end-to-end latency?
 - 1/P computation, 1/P parameter, Full activation, 2(P-1)/P communication load, 2(P-1) latency

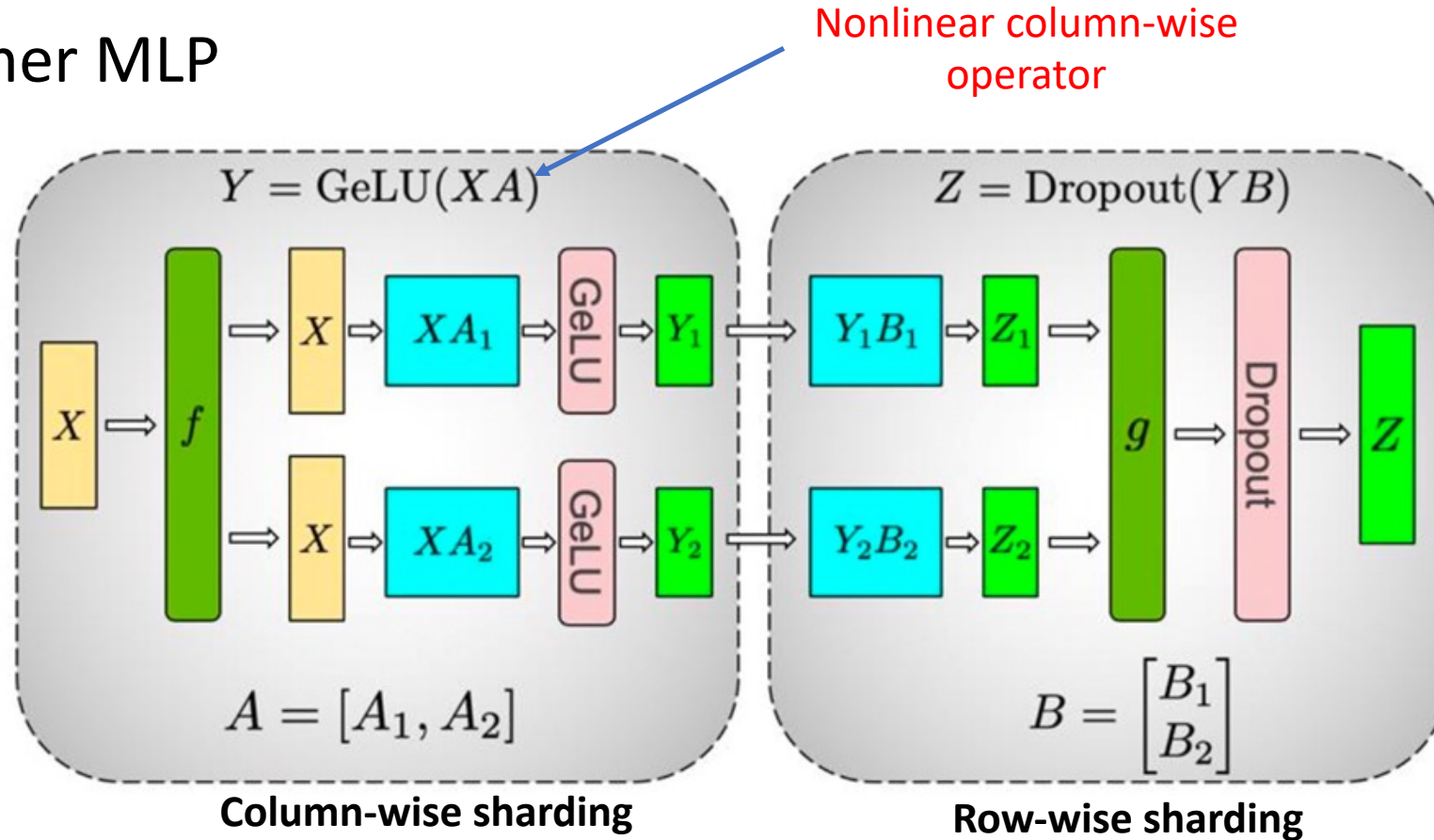
Tensor Parallelism

- How to split matrices
 - The column-wise and row-wise styles can be combined for multiple linear layers



Tensor Parallelism

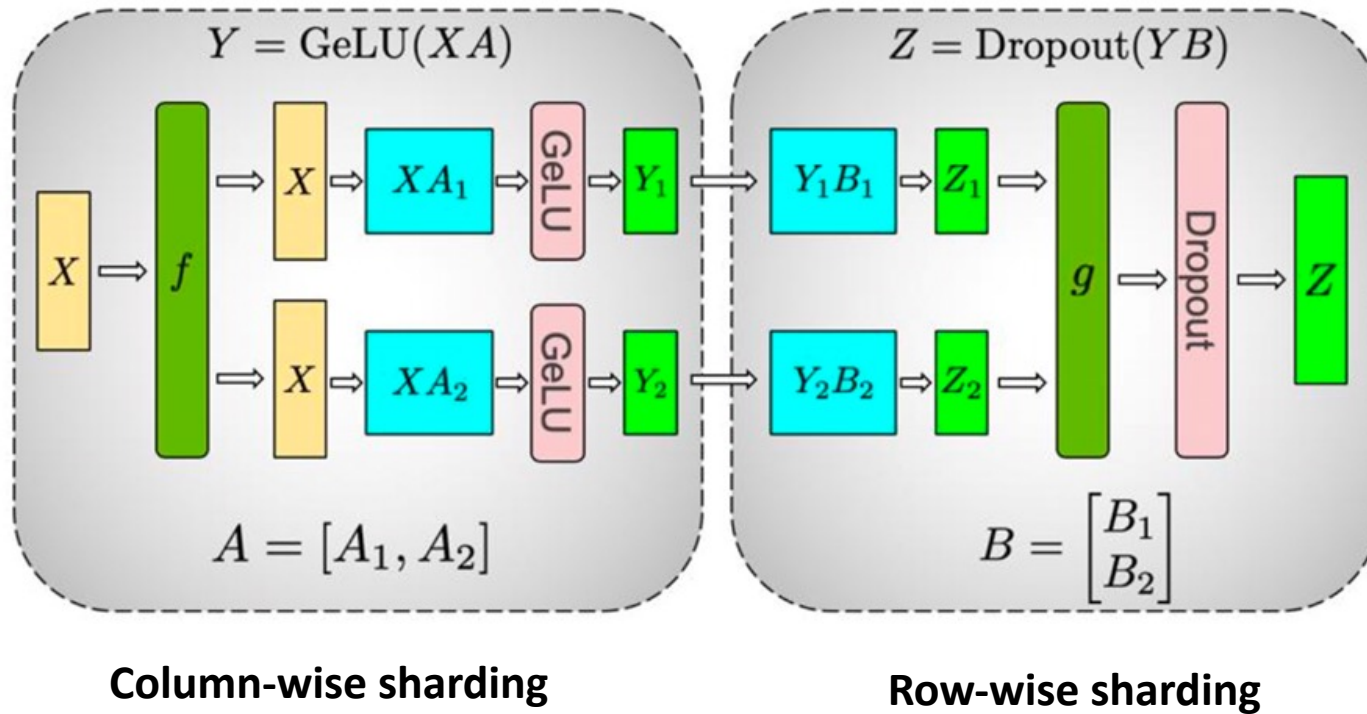
- Transformer MLP



f and g are **conjugate**, f is identity operator in the forward pass and all-reduce in the backward pass while g is all-reduce in forward and identity in backward

Tensor Parallelism

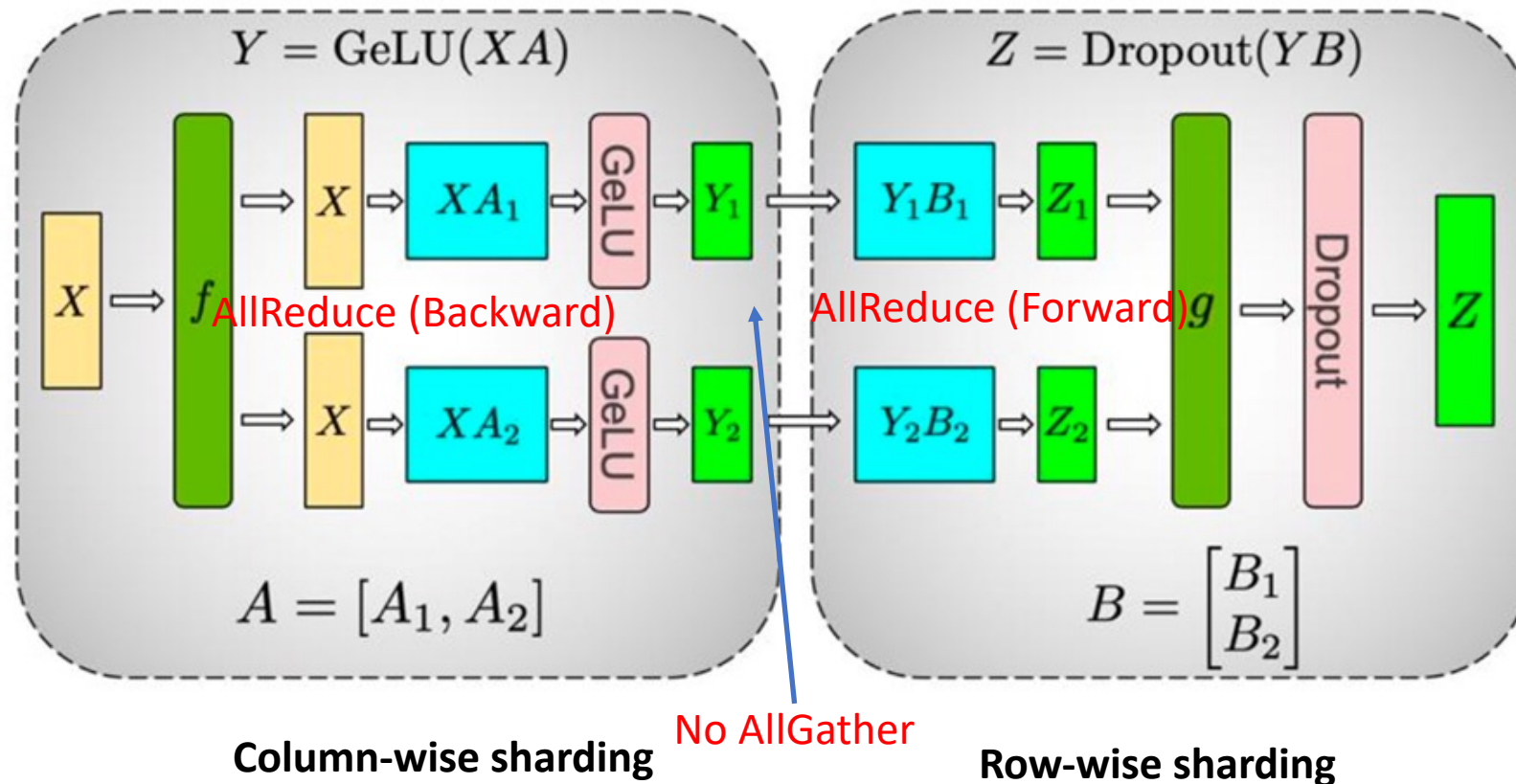
- Transformer MLP



- Partition A by columns and partition B by rows
- Forward pass
 - “Copy” X to both GPUs via f
 - Execute math operations
 - “Add” Z_1 and Z_2 for a complete output Z via g
- Backward pass
 - “copy” $\partial L / \partial Z$ to both GPUs via g
 - Execute math operations
 - “Add” partial derivatives of X on two paths via f

Tensor Parallelism

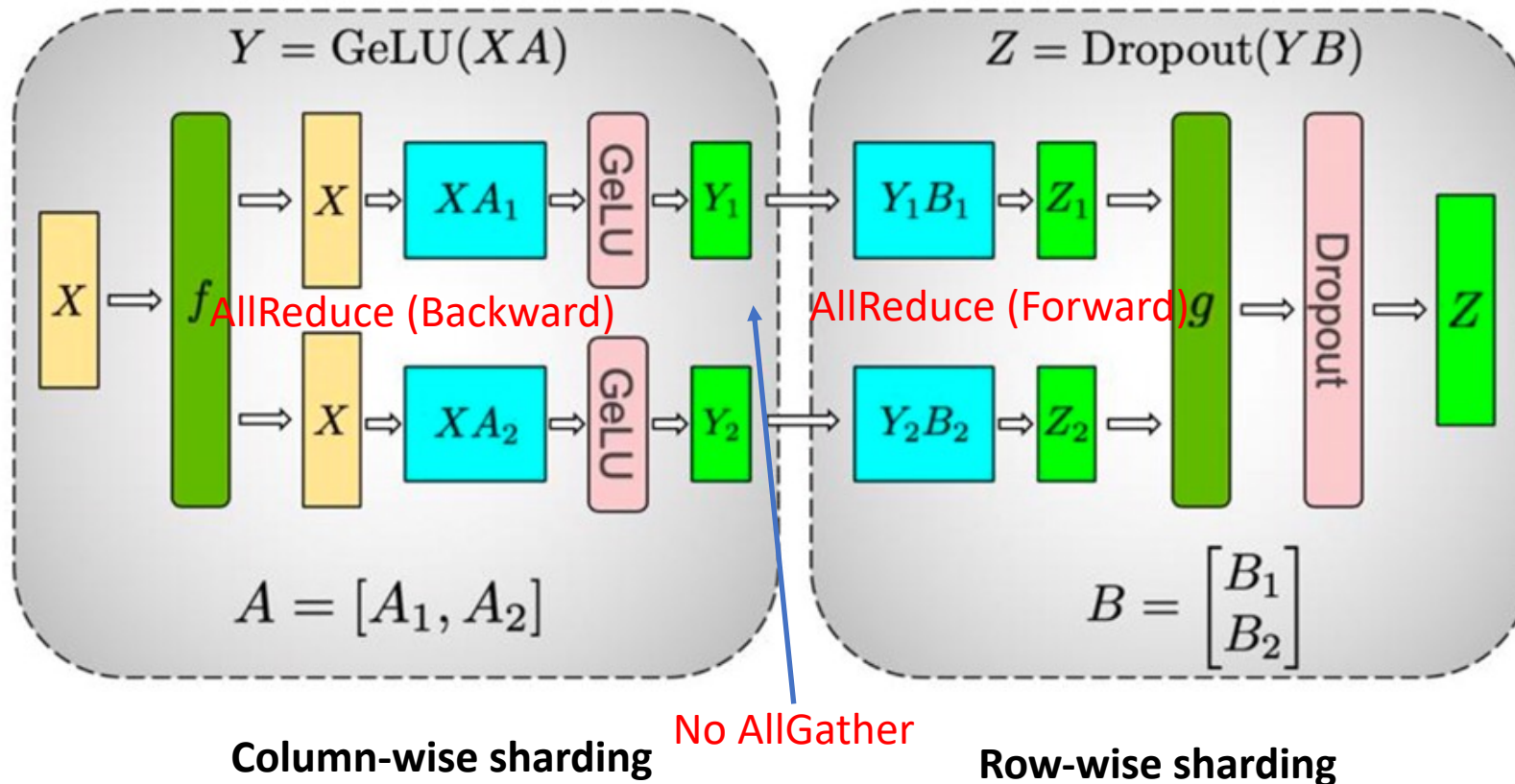
- Transformer MLP: *communication analysis*



f and g are **conjugate**, f is identity operator in the forward pass and all-reduce in the backward pass while g is all-reduce in forward and identity in backward

Tensor Parallelism

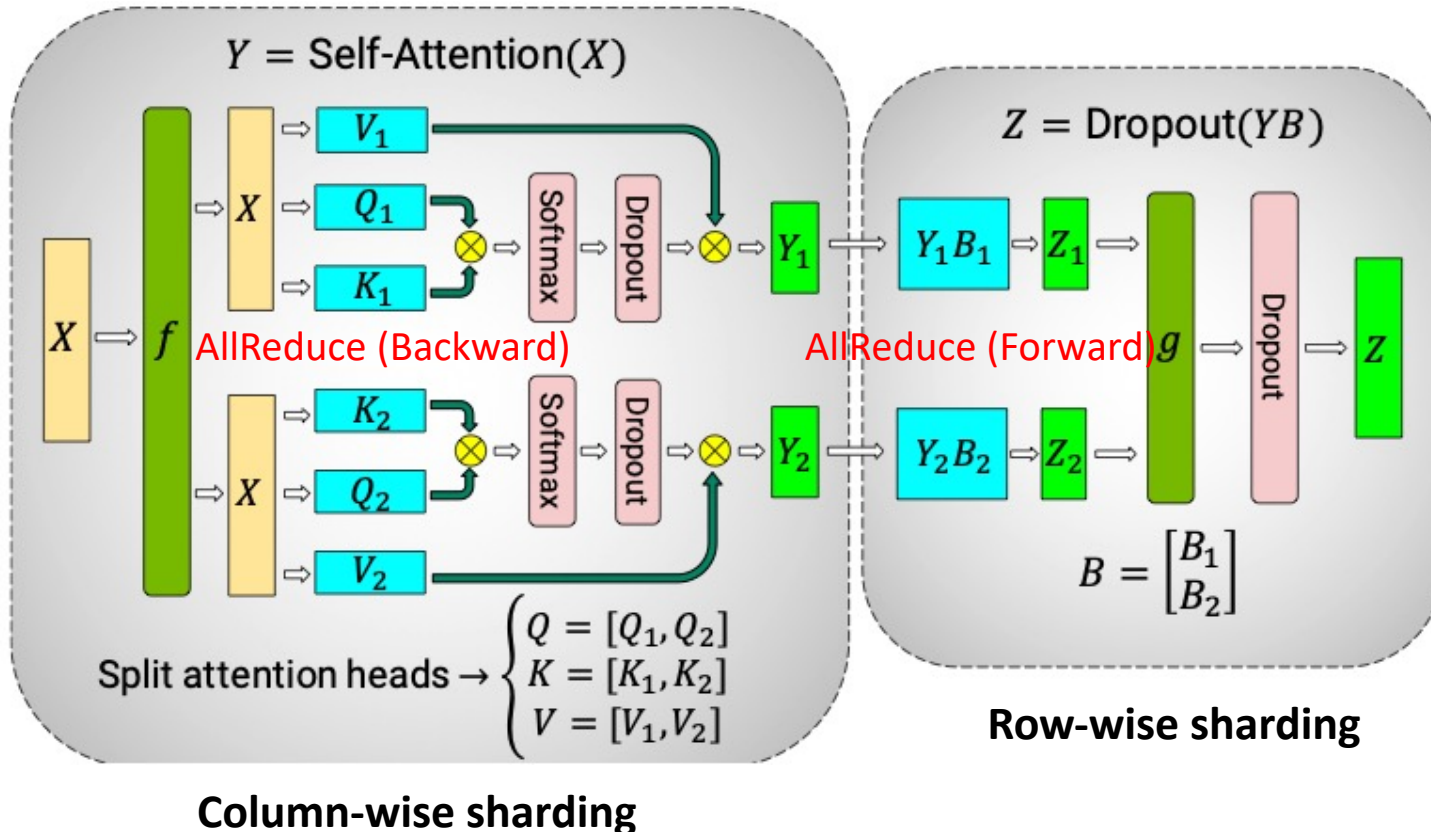
- Transformer MLP: *communication analysis*



f and g are **conjugate**, f is identity operator in the forward pass and all-reduce in the backward pass while g is all-reduce in forward and identity in backward

Tensor Parallelism

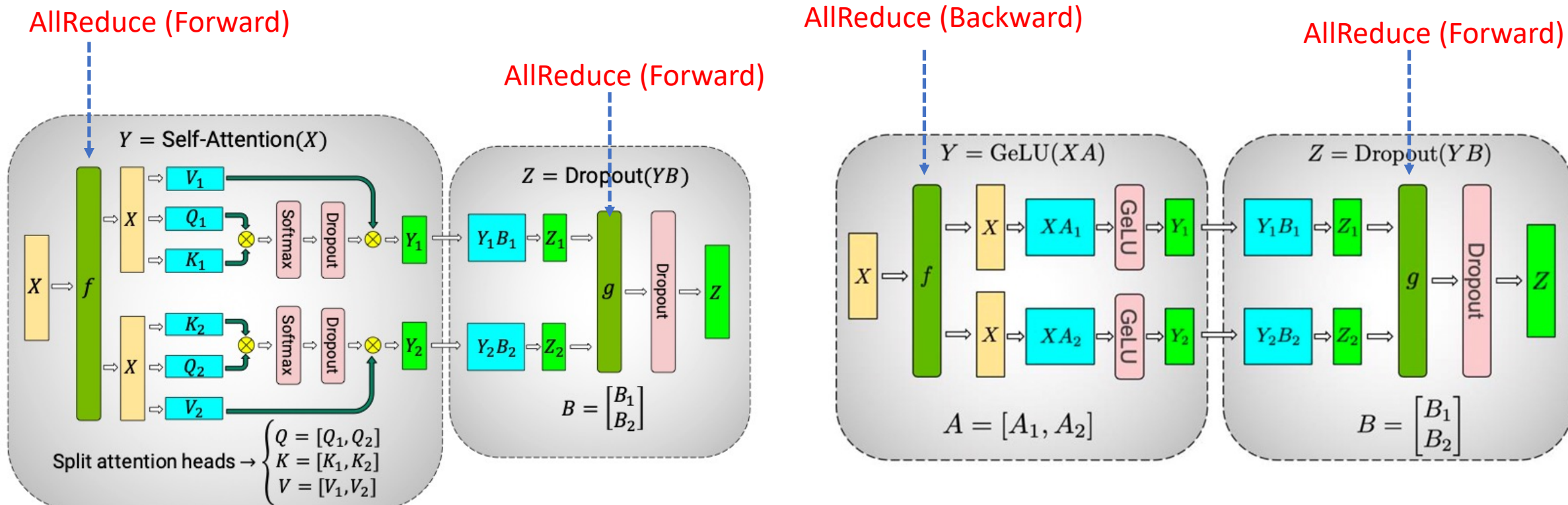
- Transformer Self-Attention



- Partition three parameter matrices W_Q , W_K and W_V by columns and partition the linear layer B by rows
- Forward pass
 - “Copy” X to both GPUs via f
 - Execute math operations such as computing Q , K , V
 - “Add” Z_1 and Z_2 for a complete output Z via g
- Backward pass
 - “copy” $\partial L / \partial Z$ to both GPUs via g
 - Execute math operations
 - “Add” partial derivatives of X on two paths via f

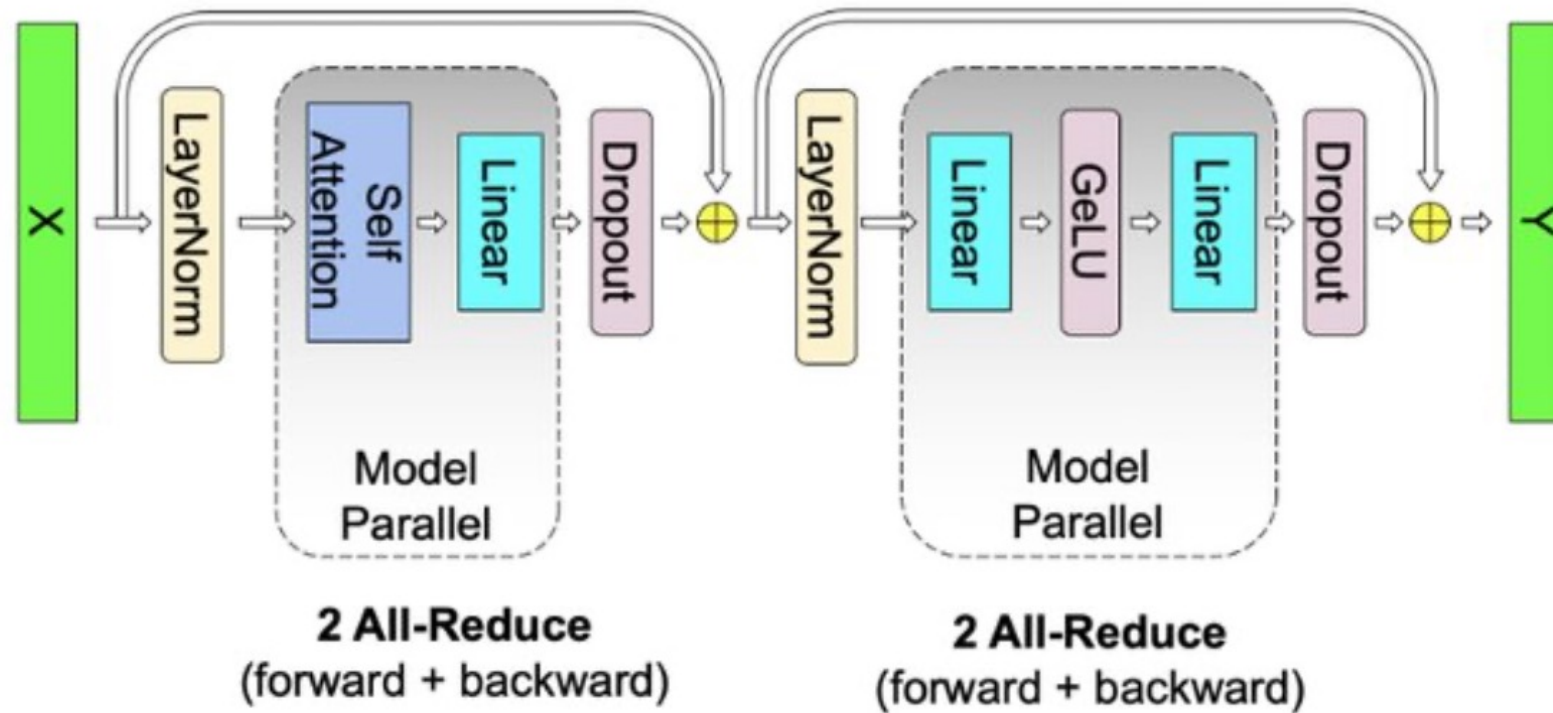
Tensor Parallelism

- TP communication analysis



Tensor Parallelism

- Transformer layer: communication analysis

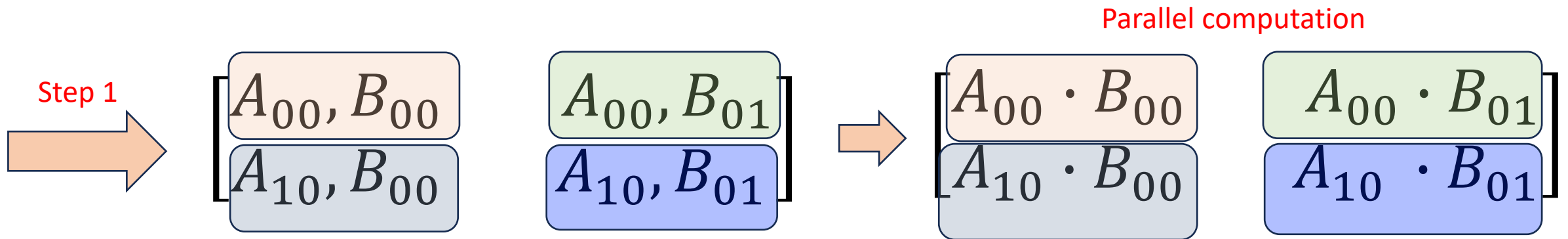
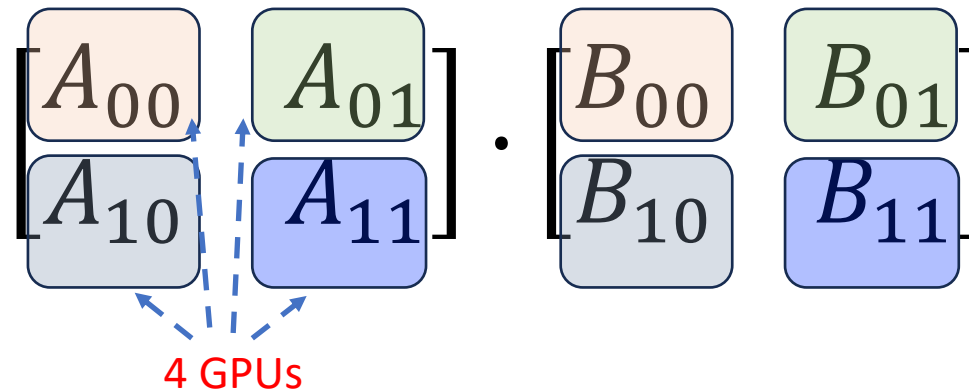


Total communication complexity: 8Φ . (Φ is the volume of the corresponding parameters)

《Accelerating Large Language Model Training with Hybrid GPU-based Compression》

Tensor Parallelism

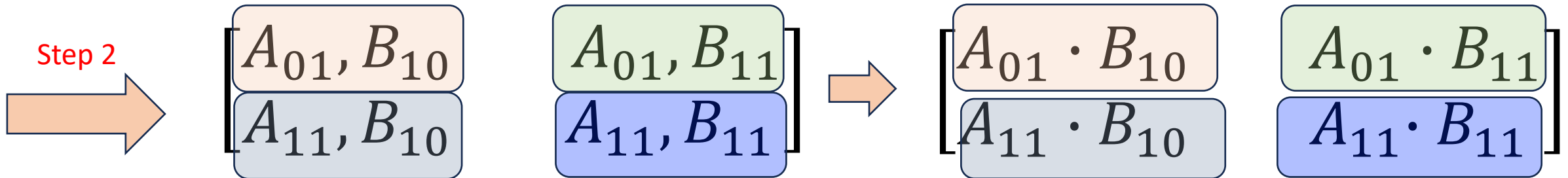
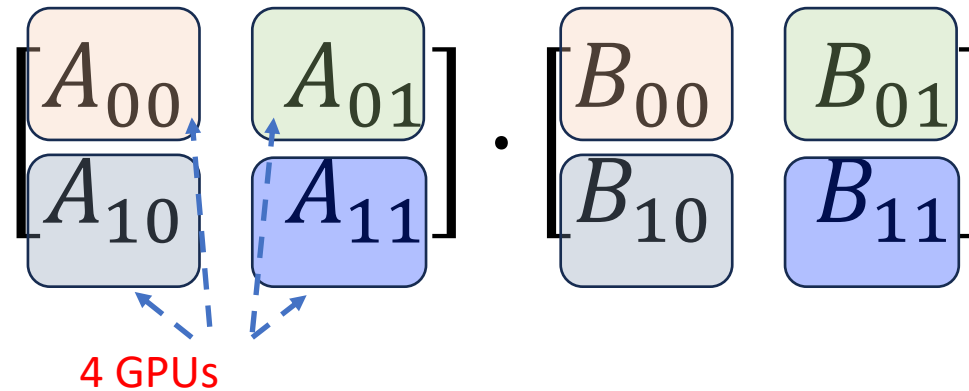
- 2D Tensor Parallelism: each color represents the parameter block hold by one GPU.



Broadcast #1 column of A to the right, and broadcast #1 row of B down, and multiply two submatrices

Tensor Parallelism

- 2D Tensor Parallelism

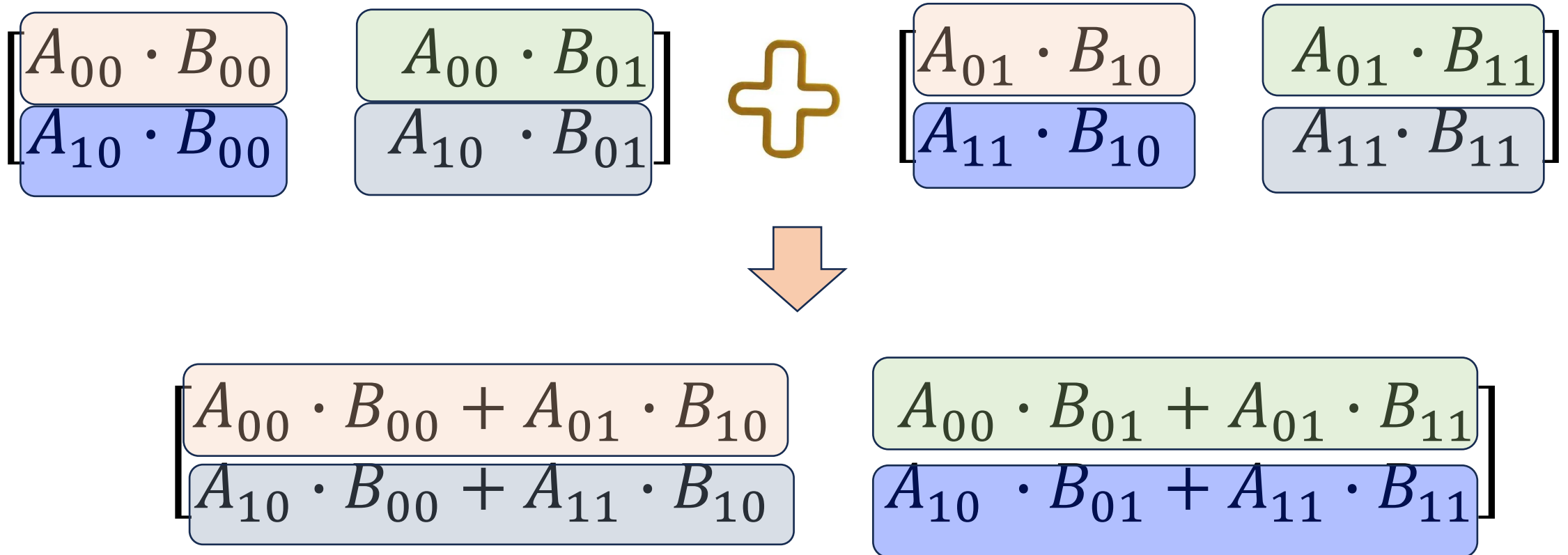


Broadcast #2 column to the left, and broadcast #2 row up, and multiply two matrices

Tensor Parallelism

- 2D Tensor Parallelism

Sequential computation



Tensor Parallelism

- 2D Tensor Parallelism: general procedure on $Y = AB$

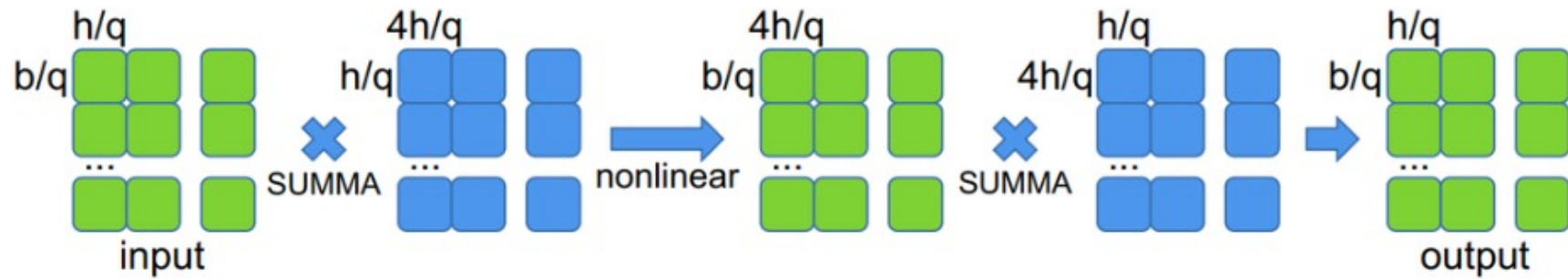
Trading communication for memory:

Less computation, less memory usage, more communication

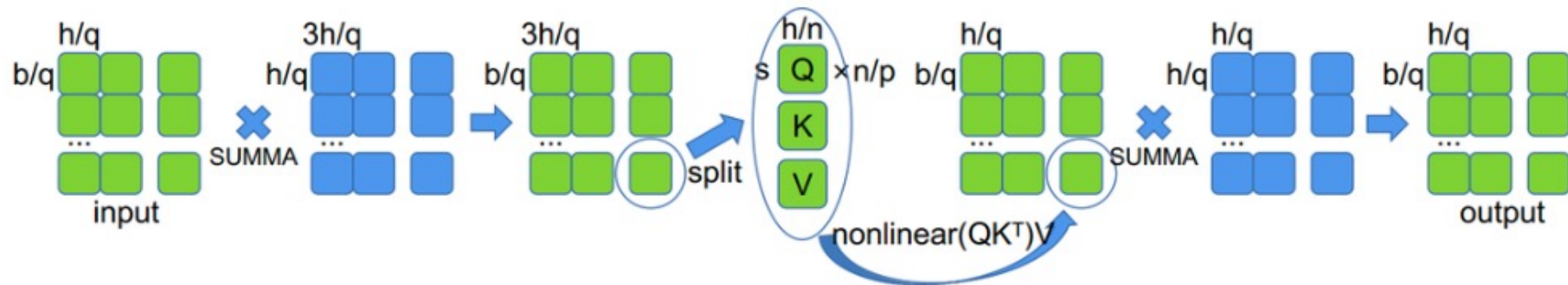
- Computation, memory and communication complexity
 - Computation: $O\left(\frac{1}{P}\right)$
 - Memory (parameter and activation): $O\left(\frac{1}{P}\right)$
 - Communication: $O\left(\frac{6(q-1)}{q}\right)$ (a constant factor of model size, not carefully checked)

Tensor Parallelism

- 2D Tensor Parallelism



(a) MLP



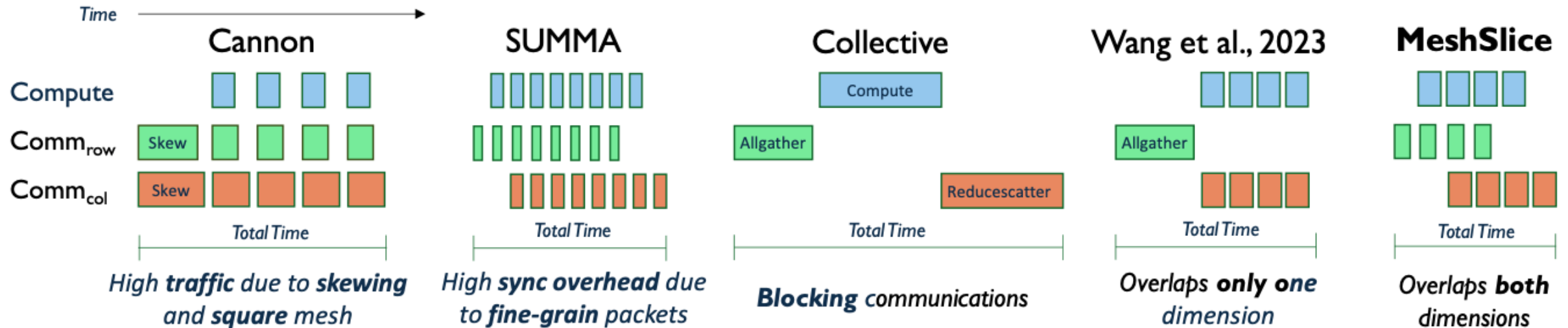
(b) self-attention

Green: activations, Blue: Transformer modules

《SUMMA: Scalable universal matrix multiplication algorithm》

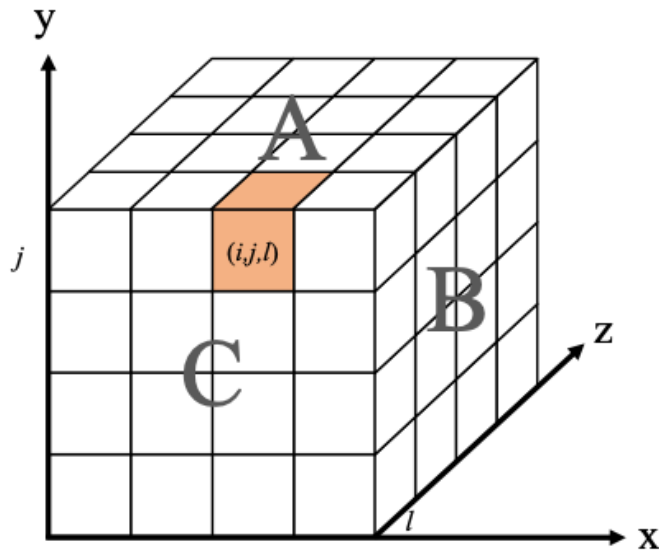
Tensor Parallelism

- 2D Tensor Parallelism: still many ongoing works, supplementary reading

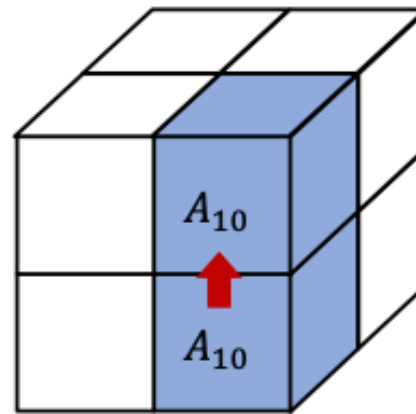


Tensor Parallelism

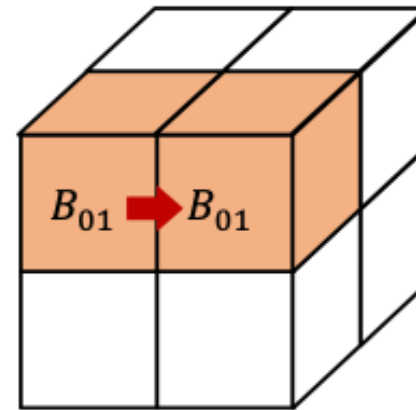
- 3D Tensor Parallelism: further segmentation for less memory usage
 - Supplementary reading



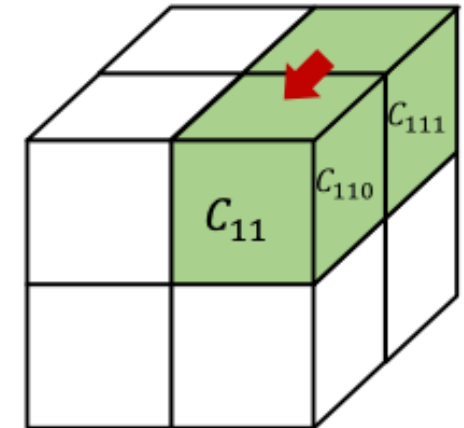
Colored block (i,j,k) represents an example of a single processor.



(a) Broadcast A_{il}



(b) Broadcast B_{lj}



(c) Reduce C_{ij}

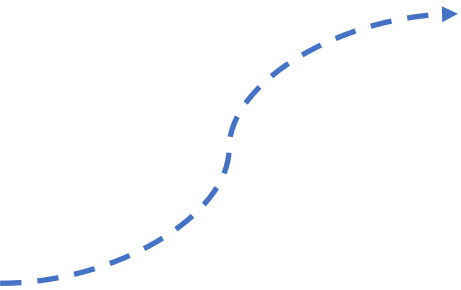
An example of the 3-D parallel matrix multiplication

Distributed LLM Training: Outline

- Data Parallelism
 - Parameter-Server
 - All-Reduce
 - Memory Optimization

- Model Parallelism
 - Pipeline Parallelism
 - Tensor Parallelism

- **Sequence Parallelism**



Sequence parallelism and context parallelism are mixed in this subsection

- Mixture of Experts

Sequence Parallelism

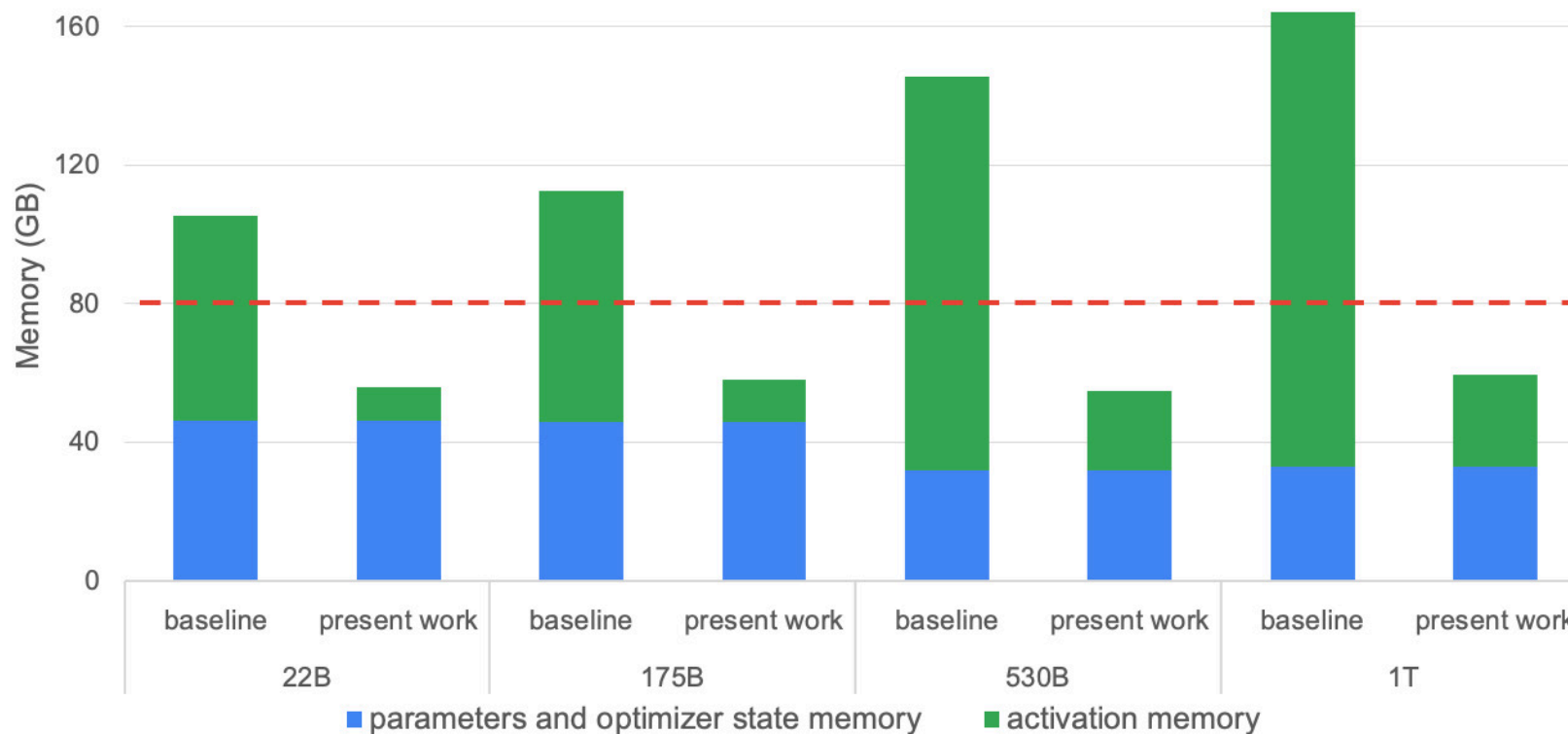
- Sequence Parallelism
 - Tensor parallelism reduces the memory footprint of **model data**, how about **non-model data**?
 - Activation memory is $O(n^2)$ in *self-attention* with n being the sequence length
 - Attention scores not stored in HBM
 - Activations include **Q , K , V** matrices

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- *Some layers have not been parallelized: layernorm and dropout*

Sequence Parallelism

- Sequence Parallelism



Parameters, optimizer state, and activations memory
《Reducing Activation Recomputation in Large Transformer Models》

Sequence Parallelism (Recap)

- Activation is non-negligible

- Forward propagation
- Backward propagation
- Size of activation

- Standard transformer: b - batch size, s - sequence length, h - hidden layer dimension

Output of a previous layer, intermediate variable

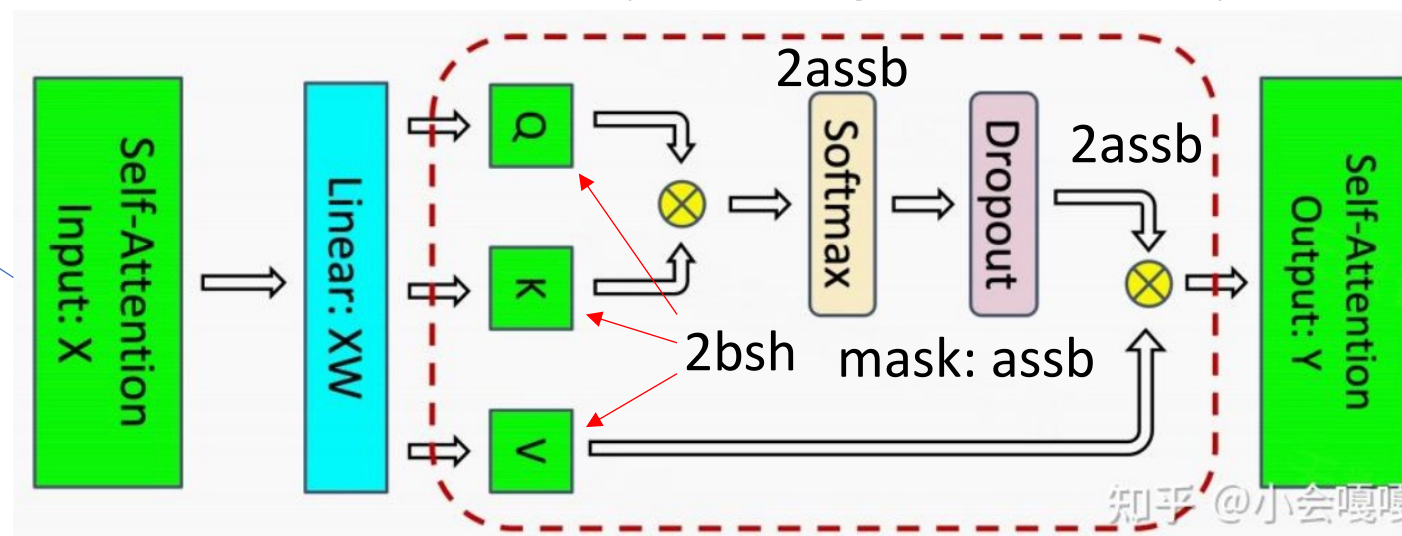
$$y = Wx + b$$

$$\frac{dL}{dW} = \frac{dL}{dy} x$$

Cannot be discarded after FP

bsh * 2 Bytes

In-total: 5abss + 8sbh



Sequence Parallelism (Recap)

- Activation is non-negligible

- Forward propagation
- Backward propagation
- Size of activation

- Standard transformer: b - batch size, s - sequence length, h - hidden layer dimension

Output of a previous layer, intermediate variable

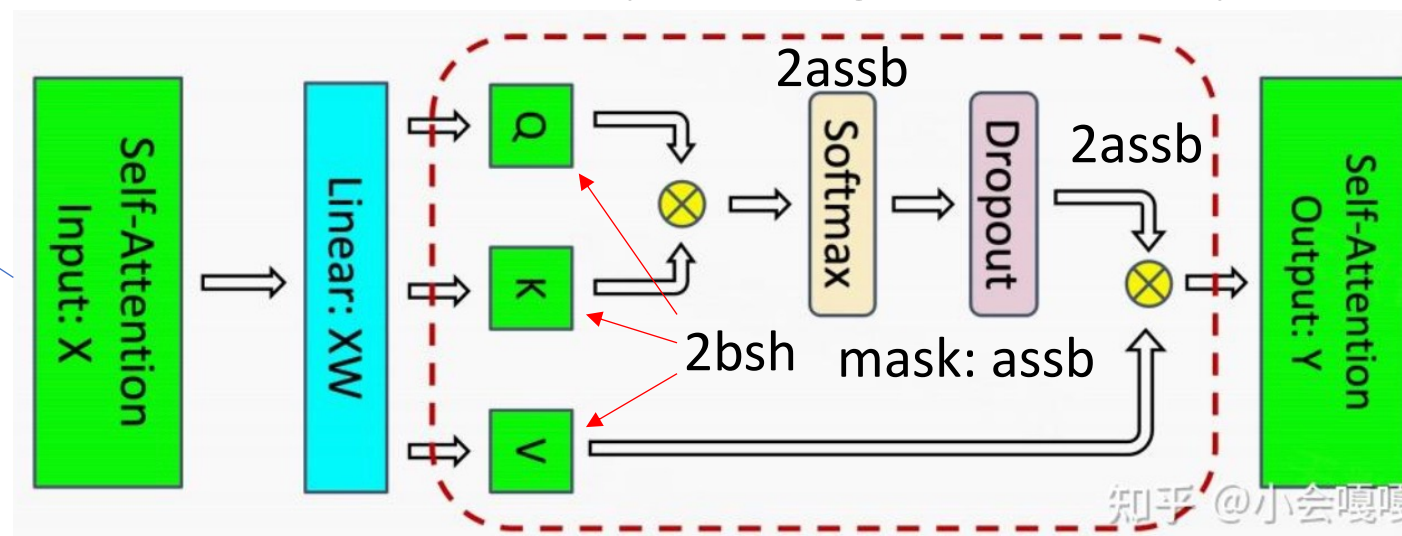
$$y = Wx + b$$

$$\frac{dL}{dW} = \frac{dL}{dy} x$$

Cannot be discarded after FP

bsh * 2 Bytes

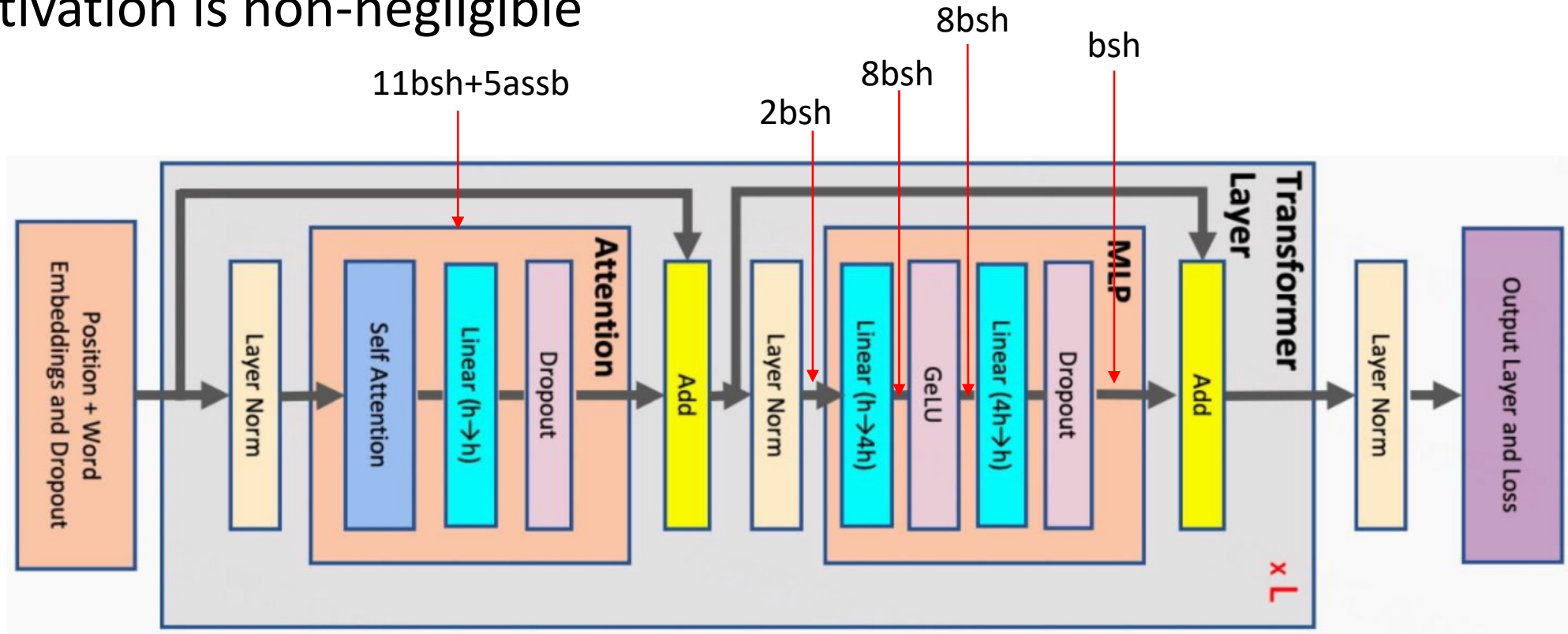
In-total: 5abss + 11sbh



3sbh: Linear layer + dropout layer

Sequence Parallelism (Recap)

- Activation is non-negligible



In-total: $5abss + 34sbh$

Sequence Parallelism

- What to keep for a Transformer block in HBM?

- MLP

- Input to MLP ✓
- Input of Dropout ✓
- Input of GeLU
- Inputs of linear matrices ✓

- Attention

- Input to Attention ✓
- Q, K and V ✓
- QK^T
- $\text{Softmax}(QK^T)$ ✓
- Dropout Mask ✓

Sequence Parallelism

- An exercise: manual gradient computation

- MLP

- Input to MLP ✓
- Input of Dropout ✓
- **Input of GeLU**
- Inputs of linear matrices ✓

- Attention

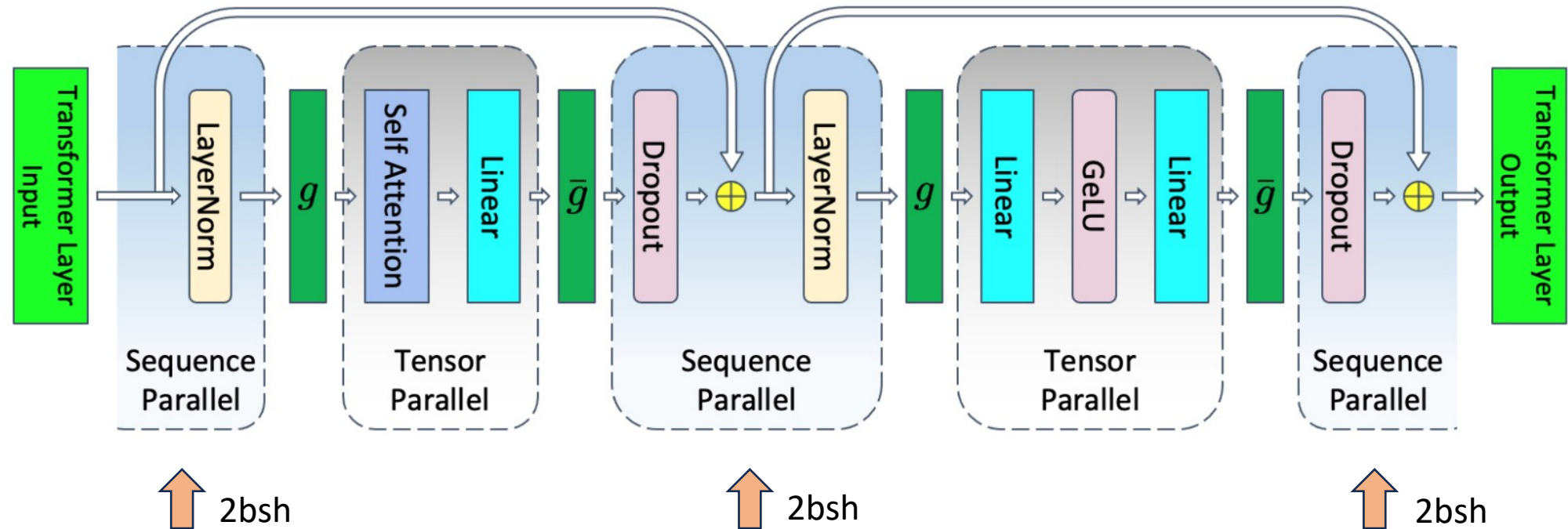
- Input to Attention ✓
- Q, K and V ✓
- **QK^T**
- $\text{Softmax}(QK^T)$ ✓
- Dropout Mask ✓

Sequence Parallelism

- Representative SP frameworks
 - Megatron-LM SP
 - 《Reducing Activation Recomputation in Large Transformer Models》
 - Deemed as a further optimization of Tensor Parallelism
 - DeepSpeed Ulysses
 - 《DeepSpeed Ulysses: System Optimizations for Enabling Training of Extreme Long Sequence Transformer Models》
 - Colossal AI SP
 - 《Colossal-AI: A Unified Deep Learning System For Large-Scale Parallel Training》
 - Megatron Context Parallelism
 - https://docs.nvidia.com/megatron-core/developer-guide/latest/api-guide/context_parallel.html

Sequence Parallelism

- Megatron-LM sequence parallelism



Tensor parallelism only segments inputs and outputs of **self-attention** and **MLP**

Sequence Parallelism

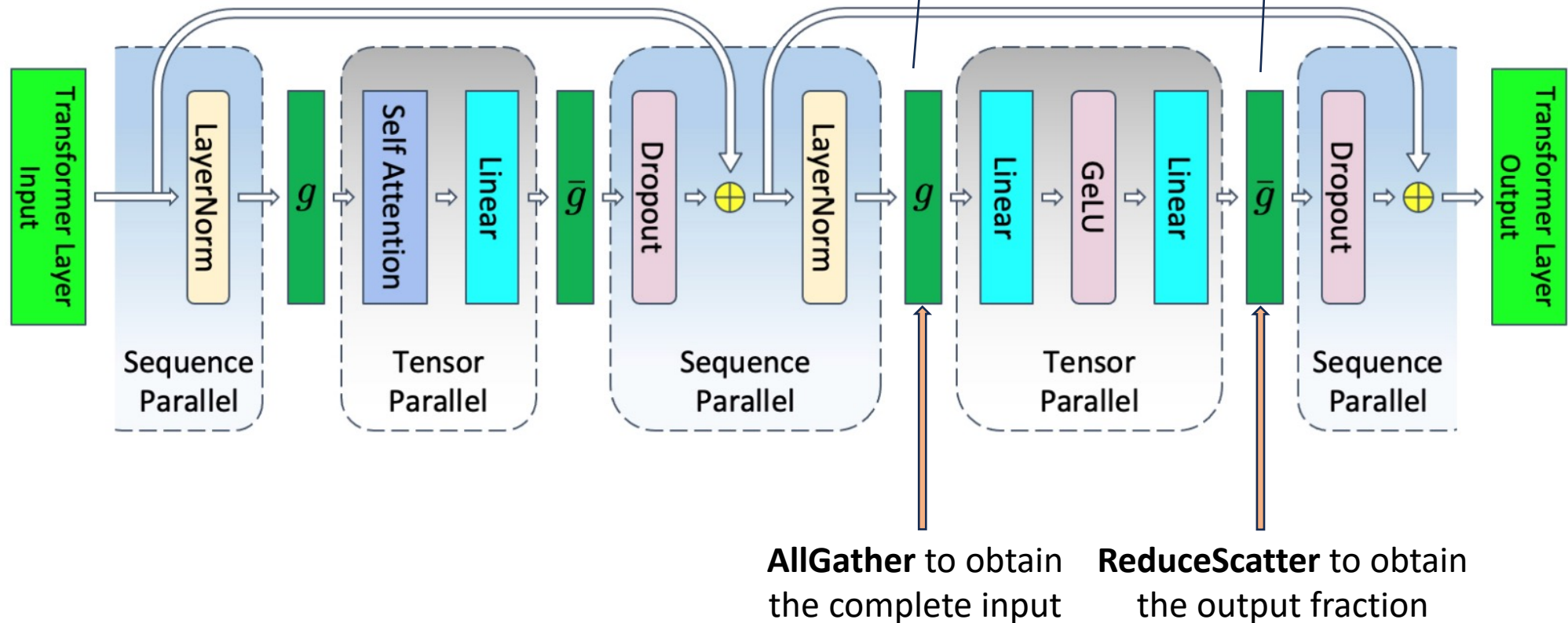
- Megatron-LM sequence parallelism
 - *layernorm* operation
 - The shape of an input matrix: [batch_size, seq_len, hidden_dim]
 - Compute mean, variance and a learnable affine transformation for each **TOKEN**
 - Can be naturally partitioned by tokens without any cross-device communication
 - *Dropout* operation
 - Each GPU compute dropout independently

Activations (input/output of layernorm) on each GPU: $\frac{2bsh}{t}$

Activations (dropout) on each GPU: $\frac{bsh}{t}$

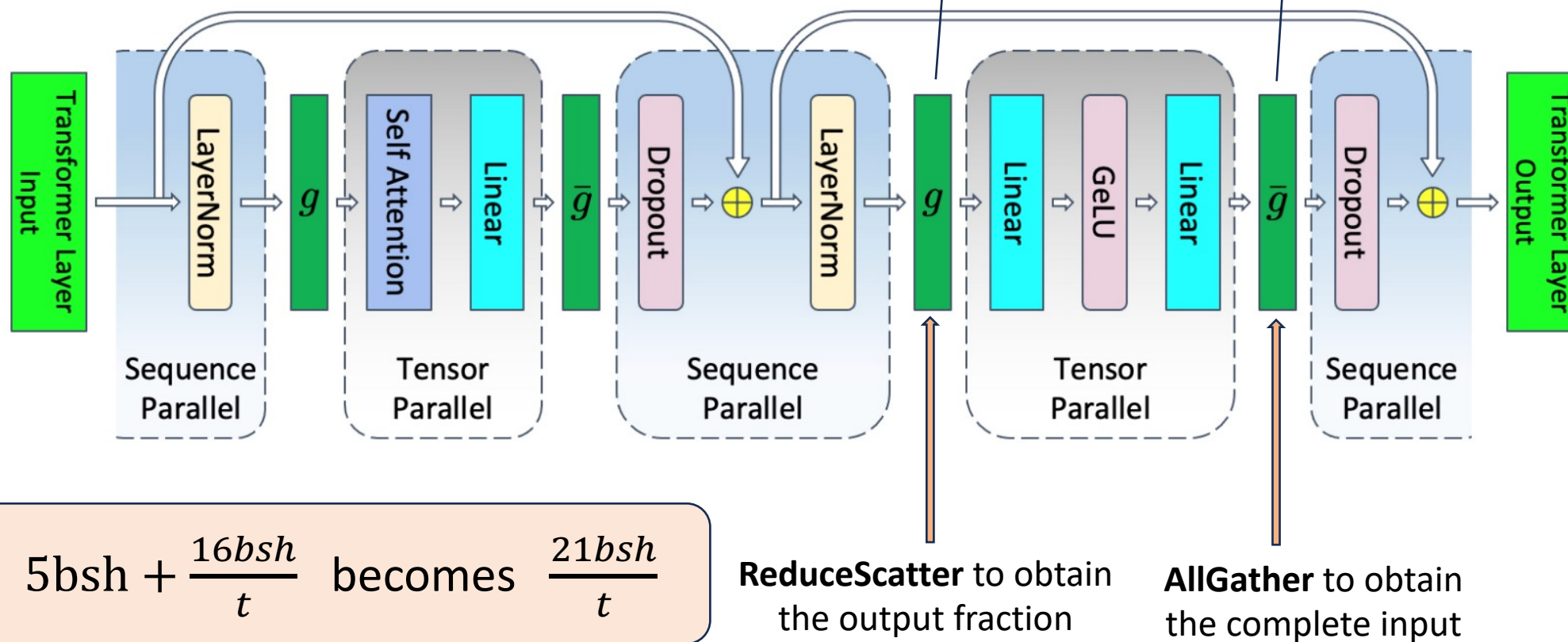
Sequence Parallelism

- Megatron-LM sequence parallelism
 - MLP layer (forward pass)



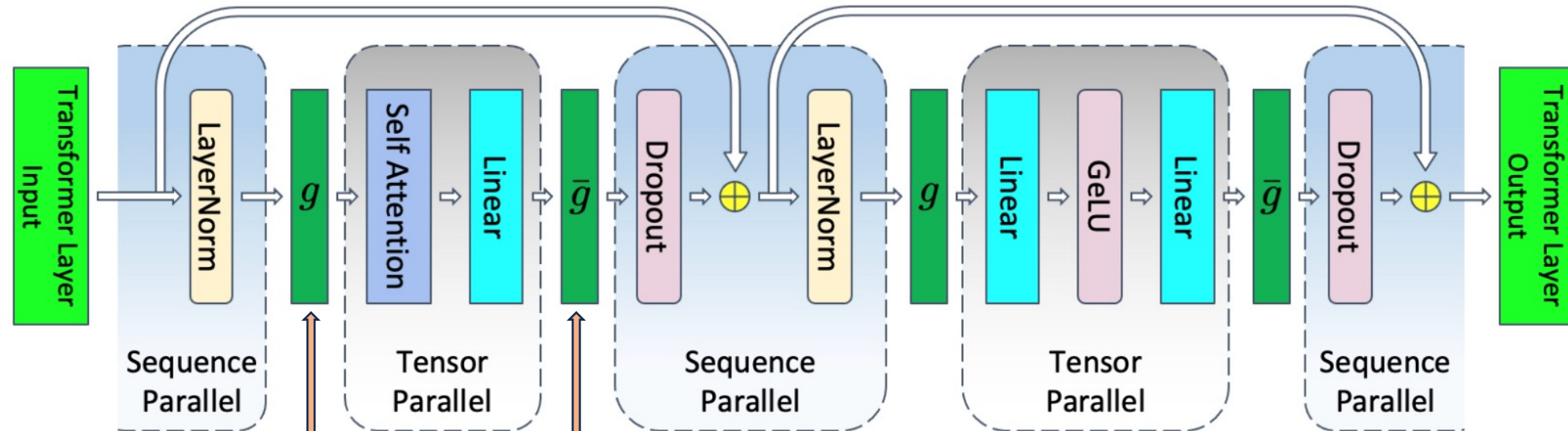
Sequence Parallelism

- Megatron-LM sequence parallelism
 - MLP layer (backward pass)



Sequence Parallelism

- Megatron-LM sequence parallelism
 - Attention (forward pass, backward pass omitted)



AllGather to obtain
the complete input

ReduceScatter to obtain
the output fraction

Finally: $5bsh + \frac{5bass+8bsh}{t}$ becomes $\frac{5bass+13bsh}{t}$

Sequence Parallelism

- Megatron-LM sequence parallelism
 - Slicing input/output of layernorm, and input of dropout
 - Changing AllReduce into ReduceScatter and AllGather
 - Almost **ZERO** extra communication overhead
 - Changing their memory consumption from $10bsh$ to $\frac{10bsh}{t}$
- An example: batch_size = 64, hidden_size = 4096, seq_len = 32768, sp = 8
 - Memory consumption per-block changes from 86 GB to 10.7 GB

Sequence Parallelism

- Megatron-LM sequence parallelism
 - Further optimization: selective recomputation

Low-hanging Fruits: Recomputing large size but less computation intensive activations, e.g. softmax, dot product, dropout, while keeping computation heavy intensive activations, e.g. layernorm, Transformer input/output.

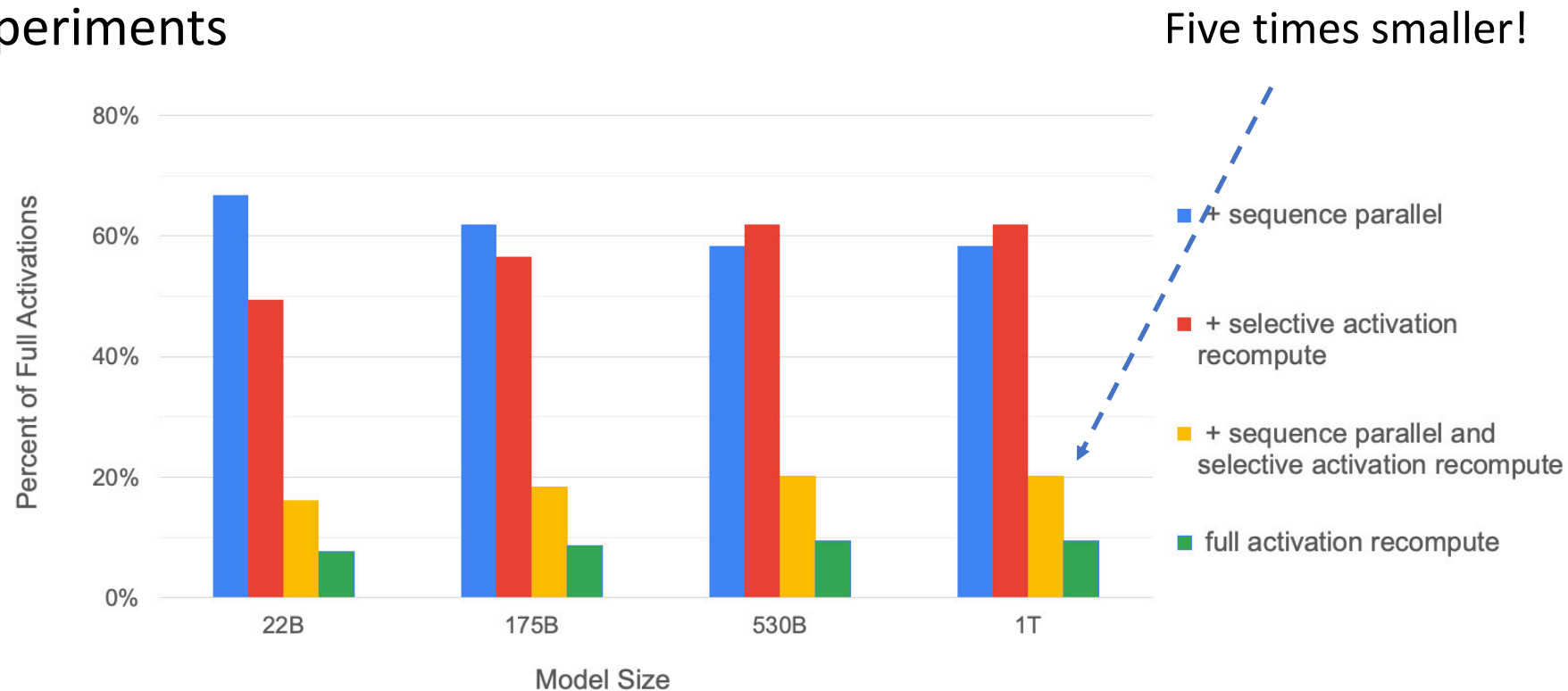
Sequence Parallelism

- Megatron-LM sequence parallelism
 - Setup

Model Size	Attention Heads	Hidden Size	Layers	Tensor Parallel Size	Pipeline Parallel Size	Number of GPUs	Global Batch Size	Micro Batch Size
22B	64	6144	48	8	1	8	4	4
175B (GPT-3)	96	12288	96	8	8	64	64	1
530B (MT-NLG)	128	20480	105	8	35	280	280	1
1T	160	25600	128	8	64	512	512	1

Sequence Parallelism

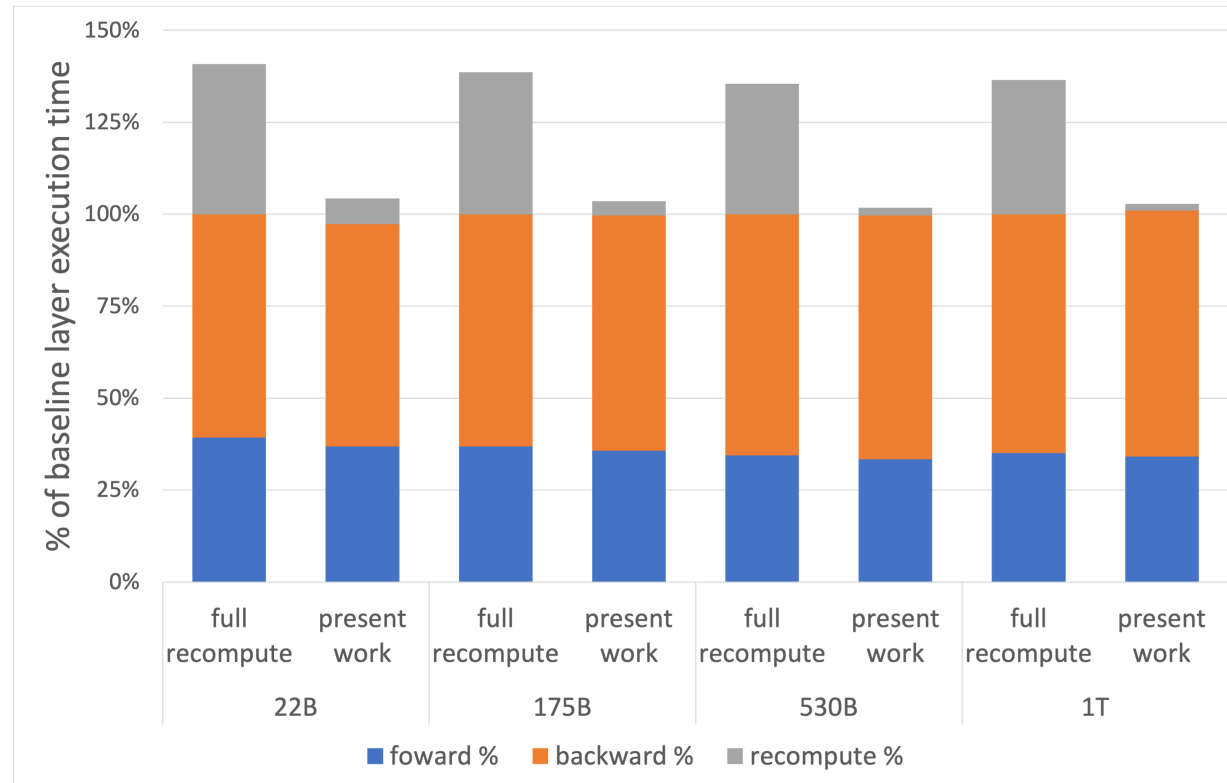
- Megatron-LM sequence parallelism
 - Experiments



Percentage of required memory compared to the tensor-level parallel baseline

Sequence Parallelism

- Megatron-LM sequence parallelism
 - Experiments



Per layer breakdown of forward, backward, and recompute times

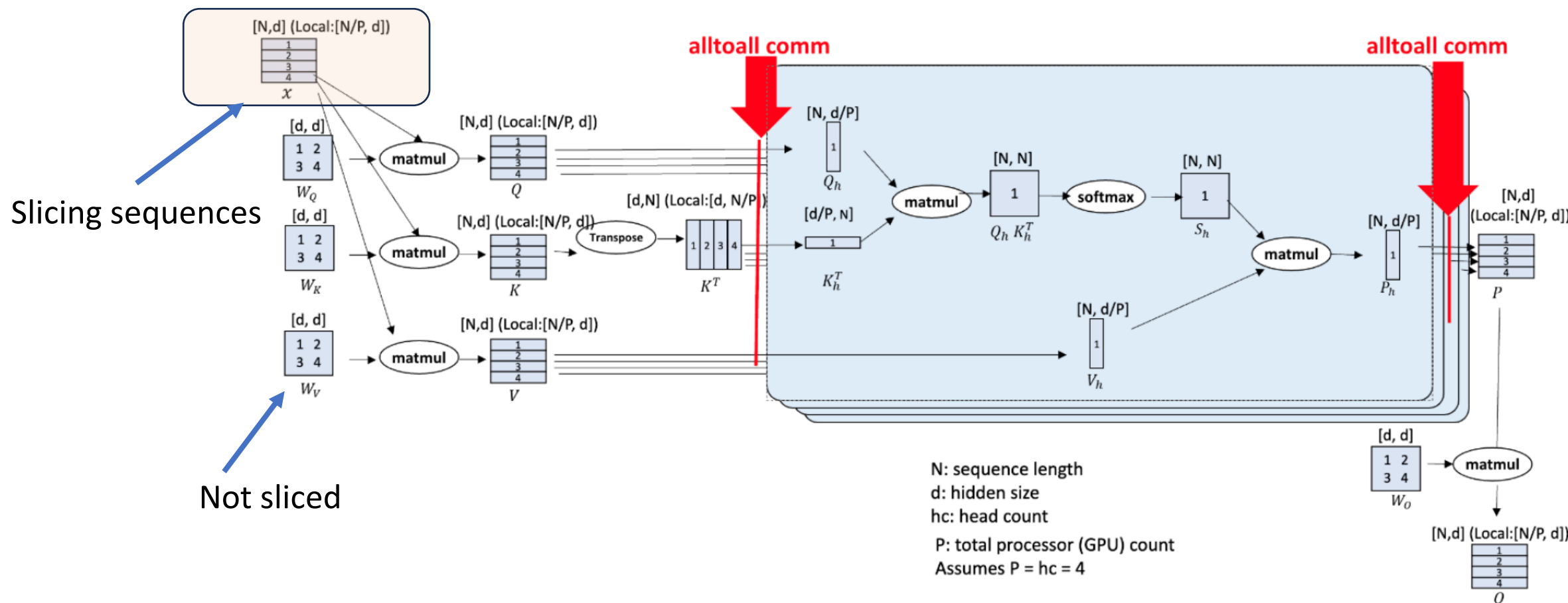
Sequence Parallelism

- DeepSpeed Sequence Parallelism
 - Megatron SP in fact slice activations at the token (sequence) dimension

How about slicing sequences in the very beginning?

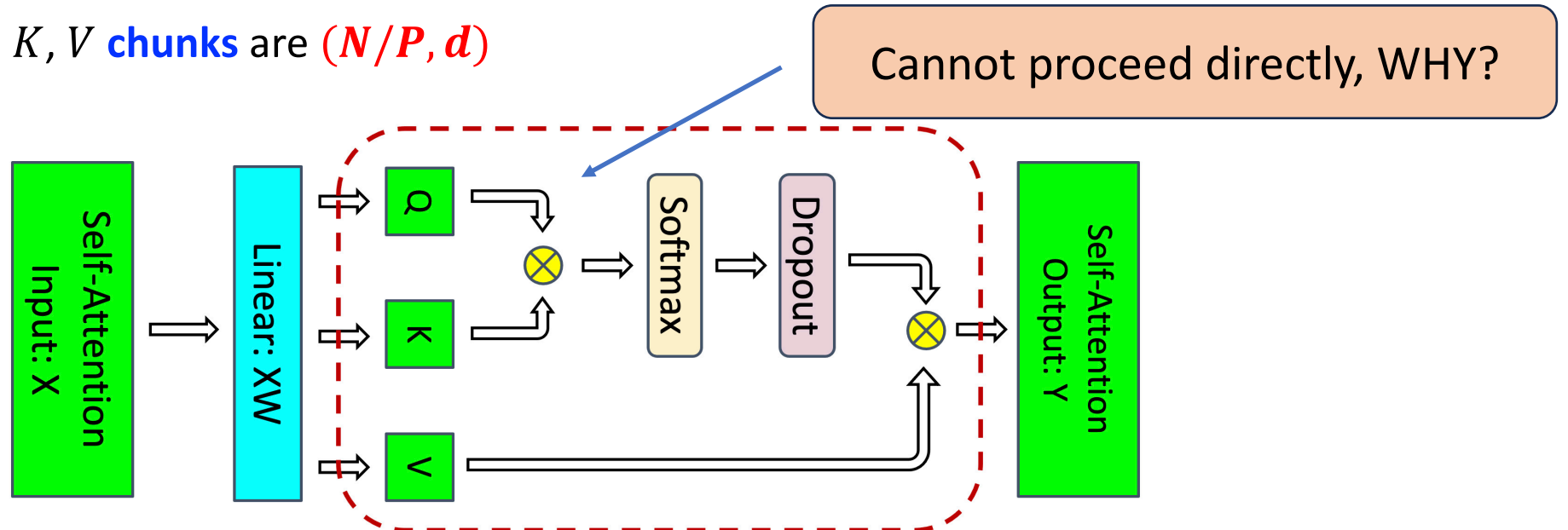
Sequence Parallelism

- DeepSpeed Ulysses Sequence Parallelism



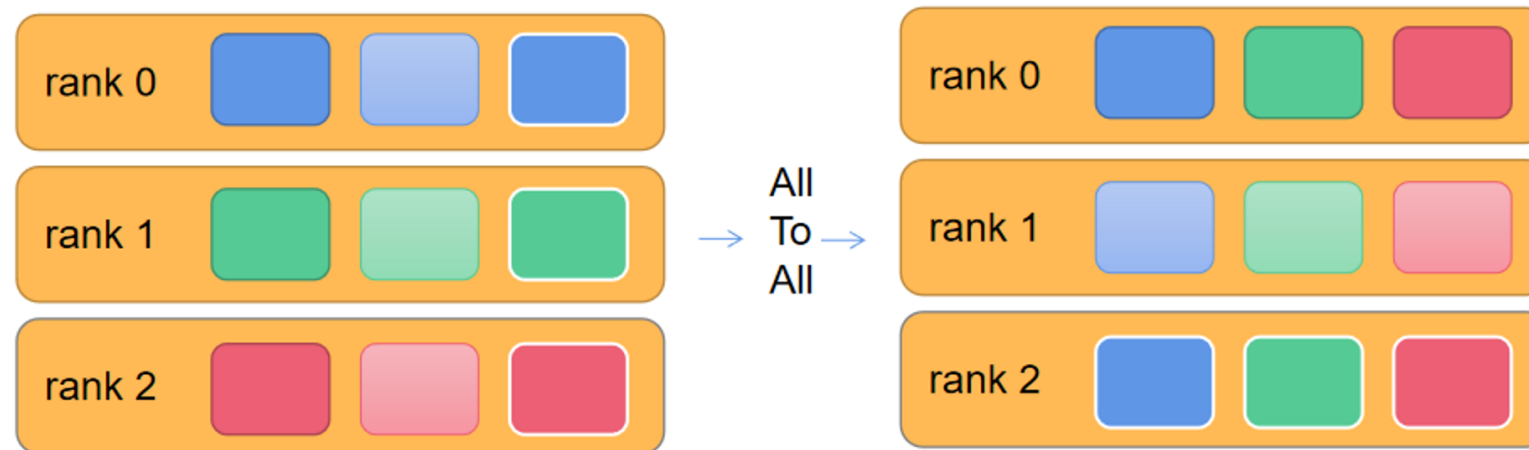
Sequence Parallelism

- DeepSpeed Ulysses Sequence Parallelism
 - Slicing input $X=(N, d)$, with N for sequence length and d for hidden dimension
 - Each GPU has an input $(N/P, d)$, with d for # of GPUs
 - Attention weight matrices W_Q, W_K and $W_V \subseteq \mathbb{R}^{d \times d}$ not partitioned
 - Sizes of Q, K, V **chunks** are $(N/P, d)$



Sequence Parallelism

- DeepSpeed Ulysses Sequence Parallelism
 - All-to-All communication
 - Before all-to-all, each GPU holds the SEQUENCE CHUNK of **ALL HEADs** (i.e. $(N/P, d)$)
 - After all-to-all, each GPU holds ALL SEQUENCES on a specific HEAD (i.e. $(N, d/P)$)

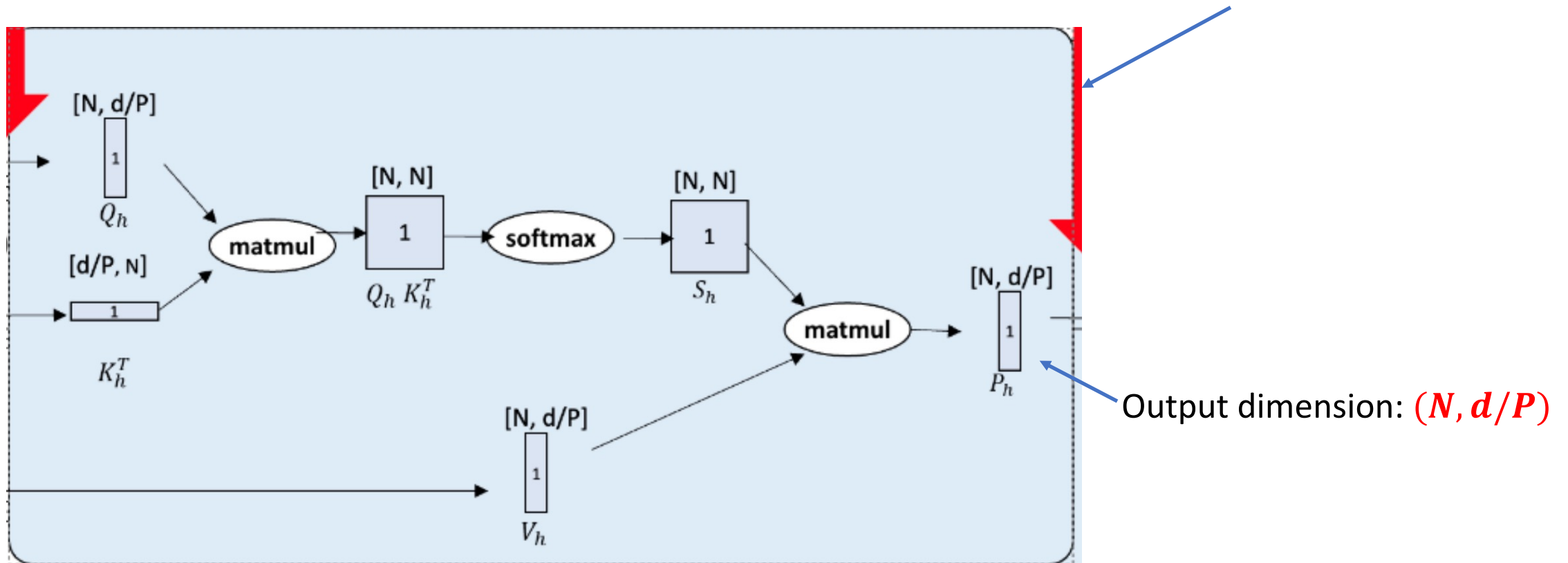


Looks like a transpose operation!

Sequence Parallelism

- DeepSpeed Ulysses Sequence Parallelism
 - All-to-All communication

Multi-head Attention Computation

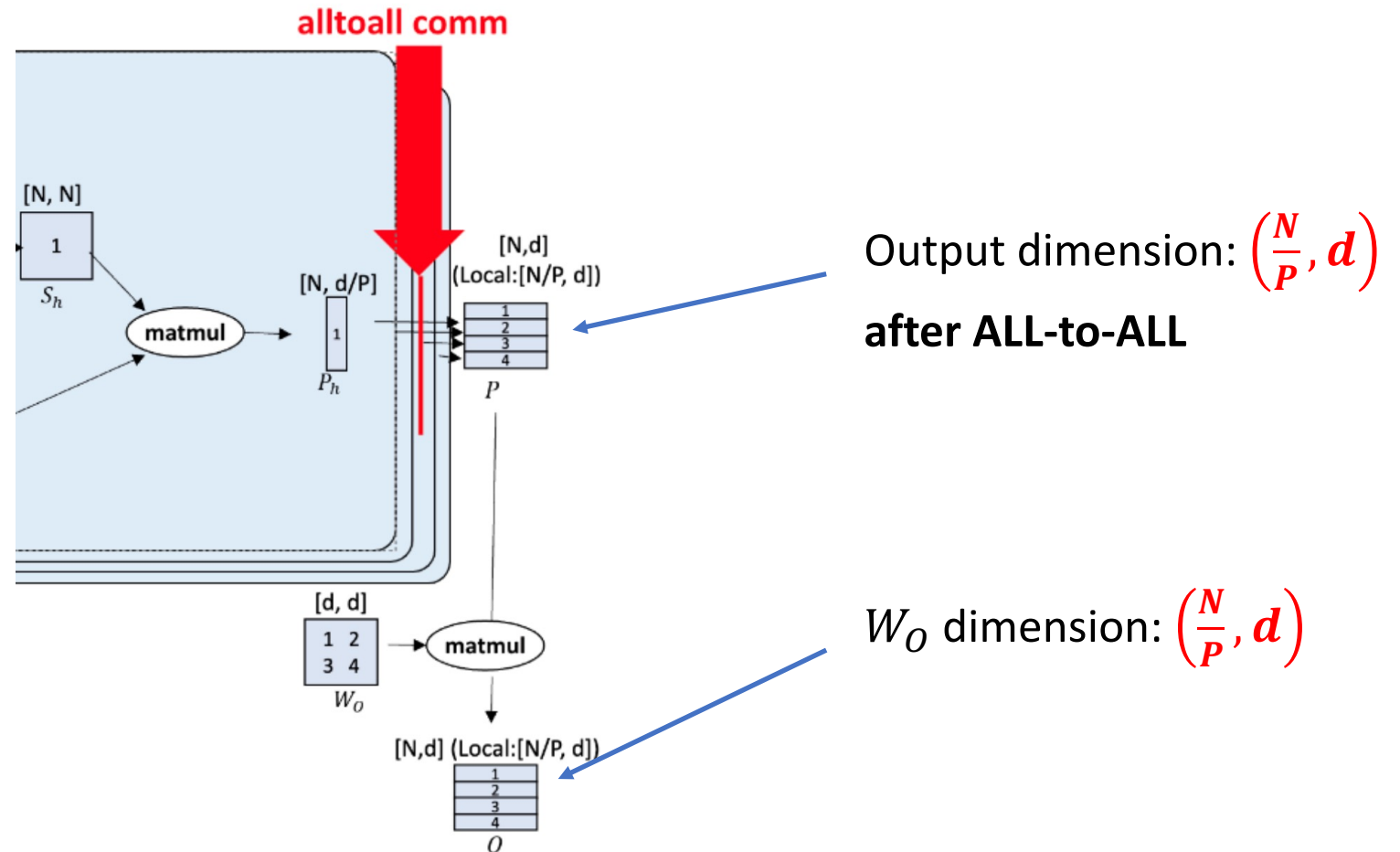


Sequence Parallelism

- DeepSpeed Ulysses Sequence Parallelism

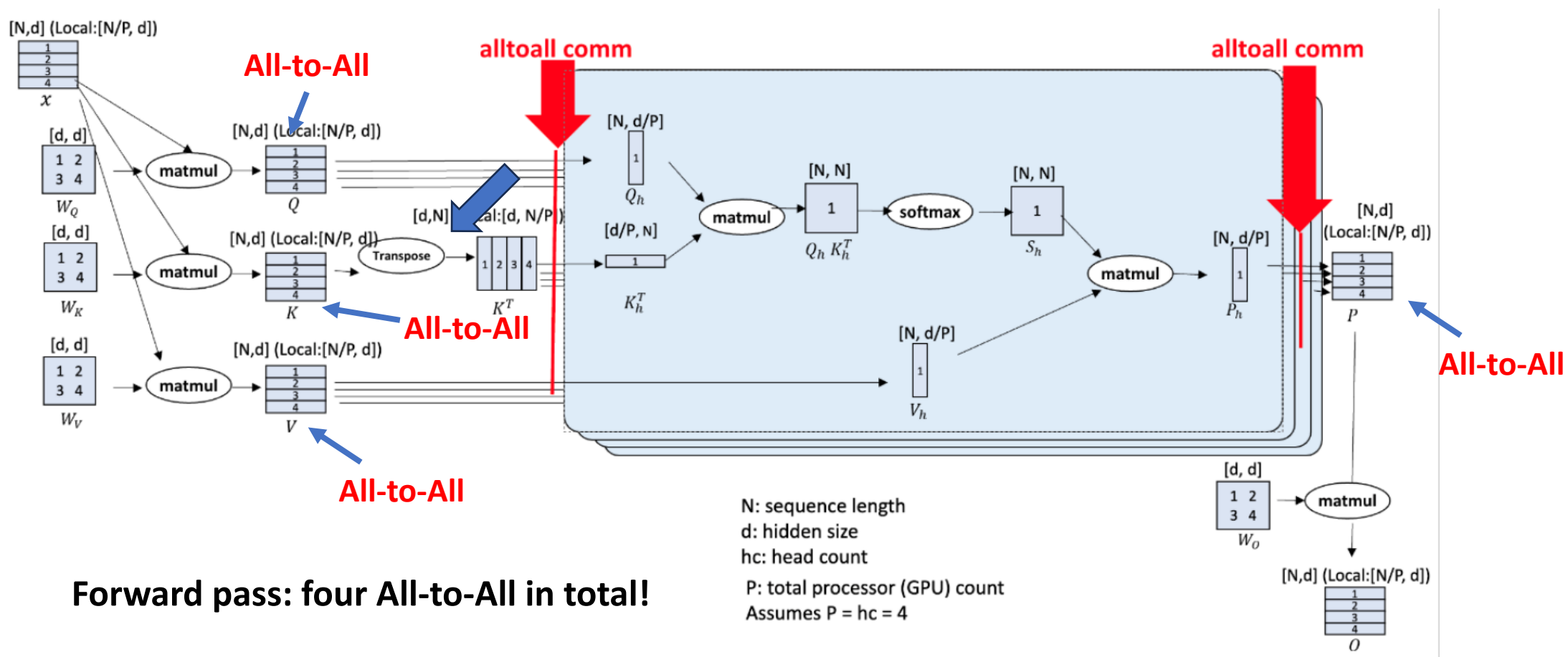
- MLP can compute at each sequence chunk granularity

- DeepSpeed Ulysses backward pass



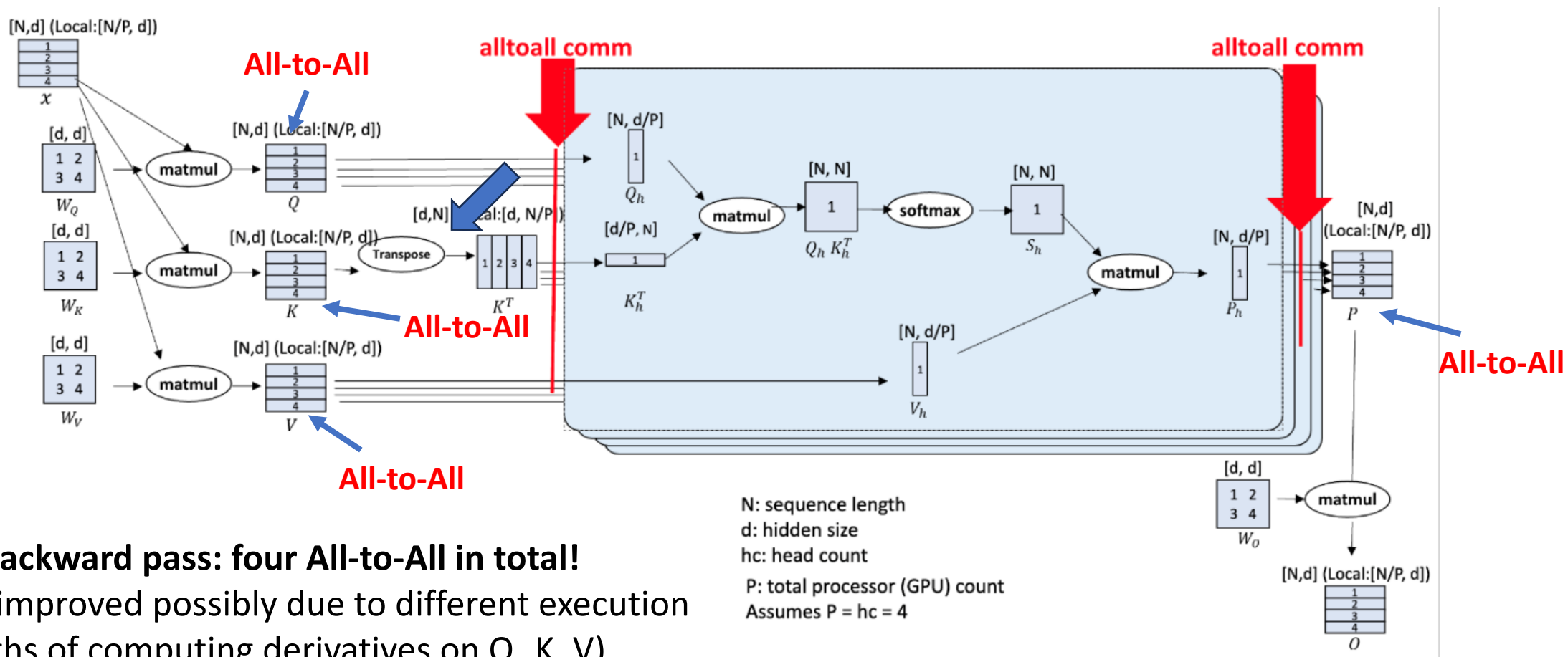
Sequence Parallelism

- DeepSpeed Ulysses Sequence Parallelism: Communication Analysis



Sequence Parallelism

- DeepSpeed Ulysses Sequence Parallelism: Communication Analysis



Backward pass: four All-to-All in total!

(Can be improved possibly due to different execution paths of computing derivatives on Q , K , V)

Sequence Parallelism

- DeepSpeed Ulysses versus NVIDIA Megatron
 - Megatron SP **splits attention and MLP layer parameters** by using tensor parallelism, and splits layernorm and dropout inputs accordingly. The computations are carried out on the heads at different GPUs.
 - Ulysses splits attention and MLP by segmenting the input sequences, and the computations are carried out on the heads at different GPUs. **No segmentation** on the attention layer parameters.

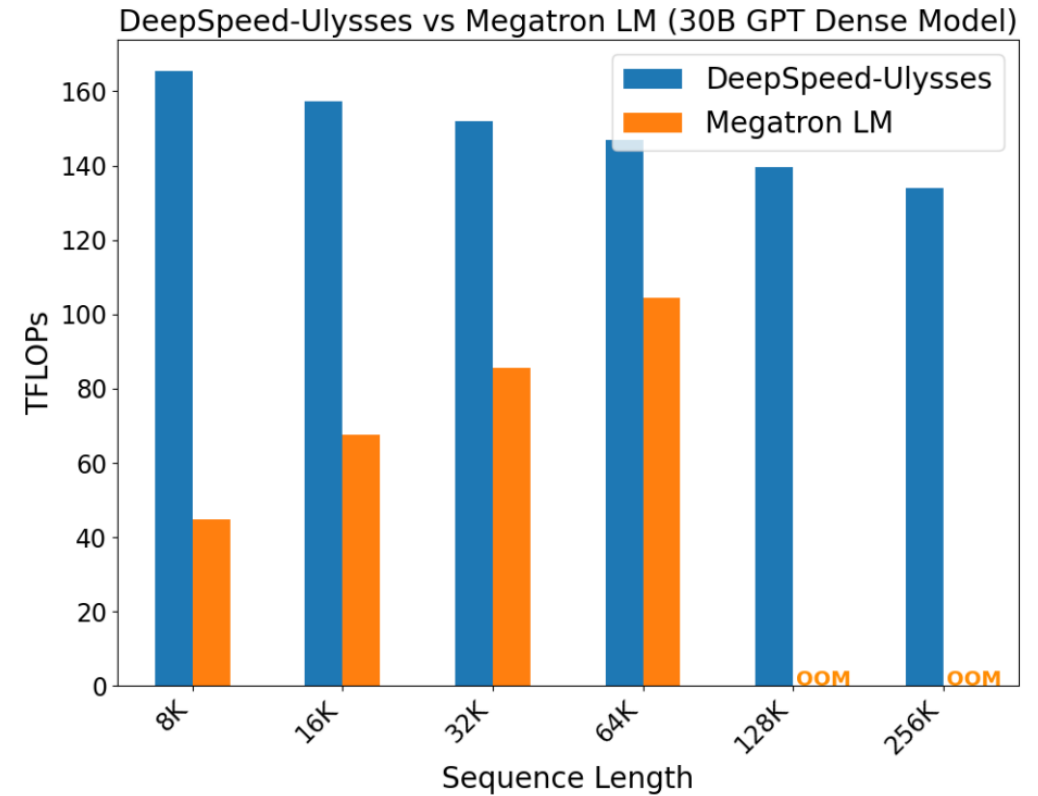
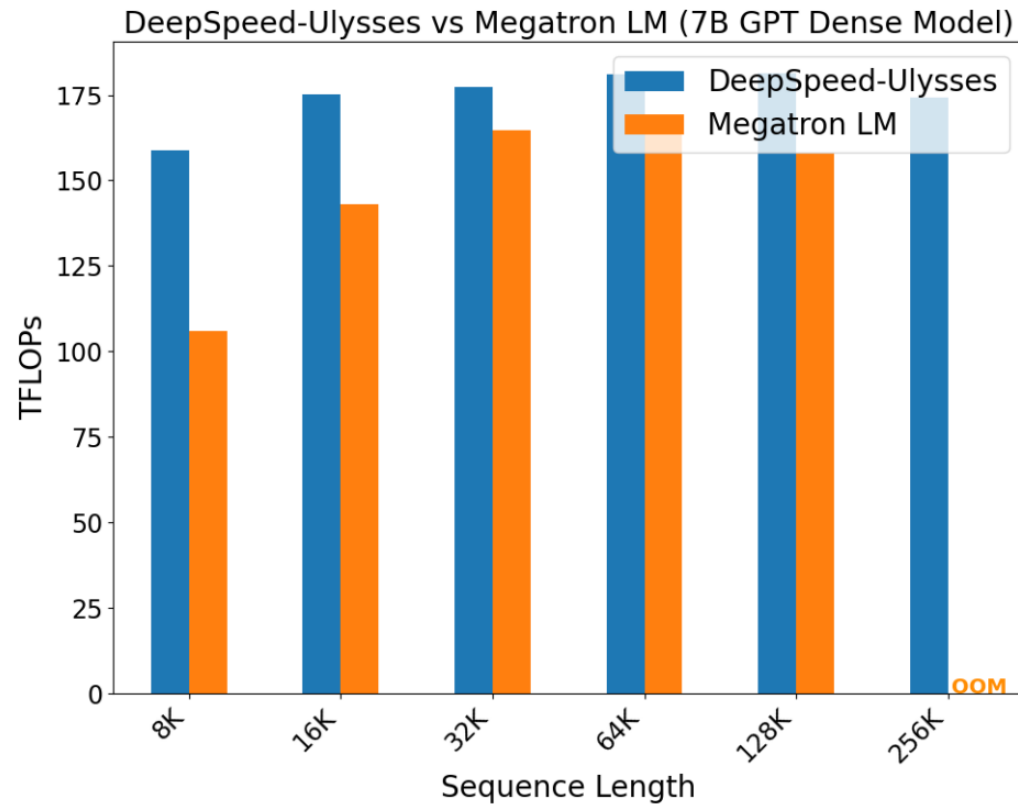
Sequence Parallelism

- DeepSpeed Ulysses versus NVIDIA Megatron

	Attention	MLP
Megatron-LM (combined with TP)	$4 N d$ Forward: ReduceScatter + AllGather, Backward: ReduceScatter + AllGather	$4 N d$ Forward: ReduceScatter + AllGather, Backward: ReduceScatter + AllGather
DeepSpeed Ulysses	$8 N d/P$ Forward: 4 All-to-All, Backward: 4 All-to-All	0 MLP is element-wise, naturally fitting SP

Sequence Parallelism

- DeepSpeed Ulysses versus NVIDIA Megatron



Sequence Parallelism



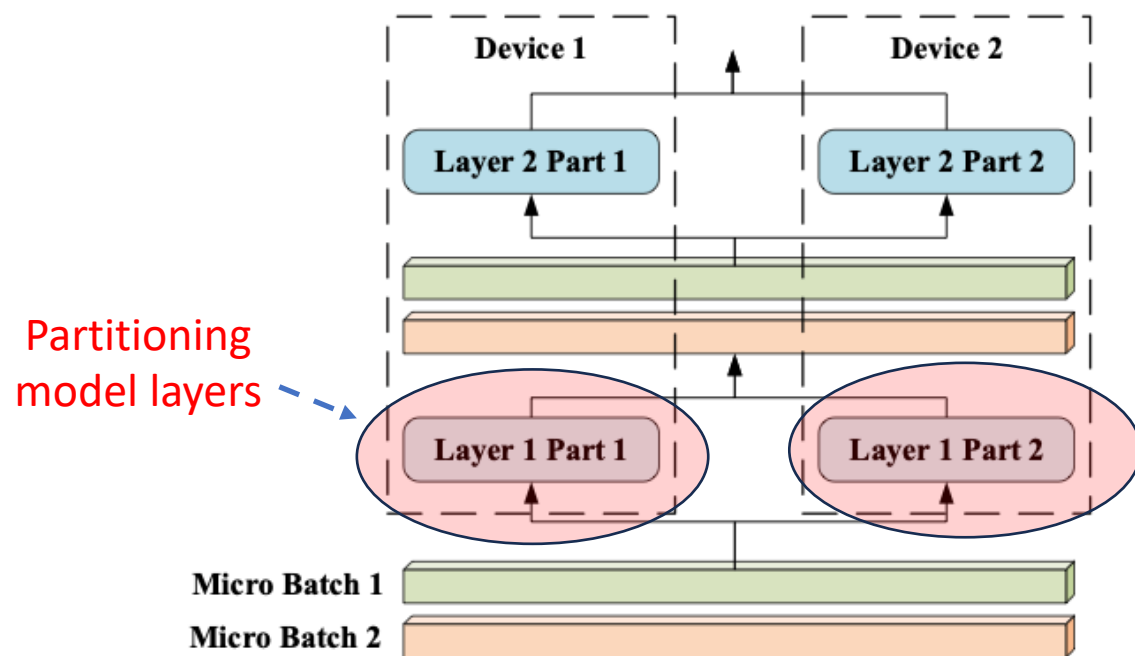
- DeepSpeed Ulysses versus NVIDIA Megatron
 - $N = 1\text{M}$ tokens, $d = 4096$, $P = 128$, FP16
 - Megatron: $8 \times N \times d = 8 \times 1\text{e}6 \times 4096 \times 2 = 65.536 \text{ GB}$
 - Ulysses: $8 \times N \times d / P = 65.536 \text{ GB} / 128 = 0.512 \text{ GB}$

“Ulysses reduces communication volume by eliminating All-Gather in MLP and using All-to-All only in attention, achieving near-linear scaling for sequence lengths up to 1M.”

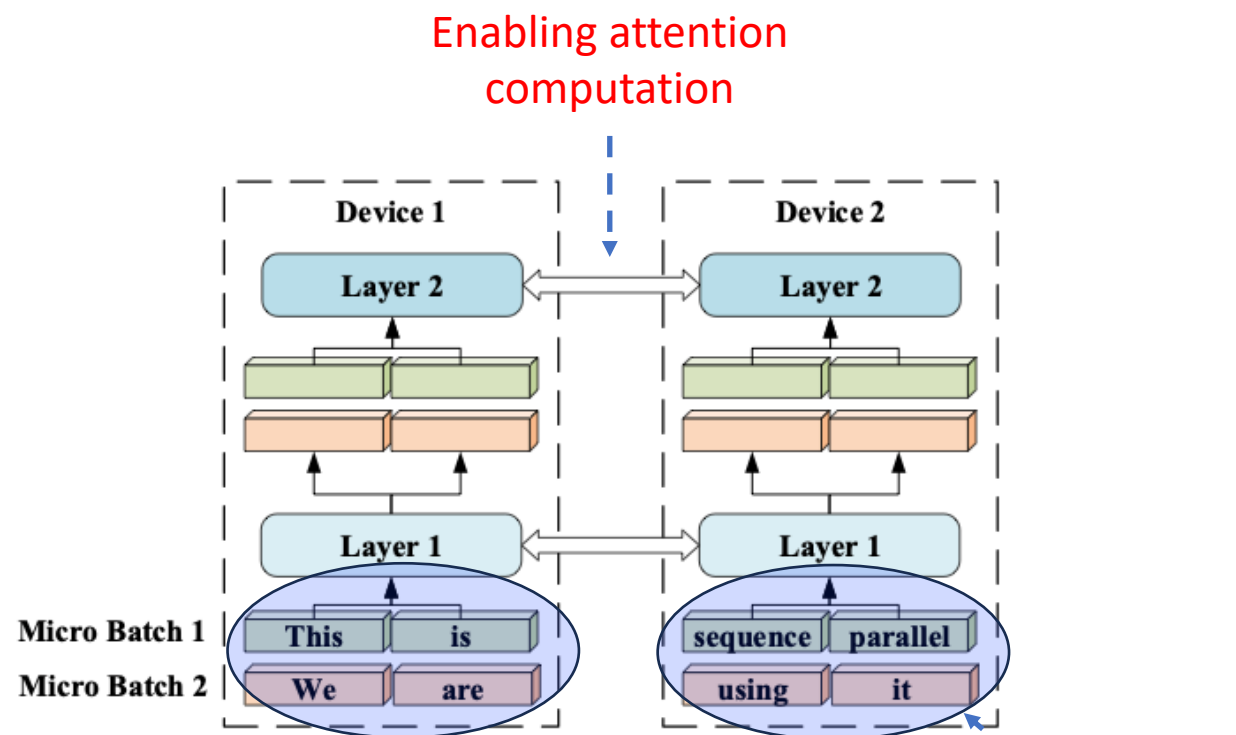
“Megatron-LM sequence parallelism incurs a communication volume per link of $4Nh$ which is P times larger than that for DeepSpeed sequence parallelism.”

Sequence Parallelism

- Colossal AI Sequence Parallelism



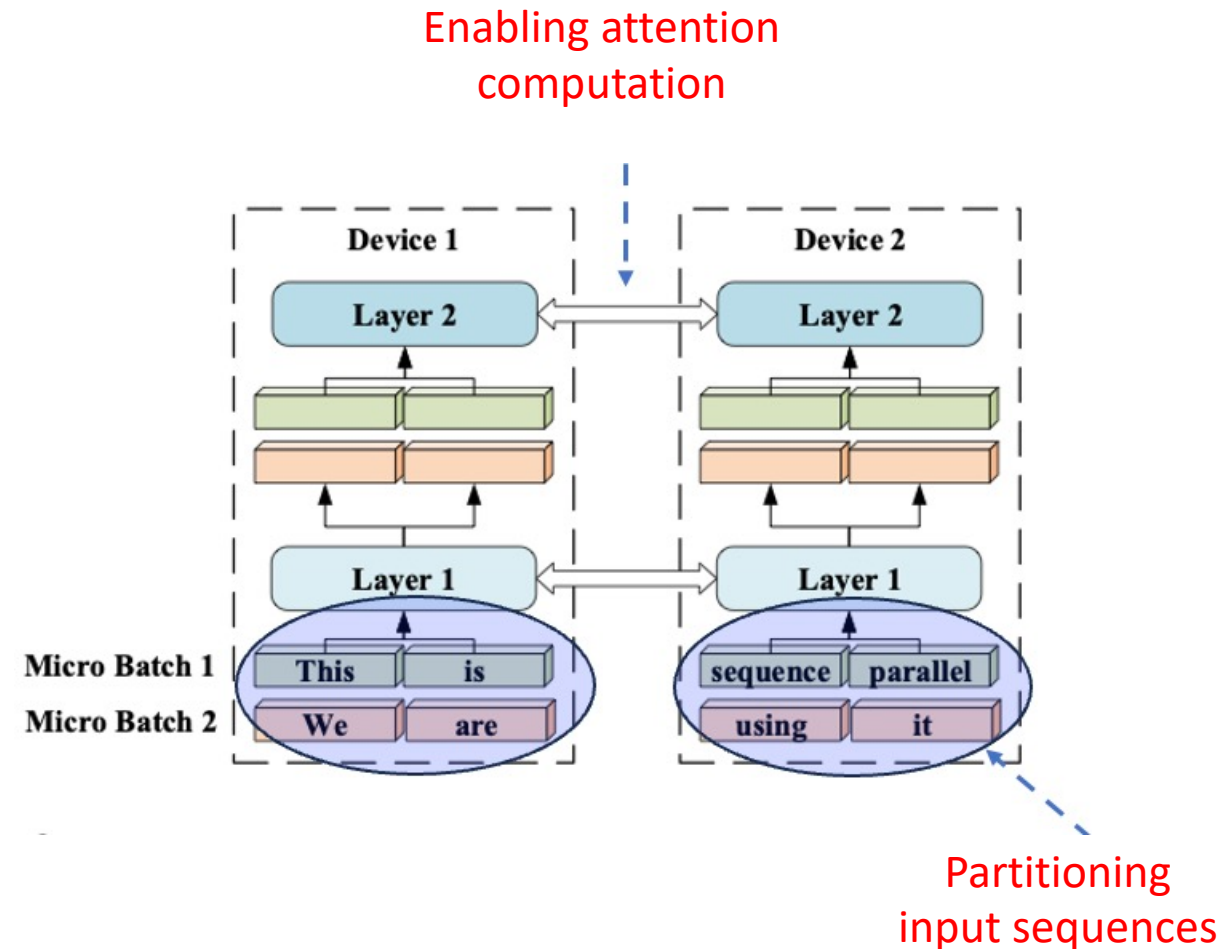
Tensor parallelism



Sequence parallelism

Sequence Parallelism

- Colossal AI Sequence Parallelism
 - Q, K, V partitioned on the sequence
 - Computing attention scores need full Q, K, V on all tokens, in which communication is inevitable
- **Ring Attention** adopted in Colossal-AI



Sequence Parallelism

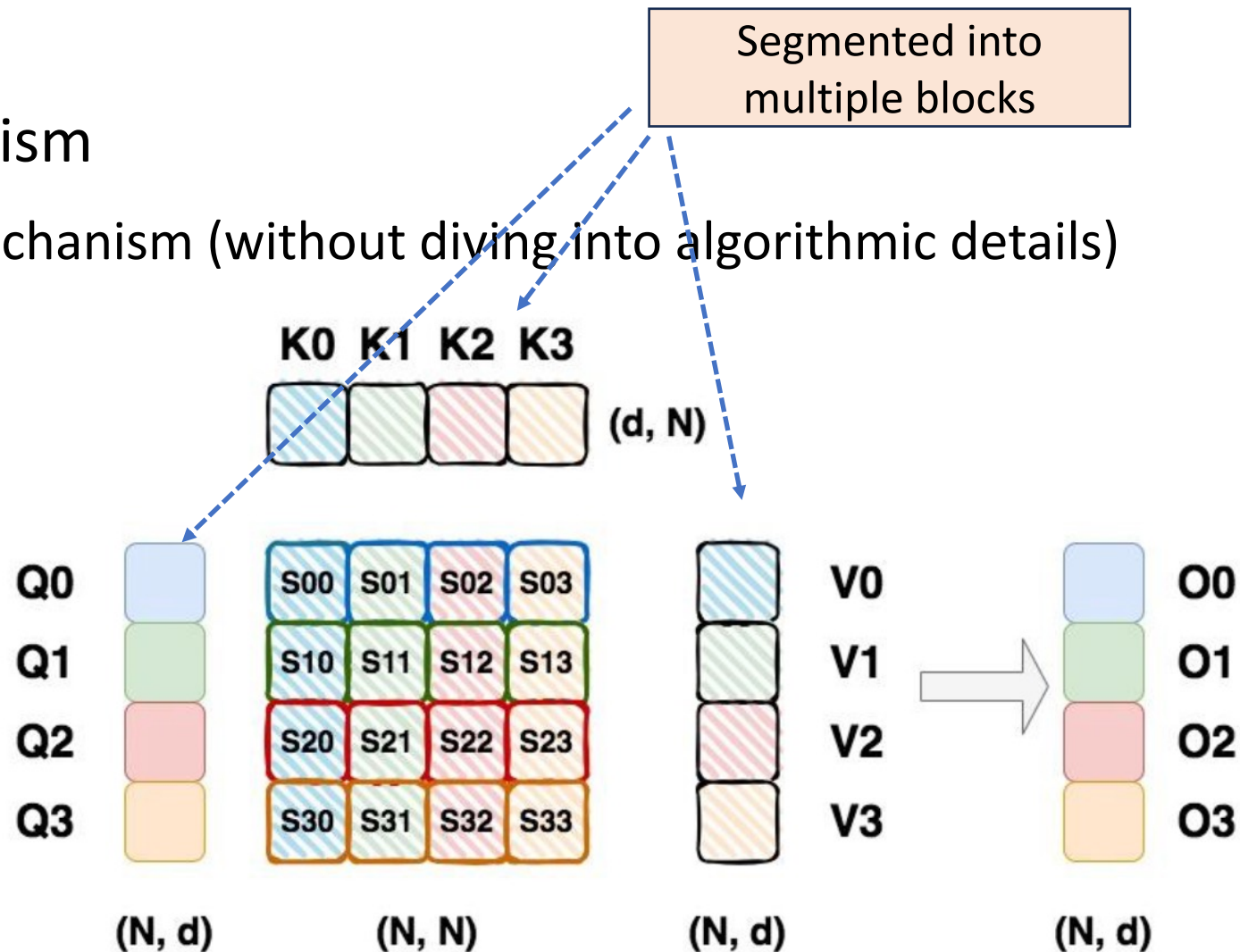
- Colossal AI Sequence Parallelism

- Convoluted with Attention mechanism (without diving into algorithmic details)

Step 1: for each Q_i , compute attention score by feeding K_j and V_j , and obtain O_{ij}

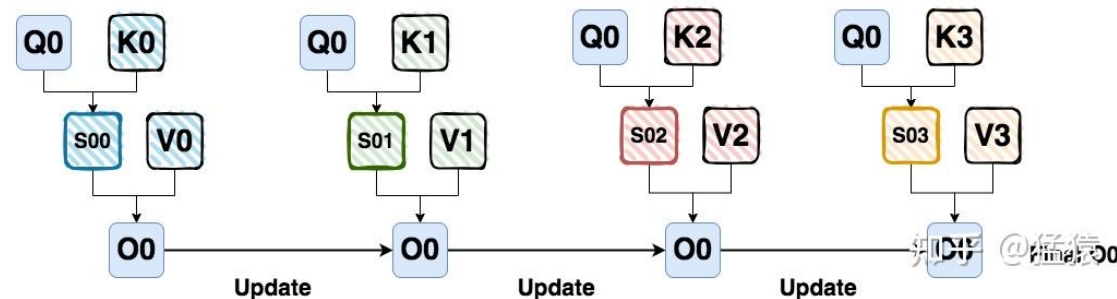
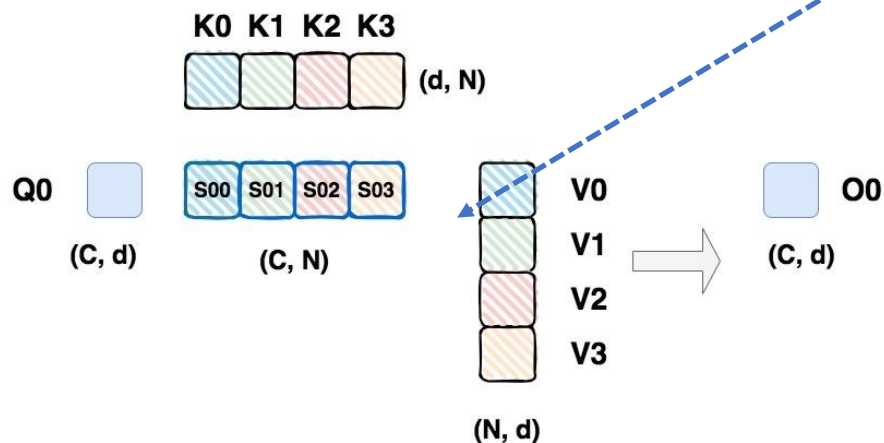
Step 2: for each K_j and V_j , repeat Step 1 and update O_{ij} , and eventually obtain O_i

Step 3: change Q_i to Q_{i+1} , repeat Step 1 and Step 2 until Q_C

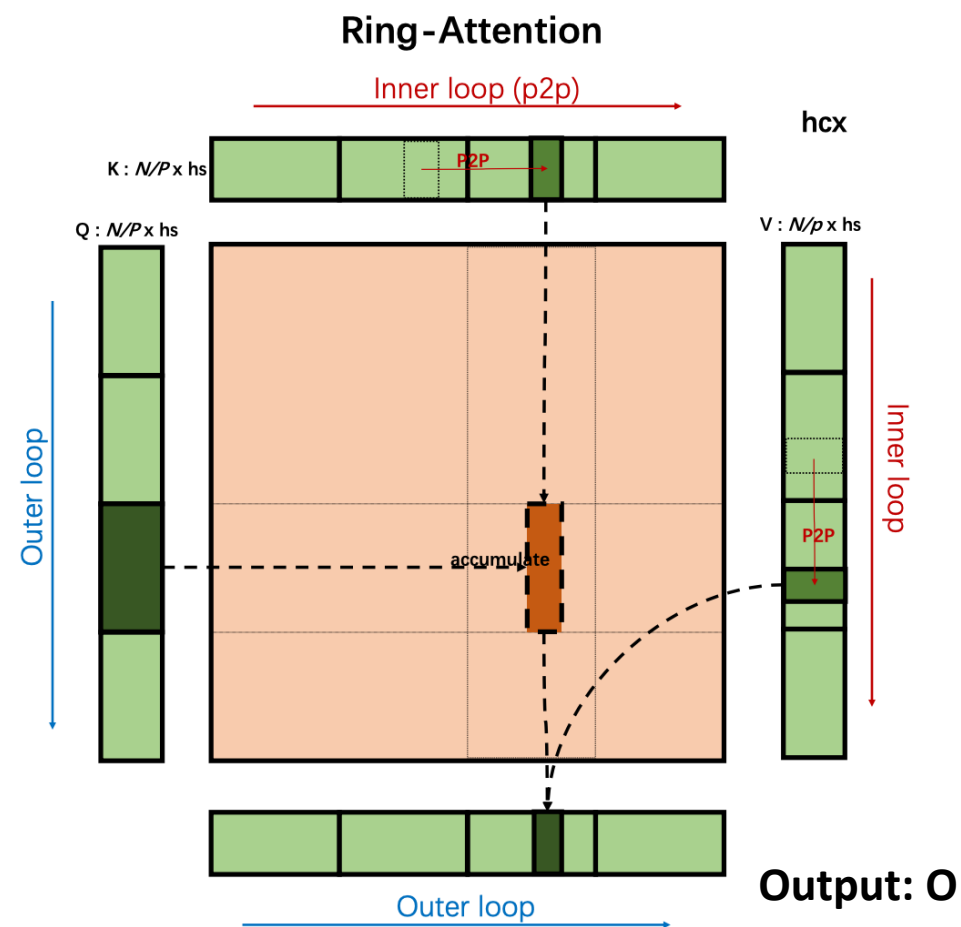


Sequence Parallelism

- Colossal AI Sequence Parallelism

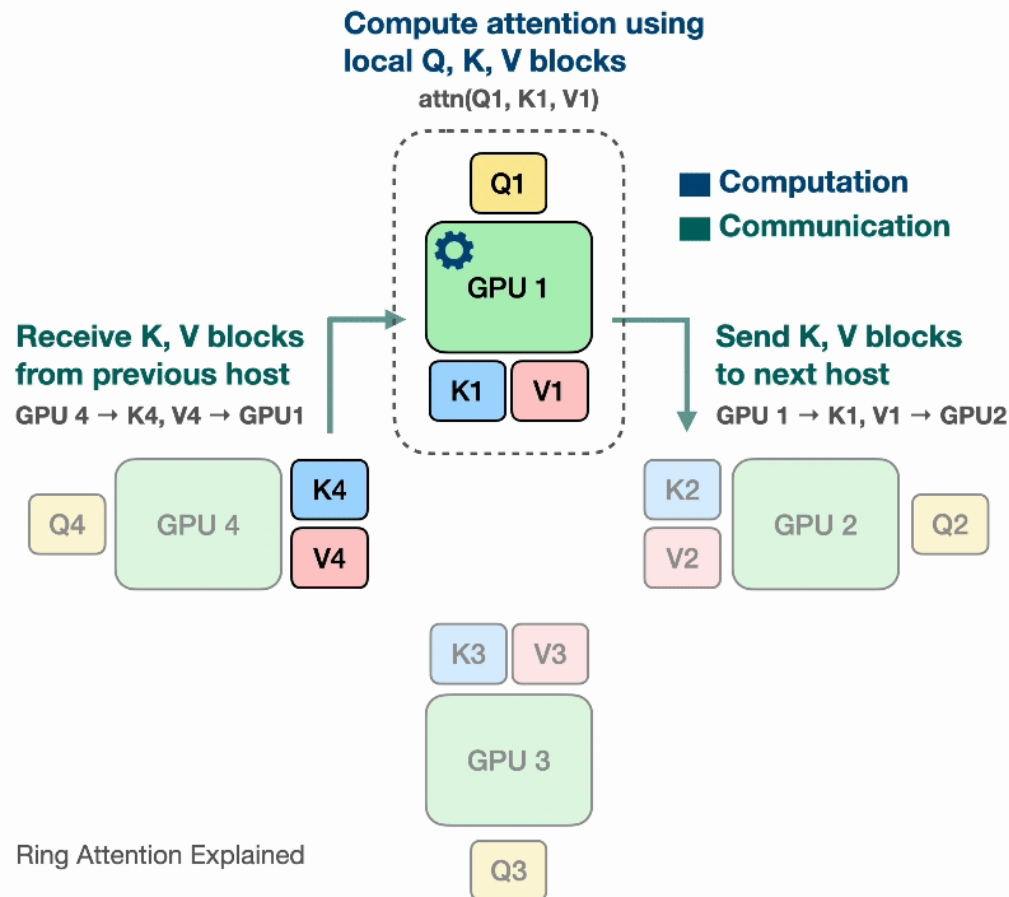


Mathematical deduction shows that S_{00}, S_{01}, S_{02} , and S_{03} can be computed in an **arbitrary order**!



Sequence Parallelism

- Colossal AI Sequence Parallelism



Step 1: GPU1 computes $O1$ using $Q1$, $K1$ and $V1$ (other GPUs do the same thing)

Step 2: GPU1 sends $K1$ and $V1$ to GPU2, and receives $K4$ and $V4$ from GPU4; compute and update $O1$ using $Q1$, $K4$ and $V4$

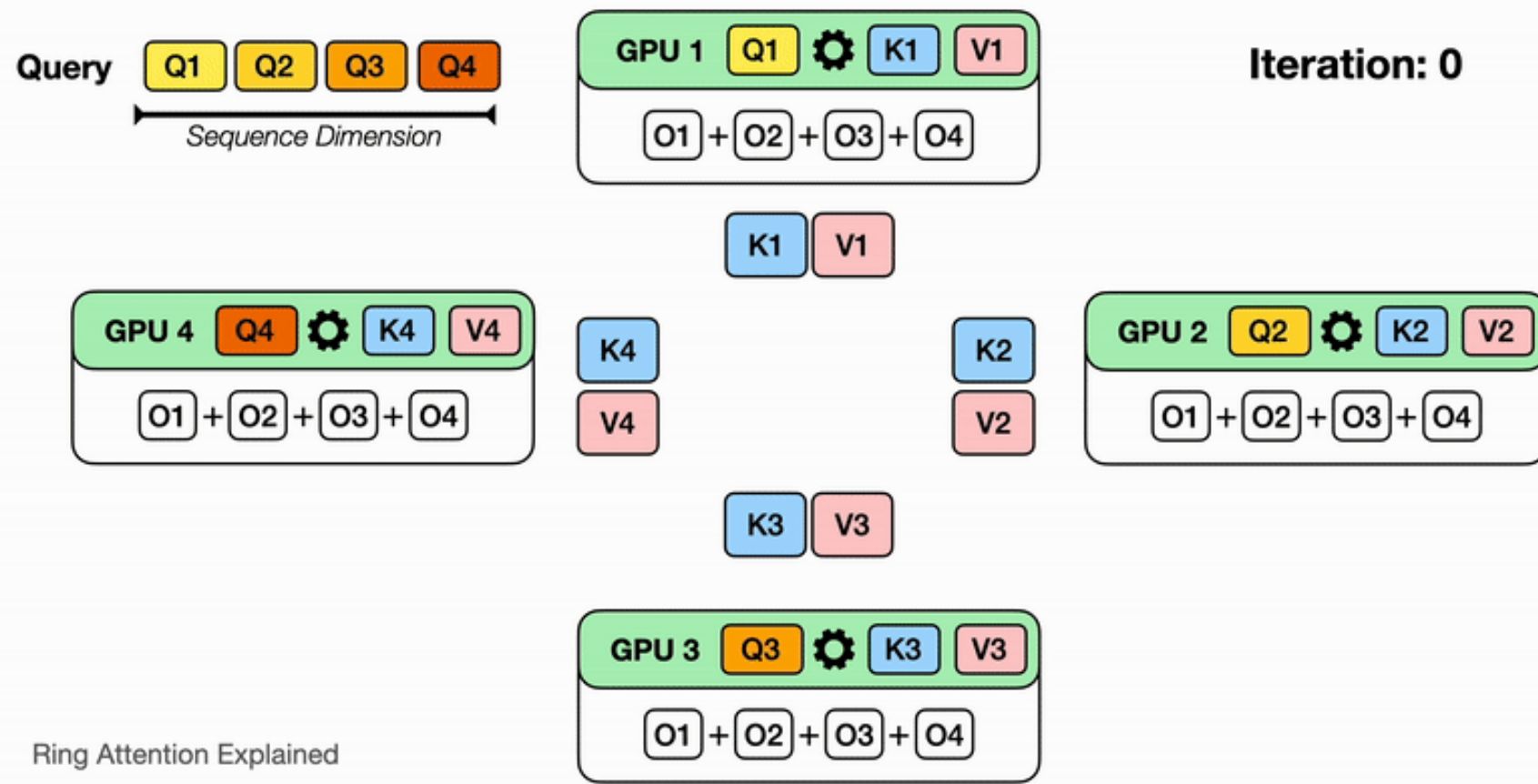
Step 3: GPU1 sends $K4$ and $V4$ to GPU2, and receives $K3$ and $V3$ from GPU4; compute and update $O1$ using $Q1$, $K3$ and $V3$

Step 4: GPU1 sends $K3$ and $V3$ to GPU2, and receives $K2$ and $V2$ from GPU4; compute and update $O1$ using $Q1$, $K2$ and $V2$

The network communications are illustrated by the blinking arrows.

From <https://coconut-mode.com/posts/ring-attention/>

Sequence Parallelism



Illustrate for N devices, it will take N iterations to finish the whole output computation. Watch for each iteration on each device, different partial sum of the output is computed with the K , V block it currently has, and it eventually sees all the K , V blocks and has all the partial sum for the output!

Sequence Parallelism

- Colossal AI Sequence Parallelism

- Memory Complexity

- Storing current K and V blocks (in FP16) needs $2dc$ floats or $4dc$ Bytes
 - Receiving new K and V blocks needs $2dc$ floats or $4dc$ Bytes
 - Storing Q_i block needs $2dc$ Bytes
 - Computing the output needs at least $2dc$ Bytes
 - In total at least $12dc$ Bytes, where c is the chunk size

- Communication Complexity

- Each step sends $2dc$ floats or $4dc$ Bytes, and the total number of rotations is around $\frac{S}{c}$
 - At each inner loop a GPU sends $4sd$ Bytes, and the outer loop has p rounds
 - Total communication load is around $O(sdp)$, which is much higher than Megatron and Ulysses

Thanks!

